INGENIERÍA DE DATOS

Ingeniería de Datos Causales

Pipelines, contratos y observabilidad que preservan la identificación

Jaime Lincovil

Ingeniería de datos, analytics y plataformas de ML

4 partes · 15 capítulos · 10 contratos numerados

2026

Promesa. Un pipeline puede entregar datos limpios, puntuales y versionados, y aun así destruir el enunciado causal que justifica la decisión. Este libro enseña el plano de datos que no lo permite: contratos con roles, versionado con linaje, una pared entre lo observado y lo reservado, y supuestos que se vigilan en producción.

Repositorio de referencia: `causalops` · benchmark `data_benchmark_causal` · casos canónicos con semilla
Edición hermana: la edición de método de este mismo libro (33 capítulos) conserva la teoría completa de identificación y estimación

Ingeniería de Datos Causales

Jaime Lincovil

Primera edición · 2026

© 2026 Jaime Lincovil
Todos los derechos reservados.

Primera edición, 2026.

Queda prohibida la reproducción total o parcial de esta obra por cualquier medio o procedimiento (incluidos la reprografía, el tratamiento informático y la distribución de ejemplares mediante alquiler o préstamo públicos) sin la autorización previa y por escrito del titular de los derechos.

Compuesto con Typst 0.15.1.

Tipografía: New Computer Modern (serif), Inter (sans-serif), DejaVu Sans Mono (monoespaciada).

Marcas de terceros. Python, scikit-learn, FastAPI, Docker y `numpy` son marcas (registradas o no) de sus respectivos titulares. Su mención en esta obra es meramente descriptiva y no implica patrocinio, respaldo ni afiliación alguna.

Repositorio de referencia del libro (`causalops`, `data_benchmark_causal`), disponible junto a la obra. Los casos canónicos se regeneran con semilla declarada.

Edición hermana: **Sistema de Inteligencia Causal** (edición de método, 33 capítulos), del mismo autor, conserva el tratamiento teórico completo de identificación y estimación.

Prefacio

Este libro está pensado para quienes construyen pipelines: ingenieros de datos, *analytics engineers*, perfiles de plataforma de ML y *data engineers* que ya saben mover datos con calidad, y sospechan —o les fue señalado en una revisión— que en su sistema hay algo más en juego que la calidad. También sirve a analistas y científicos de datos responsables de una estimación que termina en una decisión, y a quienes auditan sistemas de decisión.

La tesis es una sola. **En un sistema de decisión, el trabajo del dato no es llegar con calidad al modelo: es preservar la identificación.** Un pipeline puede ser impecable en todo lo que la ingeniería de datos sabe medir — puntualidad, completitud, consistencia, frescura — y aun así destruir el enunciado causal que justifica la decisión que el sistema toma. La fuga temporal no lanza ningún error; la variable posterior al tratamiento entra por la puerta del «mejor ajuste»; la partición filtrada hace que la evaluación mienta; la tabla que cambia bajo los pies hace que el efecto publicado no se refiera a nada. Ninguna métrica de calidad de datos estándar ve esas fallas, porque ninguna de ellas es un problema de calidad del dato: es un problema de **qué comparación sostiene el dato**.

De esa tesis salen las tres decisiones que organizan el libro.

Primera: el plano de datos y el plano de método son dos planos distintos, y este libro es el del dato. La identificación y la estimación — modelos causales estructurales, doble machine learning, meta-learners, políticas — tienen su propio tratamiento completo en la edición de método de este mismo libro, con sus teoremas y sus demostraciones. Aquí no se repite esa teoría: se responde, para cada pieza del pipeline, **qué le exige el estimador al dato**: qué tasas, qué solapamiento, qué marcas de tiempo, qué partición, qué registro. El capítulo 9 del libro (Parte III) condensa esas exigencias; los capítulos 1 a 8 construyen el plano que las satisface.

Segunda: contratos, no convenciones. Todo lo que este libro exige del plano de datos está escrito como un contrato numerado (C-01, C-02, …) con seis campos: qué operaciones ofrece, qué recibe, qué devuelve, qué debe cumplirse siempre, cómo falla y con qué prueba de aceptación se comprueba. Una convención se recuerda; un contrato se ejecuta. La prueba de aceptación es la unidad de rigor de esta edición: cada contrato tiene una, y el repositorio de referencia del libro la corre sobre datos reales. Lo que no está en un contrato con su prueba, en este libro, es una opinión.

Tercera: la pared y la abstención. Un sistema de decisión honesto declara qué datos puede usar y cuáles no — la pared entre lo observado y lo reservado para evaluación — y sabe no responder: la abstención fuera del soporte es una respuesta de primera clase, no un

fallo. La pared existe porque la alternativa es un sistema que consume su propia evaluación y reporta un mundo que no existe; la abstención existe porque un número sin sustento es peor que ningún número.

Cómo está organizado

El libro tiene cuatro partes. La Parte I instala el vocabulario causal mínimo y sus dos primeros artefactos de ingeniería: el contrato de roles (C-01) y el registro de versiones con linaje. La Parte II recorre el pipeline: particiones, la pared observado/evaluación y la validez temporal de las variables. La Parte III condensa lo que los estimadores — DML, heterogeneidad, políticas — le exigen al dato, en la dosis exacta que un equipo de pipeline necesita para garantizarlo. La Parte IV lleva el plano de datos a operación: puerta causal, servicio con abstención, observabilidad de supuestos y la ficha que viaja con el artefacto.

Cada capítulo abre con un cuadro sinóptico, desarrolla un contrato o un invariante, lo comprueba en un caso canónico con verdad conocida, y cierra respondiendo la misma pregunta, siempre la misma, que es el eje del libro: **¿qué debe ser verdad en los datos para que la afirmación de este capítulo valga, y cómo se comprueba?** Esa pregunta, y su respuesta en una caja verde al final de cada capítulo, es la firma editorial del libro.

Convenciones. La notación sigue la edición de método: $Y(d)$ es el resultado potencial bajo el tratamiento d, D la asignación observada, X las covariables previas al tratamiento, $\tau(x)$ el CATE y θ el ATE. En el texto corrido los decimales se escriben con coma; dentro de las fórmulas, con punto. Cada cifra de un caso canónico cita su script y su semilla, y todos los scripts viven en el repositorio de referencia, junto al benchmark de datos y a `causalops`, la implementación de los contratos. Los términos de ingeniería de datos que el libro usa en sentido técnico —contrato, linaje, inmutabilidad— se definen la primera vez que aparecen y están recogidos en el glosario de la edición de método.

Agradecimientos. Este libro se apoya en dos tradiciones que rara vez se citan juntas: la ingeniería de datos de (Kleppmann 2017) y (Reis y Housley 2022), y la inferencia causal de (Neyman 1923), (Rubin 1974) y (Chernozhukov et al. 2018). Los contratos numerados son deudores del aparato de la edición de método de este mismo libro, y el repositorio de referencia existe porque un curso anterior lo construyó antes de que este libro lo necesitara. El servidor MCP que acompaña esta edición — doce herramientas que hacen los contratos ejecutables por agentes — es la pieza que cierra el ciclo: lo que este libro enseña, un agente conectado ya no puede ignorarlo.

Índice general

Prefacio ... iii
Cómo está organizado ... iv
1 Causalidad elemental para ingeniería de datos ... 15
1.1 Dos preguntas que parecen la misma ... 15
1.2 El estimando es una comparación que el dato debe sostener ... 16
1.3 La reversión de Simpson: la comparación que cambia de signo ... 17
1.4 El problema fundamental de la inferencia causal ... 18
1.5 Ejercicios y notas bibliográficas ... 19
1.5.1 Ejercicios conceptuales ... 19
1.5.2 Notas bibliográficas ... 19
2 Anatomía de un sistema de decisión ... 21
2.1 Dos planos, dos responsabilidades ... 21
2.2 El sistema desplegado es una intervención ... 22
2.3 Las cinco acciones del sistema ... 23
2.4 El esqueleto en el plano de datos ... 24
2.5 Caso canónico: una traza por el sistema ... 25
2.6 Ejercicios y notas bibliográficas ... 26
2.6.1 Ejercicios conceptuales ... 26
2.6.2 Notas bibliográficas ... 26
3 Contratos de datos causales ... 28
3.1 De las convenciones a los contratos ... 28
3.2 El problema que añade el modo causal ... 29
3.3 El contrato C-01: esquema de roles causales ... 30
3.4 La prueba de aceptación sobre datos reales ... 32
3.5 Lo que el contrato no verifica ... 33
3.6 Cómo se diseña un contrato ... 33
3.7 Anti-patrones del C-01 ... 34
3.8 Los contratos y el resto del libro ... 34
3.9 El esquema como artefacto de identificación ... 35
3.10 El diseño del esquema en la práctica ... 35
3.11 El fallo que el esquema previene y el que no ... 36
3.12 Ejercicios y notas bibliográficas ... 36
3.12.1 Ejercicios conceptuales ... 36
3.12.2 Notas bibliográficas ... 37

4 Versionado, linaje e inmutabilidad ... 39
4.1 El problema: la afirmación y su sujeto ... 39
4.2 El hash de contenido: la identidad de un dataset ... 40
4.3 La ficha de procedencia del tratamiento ... 40
4.4 El contrato C-09: registro de versiones y linaje ... 41
4.5 Ejercicios y notas bibliográficas ... 43
4.5.1 Ejercicios conceptuales ... 43
4.5.2 Notas bibliográficas ... 44
5 Particiones y la pared observado/evaluación ... 47
5.1 El problema: evaluar es comparar contra lo intacto ... 47
5.2 La partición como estructura de datos ... 48
5.3 El contrato C-10: partición versionada ... 48
5.4 La pared: lo observado y lo reservado ... 49
5.5 Caso canónico A: la partición es una función de la semilla ... 50
5.6 Caso canónico B: elegir mirando la prueba ... 50
5.7 Ejercicios y notas bibliográficas ... 51
5.7.1 Ejercicios conceptuales ... 51
5.7.2 Notas bibliográficas ... 52
6 Validez temporal y control de fuga ... 53
6.1 El problema: la columna que predice bien ... 53
6.2 Qué significa «antes»: validez temporal y el as-of ... 54
6.3 El contrato C-03: auditoría de features ... 55
6.4 Caso canónico: cuatro tipos de feature y una verdad conocida ... 56
6.5 La regla conservadora y la comparación de especificaciones ... 57
6.6 Ejercicios y notas bibliográficas ... 58
6.6.1 Ejercicios conceptuales ... 58
6.6.2 Notas bibliográficas ... 58
7 Calidad y observabilidad de datos para decisiones ... 60
7.1 El problema: la calidad que importa ... 60
7.2 El chequeo de balance ... 61
7.3 La deriva: la comparación mueve sus dos lados ... 62
7.4 Lo que la comparación no ve: faltantes por selección ... 62
7.5 El contrato C-02, leído del plano de datos ... 63
7.6 De la métrica a la acción ... 65
7.7 Ejercicios y notas bibliográficas ... 65
7.7.1 Ejercicios conceptuales ... 65
7.7.2 Notas bibliográficas ... 66
8 Benchmarks y procesos generadores como producto de datos ... 67
8.1 El problema: sin verdad no hay evidencia ... 67
8.2 Dos familias de datos y dos regímenes de pared ... 68
8.3 Los cinco escenarios, leídos de sus archivos ... 68
8.4 El contrato C-11: DGP y benchmark como producto ... 70

8.5 Ejercicios y notas bibliográficas ... 71
8.5.1 Ejercicios conceptuales ... 71
8.5.2 Notas bibliográficas ... 72
9 Lo que DML le exige al dato ... 75
9.1 Este capítulo es de consumo, no de método ... 75
9.2 Exigencia 1: la partición out-of-sample (cross-fitting) ... 76
9.3 Exigencia 2: soporte común y ESS de Kish ... 77
9.4 Exigencia 3: el nuisance no es decoración ... 77
9.5 El contrato C-12: entrega de exigencias del estimador ... 78
9.6 Qué aprende el plano de datos con esto ... 79
9.7 Cómo diagnosticar una violación ... 79
9.8 El bundle como entregable ... 80
9.9 El trimming como cambio de estimando ... 81
9.10 El fallo que cada exigencia previene y el que no ... 81
9.11 Cuándo sabes que tu DML está mintiendo ... 82
9.12 Ejercicios y notas bibliográficas ... 82
9.12.1 Ejercicios conceptuales ... 82
9.12.2 Notas bibliográficas ... 83
10 Lo que la heterogeneidad y la política le exigen al dato ... 85
10.1 Este capítulo también es de consumo ... 85
10.2 Exigencia 1: CATE como predictor con verdad en el DGP ... 86
10.3 Exigencia 2: el valor de una política se estima, no se asume ... 86
10.4 Contrato C-12 extensión: heterogeneidad y valor de política ... 87
10.5 Ejercicios y notas bibliográficas ... 88
10.5.1 Ejercicios conceptuales ... 88
10.5.2 Notas bibliográficas ... 88
11 Diagnóstico sin verdad ... 89
11.1 Sin verdad, con falsación ... 89
11.2 Placebo y control negativo ... 90
11.3 Sensibilidad: cuánto confusor haría cruzar el cero ... 90
11.4 Contrato C-08: sensibilidad y falsación ... 91
11.5 Ejercicios y notas bibliográficas ... 92
11.5.1 Ejercicios conceptuales ... 92
11.5.2 Notas bibliográficas ... 92
12 La puerta causal en el plano de datos ... 95
12.1 Promover datos, no solo modelos ... 95
12.2 Las tres señales de la puerta ... 96
12.3 El contrato C-06: compuerta de registro ... 96
12.4 Caso canónico: tres versiones, tres estados ... 97
12.5 Ejercicios y notas bibliográficas ... 98
12.5.1 Ejercicios conceptuales ... 98
12.5.2 Notas bibliográficas ... 98

13 Servicio y abstención 99
13.1 El bundle viaja, el dato no 99
13.2 La abstención es una respuesta de primera clase 99
13.3 El contrato C-04: servicio de estimación 100
13.4 Caso canónico: respuesta y abstención 101
13.5 Despliegue: local y nube 102
13.6 Ejercicios y notas bibliográficas 102
13.6.1 Ejercicios conceptuales 102
13.6.2 Notas bibliográficas 102
14 Observabilidad de supuestos en producción 104
14.1 El monitoreo genérico no basta 104
14.2 Las cuatro señales 105
14.3 El contrato C-07: monitoreo causal 106
14.4 Caso canónico: cuatro cambios, cuatro señales 107
14.5 De la señal a la acción 107
14.6 Ejercicios y notas bibliográficas 108
14.6.1 Ejercicios conceptuales 108
14.6.2 Notas bibliográficas 108
15 Tarjetas, registro y auditoría end-to-end 109
15.1 El cierre del libro en una corrida 109
15.2 El contrato C-05: tarjeta causal 110
15.3 Caso canónico: la corrida completa 111
15.4 Qué audita una persona con esta traza 112
15.5 El proceso de auditoría paso a paso 112
15.6 Mantenimiento de la tarjeta 113
15.7 La tarjeta como gobernanza 114
15.8 Qué queda fuera del alcance 114
15.9 Ejercicios y notas bibliográficas 115
15.9.1 Ejercicios conceptuales 115
15.9.2 Notas bibliográficas 115
A Índice temático 116
B Índice de autores 118
C Erratas 120
Bibliografía 121

Índice de figuras

Figura 1.1 El camino del dato al estimando, con la demanda que cada pieza introduce. La comparación es computable solo si cada demanda anterior ya se cumple. 19
Figura 2.1 El sistema desplegado como bucle causal. La arista carmesí no observa el mundo: lo modifica. La muestra del modelo de mañana la selecciona el modelo de hoy, y sin un holdout permanente ni registro de la asignación, el bucle destruye la identificación que la edición de método asume. 23
Figura 2.2 Los dos planos del sistema. El plano de método (fila superior) estima y sirve; el plano de datos (fila inferior) sostiene la comparación en cada pieza. Las aristas punteadas son las exigencias que el método le baja al dato. 24
Figura 3.1 El esquema de roles: X, D, Y (y Z, cuando hay instrumento) alimentan el estimando con un rol por columna. La columna posterior al tratamiento no entra al estimando: el contrato sólo lo deduce si alguien declaró su tiempo; ese es el límite honesto que el Cap. 6 cierra. 30
Figura 4.1 El linaje mínimo auditable: cada versión de dataset con su hash alimenta la estimación que la consumió, y el alias queda apuntando a la versión promovida. El mutante de la derecha no llega al alias: la puerta de integridad lo detiene en el registro. 43
Figura 5.1 La pared del gobierno del dato: el lado izquierdo lo tocan todas las decisiones del pipeline; al lado derecho solo entra la evaluación, una vez. Las columnas de verdad (y_0, y_1, τ) existen en el DGP para medir el prototipo: entrar al entrenamiento, al servicio o a cualquier decisión intermedia es la fuga que el contrato existe para impedir. 50
Figura 6.1 El eje del tiempo y la validez de las variables. Solo lo verificable como anterior al momento de la decisión puede entrar en una pregunta de producción; y solo lo anterior al tratamiento puede entrar como control en las dos especificaciones (X_{base} y X_{ampliada}). Los agregados tardíos quedan afuera por las dos puertas. . 54
Figura 7.1 La deriva como comparación C-02: las dos poblaciones — la que ajustó y la que llega — se miden con el mismo contrato que compara tratados contra controles. El umbral se calibra y se fija antes de mirar; la acción sale del vocabulario del sistema, no del estadístico. 62
Figura 8.1 El canal productor-consumidor del C-11: de un DGP declarado sale un escenario versionado y dos ramas etiquetadas — lo observado, abierto a toda decisión, y la verdad, abierta solo a evaluación con registro. La rama punteada solo evalúa: ni entrenamiento ni servicio la tocan, y su acceso queda registrado. 71

Figura 10.1 Heterogeneidad y política como extensión C-12: del bundle con propensión OOS al CATE y su valor IPW OOS. Sin OOS el ranking es tautológico. 87
Figura 11.1 Diagnóstico sin verdad: placebo, control negativo y sensibilidad convergen en Gamma-star. Sin verdad igual hay falsación OOS. 91
Figura 12.1 La puerta causal evalúa tres señales contra umbrales declarados y produce tres estados. Solo la versión en pasa llega al alias champion; alerta y falla exigen decisión humana explícita. 96
Figura 13.1 El servicio sirve el bundle del registro, no los datos de entrenamiento. Dentro del soporte responde con efecto e intervalo; fuera, se abstiene sin número. . . . 99
Figura 14.1 Las cuatro señales convergen en un orden de preguntas cuya primera falla decide la acción. Abstenerse va primero: sin soporte no hay estimador que juzgar. Revisar identificación va antes que alertar: un supuesto roto no se arregla con un aviso. 106
Figura 15.1 La corrida end-to-end del libro: cada contrato deja su artefacto y la traza es auditable de punta a punta. La tarjeta (C-05) es el documento que une todas las piezas. 110

Índice de tablas

Índice de listados

PARTE 0

El dato como estimando

Esta parte instala el vocabulario causal mínimo que un equipo de datos necesita —estimandos, resultados potenciales, el problema fundamental— y lo traduce de inmediato a decisiones de ingeniería: qué pregunta hace el sistema a los datos, cómo se declara la respuesta a esa pregunta en un contrato con roles, y por qué una afirmación causal es un enunciado sobre una versión concreta de un conjunto de datos, con hash y linaje. Los capítulos 1 y 2 son el mapa; el 3 y el 4 son el primer contrato y el primer registro del libro.

CAPÍTULO 1

1 Causalidad elemental para ingeniería de datos

Cuadro sinóptico del capítulo

Problema:	Distinguir qué comparación sostiene un dato: asociativa o causal
Idea central:	El estimando es una comparación, y el plano de datos decide si es computable
Herramienta:	Resultados potenciales $Y(1), Y(0)$; el invariante de comparabilidad
Caso canónico:	La reversión de Simpson, con verdad conocida (script con semilla)
Prepara:	El primer contrato del libro (Cap. 3) y el registro de versiones (Cap. 4)

1.1 Dos preguntas que parecen la misma

Suponga dos preguntas sobre un programa de becas universitarias. La primera: *¿los estudiantes becados tienen, en promedio, mejores calificaciones que los no becados?* La segunda: *¿dar una beca a un estudiante mejora sus calificaciones?* Las dos se responden, en apariencia, con el mismo tipo de dato — una tabla de estudiantes, una marca de quién recibió beca y un registro de calificaciones — y sin embargo son preguntas radicalmente distintas. La primera es una pregunta **asociativa**: describe un patrón en los datos tal como existen. La segunda es una pregunta **causal**: describe qué pasaría si se interviniera el mundo, otorgando una beca que de otro modo no se habría otorgado.

La confusión entre ambas no es un error ingenuo, y no la comete solo quien estima. También la comete un pipeline. Un sistema que ingiere la tabla y computa «mejores calificaciones con beca» está respondiendo la primera pregunta con exactitud técnica impecable — ningún error de redondeo, ningún valor faltante, ninguna inconsistencia de formato — y sirviéndola en el lugar de la segunda. Quien diseñó el pipeline no mintió en ninguna parte; simplemente **nunca escribió qué comparación estaba computando**. Ese silencio es el origen del problema

que ocupa a este libro: el dato, movido correctamente, entrega respuestas asociativas a un consumidor que cree haber pedido causales.

Y el silencio se vuelve más caro cuando el modelo deja de describir y pasa a decidir. Suponga que el programa usa el sistema para repartir los cupos entre quienes tienen mayor calificación esperada. Todo parecerá funcionar: el error de predicción es bajo, el tablero está en verde. Pero el sistema estará entregando las becas a quienes habrían obtenido buenas notas de todos modos, y el efecto que el programa buscaba producir se perderá sin que ninguna métrica de calidad de datos lo registre, porque **la fuga no es de datos: es de pregunta**. De ahí la pregunta que organiza este libro, y que ningún proceso de ingesta, por correcto que sea, contesta solo: *¿cuánto mejora la calificación de quienes reciben beca gracias al sistema, comparado con lo que habrían obtenido sin él?* Este libro está dedicado a una tesis concreta: **ninguna cantidad de datos, por limpia y puntual que llegue, convierte una pregunta asociativa en una causal**. Convertir una en otra requiere supuestos explícitos sobre cómo se generaron los datos, y esos supuestos, en un sistema real, se ganan o se pierden en el plano de datos: en los roles de las columnas, en las marcas de tiempo, en las particiones, en las versiones.

1.2 El estimando es una comparación que el dato debe sostener

Un ingeniero de datos no estima efectos: preserva lo que hace posible estimarlos. La traducción directa de la sección anterior al plano de datos empieza por una definición que el libro usa con el mismo sentido en todos sus capítulos.

Definición 1.1 ***(Estimando)***

El estimando es la comparación causal que el sistema se obliga a servir: qué contraste — por ejemplo, el efecto promedio del tratamiento, $\theta = E[Y(1) - Y(0)]$ —, sobre qué población, bajo qué supuestos declarados. El estimando no se «descubre» en los datos: se declara antes de mirar, y el plano de datos verifica después si la comparación declarada es computable con lo que hay.

La definición aparenta sencillez y tiene dientes. «Qué contraste» exige saber cuál es el tratamiento y cuál el resultado, por nombre de columna. «Sobre qué población» exige saber qué filas entran y por qué regla llegaron ahí. «Bajo qué supuestos» exige identificar qué covariables son legítimas para el ajuste, y eso solo puede declararse si alguien registró **cuándo se midió cada una y con qué rol causal**. Cuatro demandas de datos que una tabla ancha no lleva escritas por ninguna parte:

1. **Rama.** El dato debe registrar, unidad por unidad, en qué rama del contraste estuvo: tratada o no tratada. Sin la marca, no hay dos lados de la comparación que computar.
2. **Comparabilidad.** Las dos ramas deben ser comparables: si la asignación depende de variables que no se midieron, la comparación directa mezcla el efecto con la selección, en la dosis que desarrolla la Sección 1.4.
3. **Precedencia.** Las covariables de control deben medirse **antes** del tratamiento; una variable posterior es una fuga temporal, y el dato estándar no la distingue de una covariable legítima salvo que lleve marca de tiempo (Capítulo 6).

4. **Unicidad.** El estimando es sobre una versión concreta de los datos; si la tabla cambia bajo los pies, la afirmación estimada ayer se refiere a nada hoy (Capítulo 4).

Estas demandas son la agenda del libro: el rol de las columnas es el contrato del Cap. 3 la unicidad de la versión, el del Cap. 4 la partición que permite evaluar con honestidad, el del Cap. 5 y la precedencia temporal, el del Cap. 6. Cada una se escribe como un contrato numerado, porque una convención se recuerda y un contrato se ejecuta.

1.3 La reversión de Simpson: la comparación que cambia de signo

La advertencia de la sección anterior se estudia en su forma más cruda con una escena clásica, que aquí se responde con números generados por un proceso con verdad conocida.

Caso canónico · La reversión de Simpson

Un hospital evalúa dos tratamientos, A y B, para una misma condición. La gravedad del caso — 72 % de los pacientes llegan graves — decide en gran parte qué tratamiento recibe cada uno: el tratamiento A se reserva para los casos leves (lo recibe el 90 % de ellos) y el B para los graves (75 %). El dato dice la verdad completa: dentro de cada estrato, **B es mejor**; en casos leves, éxito de 0,86 con A contra 0,90 con B; en casos graves, 0,55 contra 0,60.

Población	A	B	Lectura
Casos leves	0,862	0,904	B mejor
Casos graves	0,548	0,598	B mejor
Agregado	0,732	0,614	A parece mejor

Agregando, el veredicto se invierte: A tiene una tasa global de 0,732 contra 0,614 de B. No hay truco de medición: la inversión es consecuencia de que A trabajó sobre casos fáciles y B sobre casos difíciles, de modo que el agregado compara perfiles distintos. Con 40 000 pacientes simulados (semilla 7), el agregado y los estratos salen exactamente como lo indica la tabla; el script del repositorio reproduce la escena y sus tres lecturas. El problema no es estadístico, es semántico: el agregado responde la pregunta *¿cuáles pacientes mejoraron con A?* y el estrato responde *¿cuál tratamiento conviene aplicar?*; solo la segunda es la pregunta causal.

La lección no es «desagregue siempre por gravedad», que la tabla misma deja sin defensa: si la asignación hubiera dependido de alguna otra variable — la hora de ingreso, el seguro, el barrio — el desagregado correcto sería otro. La lección es que **no existe una respuesta puramente estadística a qué desagregación es la correcta**: se requiere una teoría causal — la gravedad afecta a la asignación y al resultado — y esa teoría solo puede protegerse si el dato registra la gravedad como columna, con su tiempo de medida y su rol declarado. Cuando

la gravedad no se mide, la inversión no se corrige; cuando se mide pero nadie declaró su rol, se corrige solo si alguien recuerda condicionar.

1.4 El problema fundamental de la inferencia causal

Formalicemos. Con el marco de resultados potenciales (Neyman, 1923 Rubin, 1974), cada unidad i tiene dos resultados potenciales: $Y_{i(1)}$, el que se observaría si recibiera el tratamiento, y $Y_{i(0)}$, el que se observaría si no lo recibiera. El efecto causal individual es la diferencia $Y_{i(1)} - Y_{i(0)}$.

Definición 1.2 ***(El problema fundamental de la inferencia causal)***

Para cualquier unidad i solo se observa uno de los dos resultados potenciales, el de la rama que efectivamente recibió; el otro permanece contrafactual. El efecto individual $Y_{i(1)} - Y_{i(0)}$ nunca puede calcularse directamente para ninguna unidad, sin importar cuánto dato se recolecte sobre ella (Holland 1986).

Lo que sí puede estimarse, bajo supuestos, es un efecto promedio sobre grupos distintos. Lo que el dato entrega sin supuestos es la **diferencia de medias observada**, $\Delta = E[Y \mid D = 1] - E[Y \mid D = 0]$, que con la restricción de consistencia $Y = D \cdot Y(1) + (1 - D) \cdot Y(0)$ se descompone en

$$\Delta = \underbrace{E[Y(1) - Y(0) \mid D = 1]}_{\text{ATT: efecto sobre los tratados}} + \underbrace{E[Y(0) \mid D = 1] - E[Y(0) \mid D = 0]}_{\text{sesgo de selección}}. \tag{1.1}$$

El segundo término compara los resultados que tratados y no tratados habrían tenido **sin tratamiento**: es cero solo si ambos grupos eran comparables de partida, y es incalculable con datos, porque involucra el resultado potencial no observado. Las escenas de la Sección 1.3 son casos donde ese término no es cero y domina. La aleatorización lo anula **por diseño** — si D es independiente de $(Y(0), Y(1))$, entonces Δ estima el efecto —, y lo que ningún experimento garantiza es exactamente lo que la mayoría de los pipelines hereda: una asignación decidida por alguien, con información que quedó o no quedó en los datos.

Aquí está la división de trabajo del libro. **El sesgo de selección no corrige nadie en el plano de datos**: no hay ninguna transformación, ninguna normalización y ningún test de calidad que lo mida, porque vive en la parte no observada de (1.1). Lo que el plano de datos controla, y controla por completo, es si el sistema llega a la comparación con todo lo necesario para esa corrección: las marcas de rama, las covariables pre-tratamiento con sus tiempos, la partición honesta para evaluar y la versión exacta sobre la que se estimó. El plano de método — la edición hermana de este libro — aporta después los supuestos de identificación y el estimador. El plano de datos es, entonces, la condición necesaria: si destruye la comparación antes de llegar al estimador, ninguna teoría de identificación la rescata.

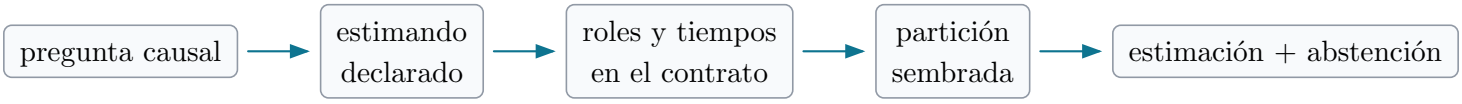


Figura 1.1: El camino del dato al estimando, con la demanda que cada pieza introduce. La comparación es computable solo si cada demanda anterior ya se cumple.

1.5 Ejercicios y notas bibliográficas

1.5.1 Ejercicios conceptuales

1. Para cada pregunta, indique si es asociativa o causal, y justifique en una oración: (a) ¿los países con más médicos per cápita tienen menor mortalidad infantil? (b) ¿contratar más médicos reduciría la mortalidad en un país específico? (c) ¿los clientes a quienes se les ofreció descuento renovaron más? (d) ¿la oferta de descuento hizo renovar a más clientes? *Pista:* en cada caso, pregunte quién decidió a quién tratar y con qué información.
2. Proponga un ejemplo propio de correlación espuria por una variable confusora. ¿Está la confusora medida en su base de datos? Si no, dígalo qué demanda de las cuatro de la Sección 1.2 no se cumple. *Pista:* si la confusora no existe como columna pre-tratamiento, el problema empieza antes del grafo.
3. En el caso canónico de Simpson, agregue un tercer tratamiento C, aplicado solo a graves, con tasa verdadera de éxito 0,55. ¿Qué lee el agregado? ¿Cambia lo que un pipeline debería registrar para discernir? *Pista:* lo relevante sigue siendo la columna de gravedad y el rol del tratamiento; ninguna métrica de calidad detecta la inversión.
4. Un equipo entrena un modelo que predice ventas a partir de una marca de promoción, con R^2 de 0,95 fuera de muestra, y concluye que «la promoción explica el 95 % de las ventas». ¿Qué comparación computó el modelo, y qué comparación cree el equipo haber computado? *Pista:* la asociación predicha mezcla efecto y selección; el R^2 alto dice solo que el patrón es estable.
5. Su equipo renombró la columna del resultado de `ingreso_2024` a `y`, y nadie lo comunicó. ¿Qué demanda de las cuatro queda rota primero, y en qué capítulo de este libro se escribe el contrato que la protege? *Pista:* el estimando no cambió, pero la tabla dejó de ser la que el estimador cree leer; la primera línea de defensa es la unicidad de los roles (Cap. 3).

1.5.2 Notas bibliográficas

El marco de resultados potenciales se origina en (Neyman 1923) para experimentos aleatorizados y se extiende a estudios observacionales en (Rubin 1974); Holland (1986) formaliza el problema fundamental, y su tratamiento completo, incluyendo cuándo el ATT y el ATE coinciden, está en (Imbens y Rubin 2015), Capítulo 1. La descomposición en ATT más sesgo de selección es estándar; la exposición aplicada más leída es (Hernán y Robins 2020). La paradoja de Simpson se describe en (Simpson 1951), documentada ya en (Yule 1903). La escalera de la causalidad de Pearl — asociación, intervención, contrafactual — se resume en

(Pearl y Mackenzie 2018) y, en su versión formal, en la edición de método de este libro, junto al tratamiento completo de la identificación que este capítulo solo nombra. La lectura en términos de ingeniería — qué demanda del dato corresponde a qué pieza del sistema — es la contribución propia de esta edición, alimentada por la tradición de la ingeniería de datos de (Kleppmann 2017) y (Reis y Housley 2022), a la que el Cap. 3 vuelve con su primer contrato.

Qué debe ser verdad en los datos

Que una comparación asociativa y una causal se calculen sobre la misma tabla no las vuelve intercambiables; el signo de la lectura puede invertirse sin que cambie ninguna celda. Lo que debe ser verdad en los datos: las cuatro demandas de la Sección 1.2 — marcas de rama, ramas comparables, covariables anteriores al tratamiento, y una versión única de la tabla — y deben serlo **a nombre de columna y de fila**, no como intención. Se comprueban declarando el estimando antes de mirar y verificando después que la comparación declarada es la que el pipeline computó; ninguna métrica de calidad de datos estándar las vigilaba en el caso canónico, y allí mismo se vio el precio.

CAPÍTULO 2

2 Anatomía de un sistema de decisión

Cuadro sinóptico del capítulo

Problema:	Un modelo entrenado y un sistema en producción son dos objetos distintos; entre ambos vive el plano de datos
Decisión:	Separar plano de método y plano de datos, y darle a cada uno su dueño y sus invariantes
Vocabulario:	Cinco acciones del sistema, siempre con el mismo sentido: abstenerse, revisar, alertar, reentrenar, continuar
Caso canónico:	La traza de una consulta por el sistema, sobre el escenario `real/jobs` del benchmark
Prepara:	Todos los contratos del libro (Caps. 3 a 6 y Parte IV)

2.1 Dos planos, dos responsabilidades

Un equipo entrena un estimador de efectos y obtiene intervalos razonables, diagnósticos satisfactorios y una historia convincente sobre el mecanismo. Si ese estimador se entrega tal cual a un equipo de producto para decidir a quién ofrecer un subsidio, algo saldrá mal en semanas: la función que calcula las covariables en producción no será exactamente la de entrenamiento; nadie sabrá qué hacer con un valor faltante; tres meses después, nadie podrá decir si la población servida sigue pareciéndose a la de entrenamiento; y si el sistema empieza a fallar, nadie recibirá una alerta hasta que un usuario se queje. Huyen (2022) lo formuló con precisión: lo que la gente llama «el sistema de machine learning» es apenas un componente de un sistema que incluye la interfaz, el stack de datos, el cómputo y la infraestructura que lo desarrolla, despliega, monitorea y actualiza — y la evidencia empírica recopilada por Paleyes et al. (2022) confirma que la fracción de fallos atribuible al algoritmo mismo es minoritaria.

Este libro estructura esa observación en dos planos con responsabilidades distintas:

- **Plano de método.** Identificación y estimación: los supuestos bajo los que la comparación de la (1.1) se hace interpretable, el estimador, la incertidumbre. Tiene su tratamiento completo en la edición de método de este libro y aquí solo aparece como

consumidor: cada estimador le exige al dato tasas de convergencia, solapamiento, covariables legítimas y particiones honestas.

- **Plano de datos.** La cadena que preserva la comparación: el contrato de roles (Capítulo 3), la ficha de procedencia y el registro de versiones (Capítulo 4), la partición sembrada y la pared (Capítulo 5), la validez temporal (Capítulo 6) y, en operación, la puerta y el monitor que Parte IV describe. Su dueño natural es la ingeniería de datos, y su unidad de rigor no es el teorema sino el **contrato con prueba de aceptación ejecutable**.

Definición 2.1 ***(El invariante del plano de datos)***

Ninguna pieza del plano de método consume datos sin pasar por el plano de datos: toda estimación llega con su esquema y su semilla, toda evaluación con su partición y su versión, y toda decisión de promoción con su registro. Si una sola pieza toma datos por un camino lateral — un cuaderno suelto, una copia descargada, una columna extraída ad hoc —, el invariante está roto aunque el estimador sea impecable, porque ya nadie puede reconstruir sobre qué versión de qué tabla se estimó qué.

La separación de planos no es burocracia: responde a un trade-off real que compite por el tiempo de todo equipo. El enfoque centrado en el modelo optimiza la especificación estadística sobre un dataset fijo, y es el modo dominante de la academia; el centrado en el sistema reparte el esfuerzo entre datos, modelo e infraestructura, y es el único que sobrevive el contacto con datos que cambian y usuarios que dependen del sistema (Huyen 2022; Sculley et al. 2015). La discusión de deuda técnica de Sculley et al. (2015) y la rúbrica operativa de Breck et al. (2017) son las referencias de ese trade-off; este libro las adopta y les añade lo que ellas no tratan: que cuando el objetivo es un **efecto**, la «deuda de datos» no solo degrada la precisión — destruye la comparación que hace de la estimación una afirmación causal.

2.2 El sistema desplegado es una intervención

Hasta aquí hablamos del sistema como si mirara el mundo y respondiera. Esa imagen es cómoda y es falsa. En cuanto el sistema decide a quién se le ofrece el subsidio, deja de ser observador: sus decisiones son una intervención sobre el mundo que mide, y esas decisiones se convierten en los datos con los que se entrenará su próxima versión. El bucle es corto:

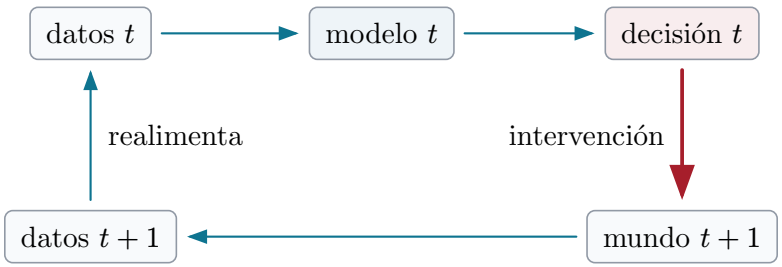


Figura 2.1: El sistema desplegado como bucle causal. La arista carmesí no observa el mundo: lo modifica. La muestra del modelo de mañana la selecciona el modelo de hoy, y sin un holdout permanente ni registro de la asignación, el bucle destruye la identificación que la edición de método asume.

Dibujado así, tres consecuencias se hacen visibles y ningún diagnóstico de predicción las detecta. **Primera: la asignación del tratamiento pasa a ser la política del propio modelo.** Si el sistema recomienda siempre la misma acción para un mismo perfil, la probabilidad de tratamiento dado X vale 0 o 1 en los datos nuevos: sin variación no hay con quién comparar, y el efecto deja de estar identificado por los datos que el sistema mismo produce. Es la condición de soporte común, rota por construcción y no por mala suerte. **Segunda: solo se observa el resultado de quienes recibieron la acción.** El crédito no concedido no registra si habría sido pagado: el modelo nuevo aprende de una muestra que el modelo viejo seleccionó, y el sesgo se acumula con cada reentrenamiento (Sculley et al. 2015). **Tercera: lo que se quiere medir puede cambiar con la política.** Un modelo que dispara una intervención preventiva consigue que muchos casos que marcó no ocurran, y su precisión aparente empeora: cuando la predicción influye en el resultado, la pregunta correcta deja de ser cuánto acertó el modelo y pasa a ser qué habría ocurrido sin él — la predicción *performativa* de (Perdomo et al. 2020).

La salida no es dejar de decidir con modelos: es diseñar el plano de datos para que el sistema siga siendo identificable mientras opera, con tres prácticas que todo pipeline de decisión debe registrar. **Reservar un holdout permanente**: un grupo pequeño, asignado al azar, que el modelo no gobierna y que garantiza comparación válida mañana (Capítulo 5). **Registrar con qué probabilidad se asignó cada acción**: sin ese log no hay corrección posible por la forma en que se decidió. **Mantener un piso de exploración**: una fracción mínima de asignación aleatoria dentro de cada perfil, de modo que ninguno quede sin comparación. Cuestan valor de corto plazo; compran que el efecto siga siendo medible el año próximo.

2.3 Las cinco acciones del sistema

Un sistema que opera necesita un vocabulario cerrado para lo que hace frente a lo que observa. Este libro usa cinco acciones, siempre con el mismo sentido, de modo que «alertar» o «abstenerse» signifiquen lo mismo en el servicio, en la ficha del modelo, en las pruebas automáticas y en el monitor. La tabla las ordena de la más restrictiva a la menos restrictiva.

Acción	Qué significa	Quién actúa
Abstenerse	El sistema no devuelve un número: responde «sin respuesta» a esa consulta, o a todas, porque no puede sostener el efecto (por ejemplo, la consulta está fuera del soporte declarado).	El sistema
Revisar identificación	Un supuesto de identificación dejó de sostenerse o dejó de poder comprobarse. No se promueve ninguna versión nueva hasta que alguien revise el diseño y los supuestos.	Una persona

Acción	Qué significa	Quién actúa
Alertar	Una señal superó su umbral de aviso. El sistema sigue respondiendo y una persona recibe la señal junto con su evidencia.	Una persona
Reentrenar	El estimador se ajusta de nuevo con datos nuevos, sin cambiar el estimando ni la población declarados.	El sistema, bajo una regla de promoción
Continuar	Ninguna señal supera su umbral. El sistema sigue respondiendo con la versión vigente.	El sistema

Cuando varias acciones aplican a la vez, prevalece la más restrictiva. El orden no es arbitrario. Abstenerse va primero porque un número sin sustento es peor que ningún número. Revisar va antes que alertar, porque un supuesto roto no se arregla con un aviso. Y reentrenar queda por debajo de alertar a propósito: ante un cambio en los datos, lo primero es preguntar qué cambió, y reentrenar solo es la respuesta cuando cambió la distribución de covariables y el efecto sigue siendo el mismo; si el cambio alteró quién recibe el tratamiento o cómo se mide el resultado, reentrenar produce un modelo nuevo para un problema distinto. Qué señal dispara cada acción y con qué umbral lo deciden los capítulos de operación de la Parte IV; aquí solo se fija el vocabulario, y la regla de oro de este libro en producción: **las señales orientan una revisión; no prueban por sí solas la causa del cambio**.

2.4 El esqueleto en el plano de datos

Las piezas del plano de datos de este libro, en el orden en que el dato las recorre:

1. **Contrato de roles (C-01, Cap. 3):** cada columna, un solo rol; el esquema viaja con los datos; nada entra como covariable sin declararse.
2. **Ficha de procedencia y registro (C-09, Cap. 4):** cada versión de datos y de artefacto con hash; el linaje dato → estimador → promoción; el alias `champion` como único camino a producción.

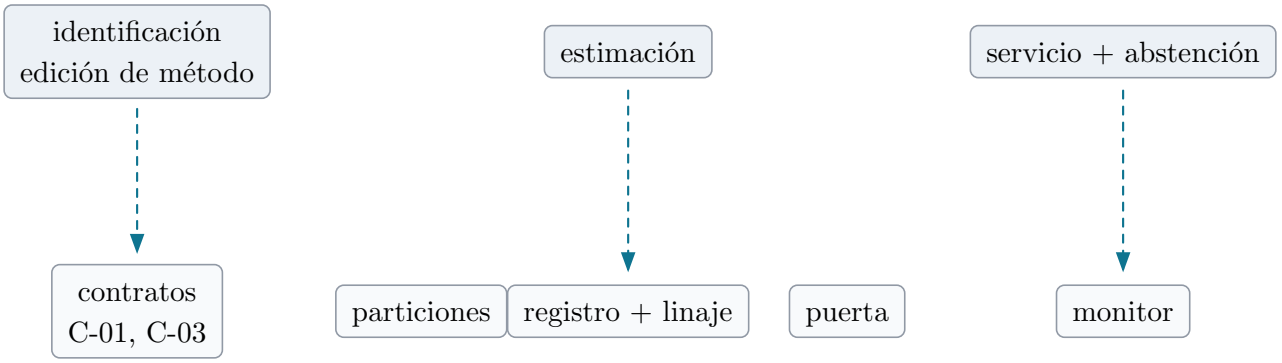


Figura 2.2: Los dos planos del sistema. El plano de método (fila superior) estima y sirve; el plano de datos (fila inferior) sostiene la comparación en cada pieza. Las aristas punteadas son las exigencias que el método le baja al dato.

3. **Partición sembrada y pared (C-10, Cap. 5):** entrenamiento, validación y prueba como estructura versionada; lo reservado para evaluación del otro lado del muro.
4. **Auditoría temporal (C-03, Cap. 6):** ninguna covariable entra sin marca de tiempo verificada ni rol resuelto.
5. **Puerta y monitor (Parte IV):** la decisión de promover con estados `pasa`, `alerta`, `falla`, y las señales que después obligan a revisar.

La lista es deliberadamente corta. Una anatomía confiable no es la que tiene más piezas sino la que, ante una consulta — **¿sobre qué versión de qué tabla se estimó este número, con qué roles, en qué partición, para qué estimando?** — responde sin depender de la memoria de nadie.

2.5 Caso canónico: una traza por el sistema

Caso canónico · La consulta completa sobre `real/jobs`

El benchmark del repositorio de referencia incluye el escenario `real/jobs`, una réplica sintética estilo LaLonde (1986) del programa NSW, con 1 722 unidades y verdad conocida guardada al otro lado de la pared. Recorremos una consulta — **¿cuál es el efecto del programa de entrenamiento sobre los ingresos de 1978?** — pieza por pieza, usando solo lo que los archivos declaran (script con traza completa en el repositorio):

Pieza	Lo que el archivo aporta
1. Dato bruto	`observed.csv`: 1 722 filas, 10 columnas.
2. Contrato C-01	Tratamiento `treatment`, resultado `re78`; 8 covariables pre-tratamiento declaradas como conjunto de ajuste en `identification.json`.
3. Procedencia	`metadata.json`: asignación 722 experimental + 1 000 observacional (estilo LaLonde); el origen declara qué es real y qué es sintético.
4. Partición C-10	`splits.json`, semilla 20240701, método estratificado por tratamiento 60/20/20: 1 033 train / 344 validación / 345 prueba, cada uno con su hash SHA-256.
5. Comparación cruda	$E[Y\|D=1]-E[Y\|D=0]=1\,\backslash\backslash,644\,\backslash\backslash,9$ USD ($E[Y\|D=1]=8\,\backslash\backslash,364\,\backslash\backslash,8$, $E[Y\|D=0]=6\,\backslash\backslash,719\,\backslash\backslash,9$; $P(D=1)=0\,\backslash\backslash,254$).
6. Referencia del DGP	El diseño fija el ATE en torno a 1 800 USD, fuera de la tabla observada: la pared, vigente.

La lectura del paso 5 no es el ATE: mezcla el efecto con la selección del programa, en la dosis exacta de (1.1) (los participantes del NSW no eran una muestra aleatoria de la población con la que se les compara). Que esa comparación crude llegue al plano de

método **con** su esquema, su procedencia, su partición y su hash — y no como una tabla suelta — es el trabajo del plano de datos, y la única razón por la que la corrección posterior puede verificarse.

Repositorio de referencia del libro

Escenario completo: `data_benchmark_causal/causal_benchmark_synthetic/datasets/real/jobs/` (`metadata.json`, `identification.json`, `splits.json`, `observed_*.csv`). Traza reproducible: `Causal_AI/scripts/cap02_traza.py`. Implementación del registro y la puerta: `04_produccion_causal/src/causalops/`.

2.6 Ejercicios y notas bibliográficas

2.6.1 Ejercicios conceptuales

1. Enumere los componentes de un sistema de ML según (Huyen 2022) y asigne cada uno a uno de los dos planos. ¿Qué componente queda disputado entre ambos? *Pista:* la preparación de variables es el punto de fricción clásico: es ingeniería, pero traiciona la identificación si ignora los tiempos (Cap. 6).
2. Explique por qué reentrenar con datos nuevos es más delicado en un sistema causal que en uno predictivo, usando el bucle de la figura 2.1. *Pista:* piense en quién seleccionó la muestra del próximo modelo.
3. Un banco decide créditos con un modelo y cada mes reentrena con los créditos otorgados. Dibuje el bucle y responda: ¿qué llega al modelo del mes siguiente sobre los solicitantes rechazados? ¿Qué cambia si el banco asigna al azar el 2 % de los créditos? *Pista:* las dos prácticas del holdout y del piso de exploración.
4. Para cada situación, indique cuál de las cinco acciones aplica y si prevalece alguna: (a) una covariable se desplazó y la señal superó su umbral; (b) llega una consulta fuera del soporte; (c) una política interna hizo que el perfil alto jamás reciba la oferta; (d) ninguna señal supera su umbral. *Pista:* la respuesta de (c) empieza por preguntarse si cambió la asignación, no por reentrenar.
5. En el esqueleto de la figura 2.2, señale qué pieza detecta cada uno de estos fallos, y en qué capítulo aparece su contrato: columna renombrada en el origen; tabla que cambió tras el registro; partición reutilizada dos veces; covariable calculada con el resultado. *Pista:* cada fallo tiene exactamente una pieza destinataria; si hay dos sin pieza, el esqueleto tiene agujero.

2.6.2 Notas bibliográficas

Huyen (2022) es la fuente de la filosofía sistema-céntrica y del marco de cuatro requisitos que este capítulo adapta a los dos planos. Sculley et al. (2015) documenta la deuda técnica

acumulada en el resto del sistema y sus bucles de retroalimentación ocultos, que aquí reaparecen como el bucle causal de la figura 2.1. Breck et al. (2017) aporta la rúbrica operativa complementaria, y Paleyes et al. (2022) la evidencia empírica de que los fallos no viven en el algoritmo. La predicción performativa es de (Perdomo et al. 2020). Sobre el eje de este libro en particular, la tradición de ingeniería de datos de (Kleppmann 2017) y (Reis y Housley 2022) es el referente de la robustez de pipelines que aquí se especializa al enunciado causal.

Qué debe ser verdad en los datos

Que el modelo entrena bien no dice nada sobre si el sistema puede sostener su afirmación: la afirmación vive en la comparación, y la comparación vive en el plano de datos. Lo que debe ser verdad en los datos: que cada pieza del sistema consuma por el canal del invariante — esquema, semilla, partición, versión — y que el bucle de intervención no borre la única variación que permite medir el efecto (holdout permanente, piso de exploración, log de asignación). Se comprueba preguntando la consulta del esqueleto — versión, roles, partición, estimando — y exigiendo respuesta sin memoria humana; en el caso canónico esa pregunta se responde con archivos, no con opiniones.

CAPÍTULO 3

Contratos de datos causales

Cuadro sinóptico del capítulo

Decisión:	Declarar los roles causales de cada columna en un esquema único que viaja con los datos, y no repetirlos en cada análisis
Pieza contratada:	El contrato C-01 (esquema de roles causales), el primero del libro
Contrato vs convención:	Una convención se recuerda; un contrato se ejecuta en una prueba de aceptación
Caso canónico:	Dos análisis, dos definiciones de X: mismo dato, efecto 1,00 frente a 1,50 (script con semilla)
Límite honesto:	El contrato comprueba ortografía de columnas y tiempos declarados; no verifica que el rol declarado sea el verdadero (Capítulo 6)

3.1 De las convenciones a los contratos

Los pipelines viejos no fallan por ignorancia de sus autores: fallan porque todo el conocimiento que los mantiene correctos estaba en la cabeza de quien las construyó. «La columna `educ` es la escolaridad, no la etiqueta; el resultado se recalcula los domingos; nunca uses `exper` porque es posterior a la decisión» — acuerdos verbales, convenciones de equipo, una página de wiki que nadie relee. En la ingeniería de datos clásica, de (Kleppmann 2017) y (Reis y Housley 2022), eso ya era una deuda: el esquema implícito se desincroniza de los datos y nadie recuerda cuándo. En un sistema causal es además una amenaza a la identificación, porque la decisión sobre qué es una covariable legítima, tomada de memoria, puede destruir la comparación del estimando sin lanzar ningún error (Sculley et al. (2015) lo anota como deuda de datos en su sección de dependencias y configuración).

La salida de este libro para esa deuda no es más documentación: es **contratos**. Un contrato describe una pieza de ingeniería sin atarla a una tecnología — sea una clase, un paso de pipeline o un proceso manual — y lo describe con seis campos fijos:

1. **Operaciones**: lo que la pieza ofrece a quien la usa.
2. **Entrada** y **salida**: lo que recibe y lo que devuelve, al nivel que necesita quien implementa.
3. **Invariantes**: lo que debe cumplirse siempre, para cualquier entrada, y de forma comprobable — si no hay manera de verificarlo, no es un invariante sino un deseo.
4. **Modos de fallo**: cómo falla la pieza, incluyendo los fallos que el contrato existe para impedir y los que declara no poder ver.
5. **Prueba de aceptación**: el procedimiento que decide, sobre datos con verdad conocida o fallos inducidos a propósito, si una implementación cumple el contrato.

La diferencia entre invariante y prueba importa y es el núcleo del rigor de esta edición: **el invariante dice qué es verdad; la prueba es la evidencia de que lo es**. Una convención se recuerda; un contrato se ejecuta en CI cada vez que algo cambia. Todo contrato del libro lleva el mismo formato, y Parte III de cada parte los colecciona: el esquema de roles (C-01), la auditoría temporal (C-03), el registro de versiones y linaje (C-09) y la partición versionada (C-10), junto a los que la Parte IV añade en operación.

Definición 3.1 ***(Contrato de datos)***

Un contrato de datos es la parte de un contrato que describe tablas, no servicios: qué columnas existen con qué rol, qué invariantes de contenido pueden decidirse con nombres y valores, cómo se reporta un fallo y con qué prueba de aceptación se comprueba. Lo que un contrato no puede verificar — que el rol declarado corresponde al rol causal verdadero — lo declara, porque firmarse a sí mismo como verificado es el fallo más caro de todos.

3.2 El problema que añade el modo causal

La ingeniería de datos genérica pregunta qué formato usar, qué modelo de datos tiene sentido y cómo pasar datos entre procesos; esas preguntas siguen vigentes aquí. Pero un sistema causal añade una que un pipeline puramente predictivo jamás necesita responder: **¿qué rol estructural juega cada columna?** En predicción las columnas de entrada son en principio intercambiables, porque el modelo usa todas las que ayuden a reducir el error. En un problema causal no lo son. Una es el **tratamiento** D, otra es el **resultado** Y, algunas son **covariables** X que pueden controlarse legítimamente y, lo más delicado, algunas que **parecen** covariables razonables son variables posteriores al tratamiento: están en la ruta causal entre D e Y, o son efecto común de ambos, y controlarlas sesga la estimación — la distinción entre controles legítimos y malos controles que la edición de método de este libro desarrolla como criterio de puerta trasera.

La diferencia no es académica: tiene un precio medible. En el caso canónico de este capítulo, dos análisis que controlan conjuntos distintos de covariables sobre la misma tabla producen efectos de 1,00 y 1,50 — una brecha de 0,50 que no es ruido sino consecuencia de controlar una

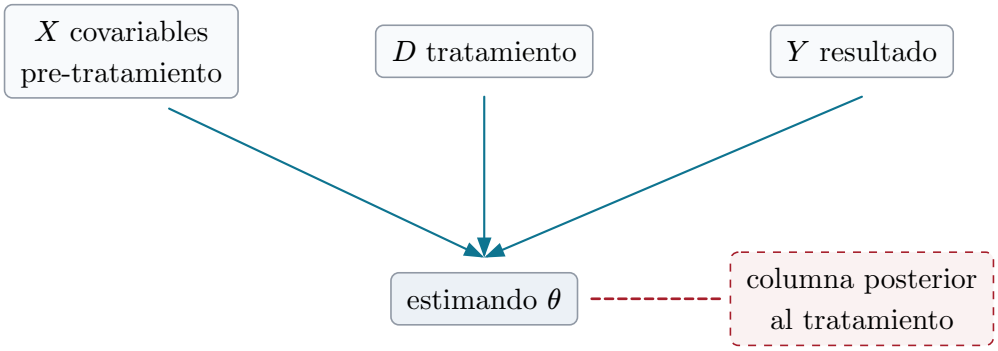


Figura 3.1: El esquema de roles: X, D, Y (y Z, cuando hay instrumento) alimentan el estimando con un rol por columna. La columna posterior al tratamiento no entra al estimando: el contrato sólo lo deduce si alguien declaró su tiempo; ese es el límite honesto que el Cap. 6 cierra.

variable que contiene al tratamiento. Ninguna métrica de calidad de datos estándar detecta esa diferencia, porque ninguna es un problema de forma: es un problema de **qué comparación sostiene el dato**. El pipeline que no distingue roles desde el origen deja esa decisión en la memoria de cada analista, y la memoria falla de forma predecible cuando el sistema crece.

Si el pipeline no distingue esos roles desde el origen, la tarea de no confundirlos recae en la memoria de quien escribe cada análisis, y eso falla de forma predecible cuando el sistema crece y más personas lo tocan. Un análisis puede controlar por una experiencia calculada con el tratamiento, otro no, y producir estimaciones no comparables sin que nadie lo note, exactamente como muestra el caso canónico de más abajo.

3.3 El contrato C-01: esquema de roles causales

El libro adopta un **esquema de roles causales** único para todo dato que alimente una estimación, y lo formaliza como el primero de sus contratos. El esquema declara cuatro cosas — el tratamiento, el resultado, las covariables y, cuando aplica, los instrumentos —, ninguna función del sistema acepta una tabla sola, y el esquema es la única fuente de verdad sobre los roles: la corrección semántica importa, el formato físico no.

Contrato C-01 · Esquema de roles causales

Operaciones	*declarar*: construye el esquema a partir de los nombres del tratamiento, el resultado, las covariables y los instrumentos. *validar*: comprueba el esquema contra una tabla de datos. *proyectar*: devuelve de la tabla la matriz de covariables, el tratamiento y el resultado, en el orden declarado. *escribir* y *leer*: pasan el esquema a un texto legible por personas y por máquinas, y de vuelta.

Entrada	Los nombres de las columnas de cada rol (un tratamiento, un resultado, al menos una covariable, cero o más instrumentos) y la tabla de datos a la que el esquema se aplica.
Salida	La matriz de covariables, el tratamiento y el resultado. El texto del esquema, en el formato que registra la ficha del modelo (Parte IV) y la edición de método.
Invariantes	(1) Cada columna tiene un solo rol: ninguna aparece dos veces en el esquema. (2) Hay al menos una covariable. (3) Una tabla solo valida si contiene todas las columnas declaradas y si el tratamiento toma al menos dos valores distintos. (4) El orden de las covariables es parte del esquema y las proyecciones lo respetan. (5) Escribir y volver a leer devuelve un esquema igual al original. (6) El esquema no se modifica una vez creado: cambiar los roles produce otro esquema y deja intacto el que ya usan otras piezas. (7) Declarar una covariable es una afirmación causal que este contrato no verifica; la verifica la auditoría del C-03 (Capítulo 6).
Modos de fallo	Una columna declarada que ya no está en los datos, porque cambió de nombre en el origen. El tratamiento o el resultado repetidos entre las covariables, o un instrumento incluido entre ellas. Un tratamiento constante en la muestra, porque un filtro dejó una sola rama. Un esquema que se desincroniza de los datos porque viaja por separado. Una covariable posterior al tratamiento que pasa todas las comprobaciones, porque el contrato solo ve nombres.
Prueba de aceptación	(a) Cada fallo estructural de la lista se provoca a propósito y la operación correspondiente lo rechaza con un mensaje que nombra la columna. (b) Escribir y leer devuelve un esquema igual, y el texto escrito valida contra el formulario de roles de la ficha del modelo. (c) Dos análisis que consumen el mismo esquema reciben la misma matriz de covariables y obtienen la misma estimación sobre el caso canónico de más abajo. (d) Una covariable posterior al tratamiento pasa la validación; el contrato no pretende detectarla y la prueba lo deja a la vista.

Caso canónico · Dos análisis, dos definiciones de X

Un conjunto de datos laborales estilizado, con efecto conocido. La educación de los padres P y la edad E son normales estándar independientes; la escolaridad depende de los padres, $D = 0.5P + \varepsilon_D$, y el salario es $Y = D + 0.8P + 0.5E + \varepsilon_Y$, de modo que el efecto de un año de escolaridad vale exactamente 1. La tabla incluye una columna de

experiencia potencial, $S = E - D$, calculada con la edad y la escolaridad — una práctica habitual en datos laborales: en el conjunto de datos de Card (1995), tal como se distribuye públicamente, la experiencia se computa como edad menos escolaridad menos seis.

Dos análisis leen la misma tabla. El primero controla por $\setminus \{P, E \setminus\}$ y el segundo por $\setminus \{P, S \setminus\}$. Los dos conjuntos existen en los datos, ninguno repite el tratamiento y ambos pasan cualquier validación estructural. Con 5 000 unidades y 200 repeticiones (semilla 11), el primero estima **1,00** y el segundo **1,50**, con desviaciones de 0,01 y 0,02 entre repeticiones.

La brecha de 0,50 no es ruido. Como S contiene a D, controlarla abre el camino $D \to S \leftarrow E \to Y$, y el coeficiente de D absorbe el efecto de la edad: sustituyendo $E = S + D$ queda $Y = 1.5D + 0.8P + 0.5S + \varepsilon_Y$. Con un esquema único no hay dos definiciones de X que comparar, y la discusión sobre si S pertenece a él ocurre en un solo lugar y queda registrada. Lo que el esquema no decide es cuál de las dos es la correcta: eso lo decide la teoría causal y su auditoría temporal (Capítulo 6), que deja S fuera porque se calcula con el tratamiento.

Repositorio de referencia del libro

Regeneración exacta: `Causal_AI/scripts/cap03_dos_definiciones.py` (semilla 11). La clase de datos inmutable del esquema, que rechaza columnas con dos roles y valida contra la tabla, es la referencia del contrato C-01: `03_notebooks_caso_estudio/lib/schema.py` y su puente `CausalSchema.from_scenario(«real/jobs»)` desde los metadatos del benchmark.

3.4 La prueba de aceptación sobre datos reales

La promesa del contrato se mide contra tablas que existen. Sobre el escenario `real/jobs` del benchmark — esquema `treatment` / `re78` / 8 covariables — la implementación de referencia toma la tabla de entrenamiento y la valida (debe pasar), y toma versiones corrompidas a propósito (deben fallar, nombrando la columna):

Caso inducido	Espera	Sale
Tabla de entrenamiento real	Pasa	Pasa: columnas presentes, el tratamiento varía
El origen renombró `educ`	Falla	Falla: `Columnas ausentes en los datos: ['educ']`
Un filtro dejó el tratamiento constante	Falla	Falla: `El tratamiento 'treatment' no varía`
`treatment` colado entre covariables	Falla	Falla: `Columnas con dos roles: ['treatment']`

Cada salida es la columna responsable, no un error genérico. Es la primera propiedad de la lista de la aceptación — todo fallo nombra a su culpable — y es la diferencia visible entre un contrato y un filtro de calidad de datos estándar: un test de nulidad también detectaría la columna ausente, pero solo después de que la estimación saliera con una interpretación equivocada; el contrato detiene antes de que haya un número que interpretar.

El mismo contrato viaja, en el benchmark, como datos adyacentes a los datos: `metadata.json` declara tratamiento, resultado y las covariables por nombre, e `identification.json` declara el conjunto de ajuste pre-tratamiento y la anotación causal completa del escenario (Capítulo 2, la traza del Cap. 2). Quien consume el escenario no hereda roles de la memoria de nadie: los lee, los valida y los registra. El formato — CSV con metadatos vecinos, como aquí, o Parquet con el esquema embebido — no cambia el contrato; lo que el contrato exige es unicidad y versionado (Capítulo 4).

3.5 Lo que el contrato no verifica

Un contrato firme sobre su propio límite es la diferencia entre gobernanza y decoración. El C-01 verifica ortografía y consistencia: ninguna covariable ha dicho en él ser pre-tratamiento salvo declaración de quien la inscribe. Una experiencia potencial S pasa la validación tal como pasan las covariables legítimas, porque el contrato solo lee nombres. Por eso la entrega del esquema no es «los roles están bien» sino «los roles están **escritos**, y están escritos donde dos análisis los comparten por fuerza»: la afirmación causal de las covariables la audita el C-03 con marcas de tiempo, y es el trabajo del Cap. 6. Si algún sistema se comportara como si el C-01 validara la causalidad por sí solo, daría por revisada la identificación cuando solo revisó la ortografía.

3.6 Cómo se diseña un contrato

El paso de una convención a un contrato no es burocrático: es un proceso de decisión que este libro repite doce veces. El procedimiento que sigue funciona para cualquiera de ellos:

1. **Identificar la convención implícita.** Qué regla verbal o de wiki existe ya, y a qué fallo concreto protege. Si no hay convención, hay que escribir una desde el fallo: ¿qué salió mal la última vez que algo rompió?
2. **Escribir los invariantes como afirmaciones comprobables.** Cada invariante debe poder convertirse en un test que falla o pasa — no en una intención. «Las covariables deben ser pre-tratamiento» no es un invariante hasta que existe una forma de verificarlo; «ninguna columna aparece dos veces en el esquema» sí lo es.
3. **Declarar los modos de fallo que el contrato no puede ver.** Un contrato que promete más de lo que verifica es peor que no tener contrato: da una falsa sensación de control. El C-01 declara que no detecta variables posteriores al tratamiento; si no lo declarara, un equipo daría por revisada la identificación cuando solo revisó la ortografía.
4. **Escribir la prueba de aceptación.** El procedimiento que decide si una implementación cumple. Debe ser ejecutable: un script que corra en CI, un comando que cualquiera

puede correr, un conjunto de corrupciones inducidas que produzcan fallos con mensajes esperados.

5. **Versionar el contrato.** Cuando cambia un invariante, es otro contrato. El registro (Capítulo 4) conserva la historia.

El resultado es que una convención que vivía en la memoria de una persona se convierte en un artefacto que una máquina ejecuta y que cualquier persona nueva puede leer. El costo es la fricción inicial de escribirlo; el beneficio es que el conocimiento deja de caducar.

3.7 Anti-patrones del C-01

En la práctica, los equipos que adoptan el C-01 sin entender su propósito cometen errores característicos:

El esquema espejo. Copiar los nombres de las columnas de la tabla y llamarlo esquema, sin decidir roles. Un espejo no decide nada: si el origen renombra una columna, el espejo la renombra también y el fallo pasa igual. El C-01 existe para que los roles se decidan una vez y no se redecidan por accidente.

El esquema tentáculo. Declarar el esquema en cinco lugares — un YAML, un docstring, un README, un comentario en el pipeline y la cabeza de alguien — y esperar que los cinco coincidan. El C-01 exige unicidad: un solo lugar, versionado, que viaja con los datos.

El esquema confirmatorio. Construir el esquema después de mirar los resultados, eligiendo los roles que hacen que el efecto salga mejor. El estimando se declara antes de mirar (Cap. 1); el esquema es la traducción de esa declaración a nombres de columna. Si se construye después, no es un esquema: es una justificación.

El esquema sin prueba. Tener el esquema pero nunca correr la validación. El C-01 sin su prueba de aceptación es una convención disfrazada de contrato — más peligrosa que la convención honesta, porque parece auditable y no lo es.

El esquema que verifica causalidad. Pedirle al C-01 que valide que las covariables son legítimas. El contrato no puede: solo ve nombres. La afirmación causal la audita el C-03 con marcas de tiempo, y la teoría causal la decide el grafo. Un C-01 que parece validar causalidad está mintiendo.

3.8 Los contratos y el resto del libro

El C-01 no está solo: es el primero de doce. Los demás lo consumen o lo extienden, y esa dependencia es deliberada:

Contrato	Qué añade	Qué exige del C-01
C-03 auditoría temporal	Marcas de tiempo por covariable	El esquema declara los roles
C-09 registro y linaje	Hash, versión inmutable, champion	El esquema viaja con la tabla
C-10 partición y pared	Semilla, cobertura, la pared	El esquema define las columnas a particionar
C-12 exigencias del estimador	Nuisances OOS, ESS, trimming	El esquema alimenta el bundle

La lectura es: sin C-01, los demás no tienen qué validar. Un C-03 sin esquema no sabe qué covariable auditar; un C-09 sin esquema no sabe qué tabla hashear; un C-10 sin esquema no sabe qué columnas particionar. El C-01 es la base porque declara la estructura que todo lo demás presupone.

3.9 El esquema como artefacto de identificación

Hay una lectura del C-01 que va más allá de la ingeniería: el esquema no es solo una interfaz de datos — es un **artefacto de identificación**. Cuando alguien declara que una columna es una covariable, está afirmando que esa variable puede controlarse legítimamente, es decir, que satisface el criterio de puerta trasera. Esa afirmación es causal, no técnica, y el esquema la registra en un formato que una máquina puede validar.

La distinción importa porque tiene consecuencias operativas. Si el esquema es solo una interfaz, cambiar un rol es una decisión de ingeniería: se actualiza el YAML, se versiona, se recompila. Si el esquema es un artefacto de identificación, cambiar un rol es una decisión **científica**: cambia qué estimando se computa, qué población cubre y qué supuestos se sostienen. Un equipo que trata los roles como decisiones de ingeniería puede cambiarlos sin pensar; un equipo que los trata como decisiones de identificación se detiene antes y pregunta: **¿qué afirmación causal estoy cambiando?**

El C-01 captura esa distinción en su invariante (6): el esquema no se modifica una vez creado. Cambiar los roles produce **otro** esquema — y ese otro esquema es otra pregunta causal, con otro estimando y otra población. El registro (Capítulo 4) conserva la historia: nadie puede cambiar la pregunta sin que quede registrado.

3.10 El diseño del esquema en la práctica

El paso de la convención al contrato tiene un procedimiento que se repite en cualquier contexto:

Identificar los roles que la pregunta exige. La pregunta causal del Capítulo 1 determina qué roles se necesitan: si es un ATE, se necesita tratamiento, resultado y covariables; si hay instrumentos, se añaden; si la pregunta cambia, los roles cambian con ella. El esquema no se diseña en abstracto: se diseña para una pregunta.

Elegir las columnas que llenan cada rol. Aquí es donde la teoría causal entra: qué covariables son legítimas, cuáles son posteriores al tratamiento, cuáles son instrumentos. El C-01 no decide: registra la decisión. Pero al registrarla, la hace auditable — alguien más puede leerla y discrepar.

Verificar contra la tabla. El esquema que declara una columna inexistente no es un esquema: es un deseo. La validación contra la tabla real es el paso que separa la intención del hecho. El caso canónico muestra lo que pasa sin esa validación: dos análisis leen la misma tabla y obtienen efectos distintos.

Versionar y registrar. El esquema válido se registra con hash de la tabla a la que aplica. Cuando la tabla cambia — reextracción, filtro de negocio, cambio en el origen — el hash deja de coincidir y el registro lo detecta. Sin esa detección, el esquema de ayer se aplica a la tabla de hoy sin que nadie lo note.

3.11 El fallo que el esquema previene y el que no

El caso canónico muestra el fallo que el C-01 existe para impedir: dos análisis discrepando sobre los roles sin que nada lo detecte. Pero hay otro fallo que el C-01 **no** impide, y saber distinguirlos es la diferencia entre usar el contrato bien y usarlo como decoración.

El fallo que previene: **la deriva semántica silenciosa**. Sin esquema único, cada análisis define sus roles en su propio código, y dos definiciones pueden divergir sin que nadie lo note. El esquema único cierra esa puerta: hay un solo lugar donde los roles se deciden, y todo lo demás lo consume.

El fallo que no previene: **la deriva causal silenciosa**. Aunque los roles estén escritos y compartidos, pueden estar mal: una covariable declarada legítima puede ser un collider, una variable declarada pre-tratamiento puede ser post. El C-01 no lo ve porque solo lee nombres. Lo ve el C-03 con marcas de tiempo, y lo decide el grafo. El C-01 es necesario pero no suficiente: garantiza que la discusión sobre roles ocurra en un solo lugar, no que la discusión tenga razón.

3.12 Ejercicios y notas bibliográficas

3.12.1 Ejercicios conceptuales

1. Para el dataset de becas de la Sección 1.1, proponga el esquema C-01 completo (tratamiento, resultado, al menos tres covariables legítimas) y una columna que sería incorrecto incluir por posterior al tratamiento. *Pista:* una calificación medida después de otorgar la beca no puede ser covariable, sí puede ser resultado.
2. En el caso canónico, suponga que el efecto de la edad sobre el salario pasa a 0. Dibuje el grafo y diga qué estima ahora el segundo análisis y por qué desaparece el sesgo. ¿Qué dos arcos hacían falta para que controlar S sesgara? *Pista:* el collider necesita la entrada de E a S y la salida de E a Y.

3. La regla del esquema dice «un solo rol por columna». Dé un ejemplo de un instrumento que alguien incluyó también como covariable y explique qué va mal en cada uno de los dos errores posibles. *Pista:* si el instrumento entra como covariable el estimador corre y devuelve un número; el fallo es silencioso igual que el del tratamiento repetido.
4. La validación (a) pasa sobre `observed_train.csv` y falla con un mensaje que nombra `educ` en (b). El equipo decide «arreglar el nombre en el pipeline» en vez de tocar el esquema. ¿Qué invariante del C-01 protege esa decisión, y por qué es la respuesta correcta aunque el esquema no haya cambiado? *Pista:* (1) y (6): el esquema describe roles, no ortografía del origen; el punto del fallo es informar antes de estimar.
5. Escriba la prueba de aceptación conceptual de un nuevo fallo: una tabla donde el resultado llega con el tratamiento desplazado una fila. ¿Qué operación del contrato lo atrapa y con qué mensaje? Si ninguna, ¿por qué no puede el C-01 verlo? *Pista:* desplazar filas no cambia nombres ni únicos: el fallo es de alineación, no de esquema; lo vería el registro por hash (Capítulo 4), no el validador de roles.
6. Su equipo tiene tres tablas con nombres de columnas distintos para el mismo concepto (`educ`, `educacion`, `years_of_educ`). Proponga cómo manejar esto sin crear un esquema tentáculo, y diga qué gana y qué pierde su propuesta frente a unificar los nombres en el origen. *Pista:* el C-01 exige unicidad del esquema, no unicidad de nombres: cada tabla puede tener su esquema, pero cada esquema es uno.
7. Describa los tres pasos del diseño de un contrato (identificar la convención, escribir invariantes comprobables, declarar límites) aplicados al C-09 (registro con hash) en vez del C-01. ¿Qué cambia y qué se mantiene? *Pista:* el procedimiento es el mismo; los invariantes y los fallos cambian.
8. Un colega dice: «el esquema es solo un YAML que documenta la tabla, no tiene efectos reales». Dé dos ejemplos concretos del capítulo donde el esquema produce un comportamiento verificable que el YAML sin contrato no produciría. *Pista:* la validación que nombra la columna culpable y la inmutabilidad que impide cambiar roles sin versión nueva.

3.12.2 Notas bibliográficas

La fuente del marco datación-vs-contrato de este capítulo es la tradición de pipelines robustos de (Kleppmann 2017) (esquemas explícitos como frontera de compatibilidad) y (Reis y Housley 2022) (calidad como calidad-de-decisión, no como limpieza). La dependencia de configuración oculta que el esquema único cierra es deuda de datos en (Sculley et al. 2015). La distinción entre controles legítimos y malos controles, que el contrato registra y la auditoría temporal verifica, se formaliza en la edición de método de este libro (criterio de puerta trasera; (Bareinboim 2025)). El ejemplo de la experiencia potencial viene de (Card 1995) y es la advertencia operativa del libro: una variable «estándar del campo» puede ser exactamente la fuga, como el caso canónico muestra en cifra.

Qué debe ser verdad en los datos

Que el esquema exista y sea único no verifica la causalidad: verifica que dos análisis no pueden discrepar sobre el rol de una columna sin que alguien lo corrija en un solo lugar. Lo que debe ser verdad en los datos: un rol por columna, el esquema viajando con la tabla y validado contra ella, y toda operación de lectura del sistema recibiendo esquema + tabla, nunca tabla sola. Se comprueba corriendo la prueba de aceptación de la Sección 3.4 sobre la tabla real y sus corrupciones, antes de permitir la primera estimación; los cuatro casos de la tabla son los únicos que el C-01 promete pillar, y lo dice.

CAPÍTULO 4

4 Versionado, linaje e inmutabilidad

Cuadro sinóptico del capítulo

Problema:	Una afirmación causal es un enunciado sobre un conjunto de datos concreto; si la tabla cambia, el enunciado queda huérfano
Decisión:	Versiones inmutables con hash de contenido, y promoción atómica a través de un alias único (`champion`)
Pieza contratada:	El contrato C-09 (registro de versiones y linaje), con su puerta de integridad
Acompaña:	La ficha de procedencia del tratamiento: quién asignó, con qué información, bajo qué regla
Caso canónico:	Registro de `real/jobs`, mutación detectada y promoción atómica (hashes reales, script con salida verificable)

4.1 El problema: la afirmación y su sujeto

Una afirmación causal es un enunciado sobre un sujeto concreto: *el efecto del programa es de 1 800 dólares, estimado sobre la tabla X con el esquema Y* — y su fuerza depende de que el sujeto siga existiendo y siga siendo el mismo. En un pipeline sin versionado, la frase se dice igual pero el sujeto ya no significa nada: el «mismo» archivo se sobrescribe con una extracción nueva, dos personas leen dos copias llamadas igual, y nadie decide si la estimación publicada la semana pasada corresponde a la tabla de ayer o a la de hoy. El caso más barato es el teorista que dice «estime con los datos de enero» y nadie sabe cuál archivo era; el más caro es el sistema que promueve a producción sobre una tabla `train_latest.csv` que ayer tenía 1 033 filas y hoy 1 041, y la promoción queda sellada contra algo que ya nadie puede reconstruir.

La tradición de los pipelines robustos entiende la solución como inmutabilidad: lo que entró no se edita, se sustituye por una versión nueva que deja la anterior intacta. (Kleppmann 2017) la defiende como el principio de los logs de eventos, y (Reis y Housley 2022) la traslada al ciclo de vida del dato. Este libro la exige con una razón adicional, que los dos no tratan: **el enunciado causal es más frágil que el predictivo**. Una predicción sobre una tabla que

cambió es algo obsoleto pero sigue existiendo; un efecto estimado sobre una tabla que cambió ya no es la misma comparación: cambió posiblemente su mecanismo de asignación, su tiempo y su soporte, los tres platos de la identificación. Por eso el versionado de datos, aquí, es una prueba de identificación además de una higiene de pipeline.

4.2 El hash de contenido: la identidad de un dataset

La herramienta mínima del versionado es el **hash de contenido**: una función que toma el archivo y devuelve una larga cadena hexadecimal, con la propiedad de que cambiarle un byte cambia la cadena. Este libro usa SHA-256 por convención, y la regla operativa es la de los *content-addressable storage* de la ingeniería de datos: **el nombre de una versión de datos es su hash, no una fecha ni un adjetivo del equipo**. Una fecha engaña («enero» existe dos veces, si se reextrajo el proceso); un hash no.

El benchmark del repositorio de referencia lo hace literal: en `splits.json` del escenario `real/jobs` el productor publica, por cada partición, el `file_sha256` del archivo y un `row_id_sha256` de los índices involucrados, más el bloque `validation` con `covers_all_rows: true` y `no_overlap: true`. Es una promesa escrita: el consumidor recalcula y comprueba. Sobre los archivos reales, ambos cálculos coinciden byte a byte (el entrenamiento, `cd5a772ead64f4164c…`, es lo que el productor declara y lo que el consumidor recomputa), y si algún byte cambiara en camino, la rectificación lo detecta sin análisis de contenido alguno: la identidad de un dataset es una sola línea.

4.3 La ficha de procedencia del tratamiento

El hash le dice a un equipo qué tabla está describiendo; no le dice nada sobre cómo esa tabla llegó a la forma que tiene. La pieza de procedencia que este libro exige, inspirada en la lectura de los datos de entrenamiento como diseño antes que como insumo ((Huyen 2022), Cap. 4), es la **ficha de procedencia del tratamiento**:

1. **Mecanismo de asignación**: ¿la asignación fue aleatorizada, cuasi-aleatorizada (un umbral administrativo, una lotería institucional) u observacional pura — la decisión histórica de personas o reglas?
2. **La información del asignador**: ¿qué sabía quien asignó, y coincide con las covariables observadas? Si el asignador usó algo que no se midió, lo dota todo: la «no confusión condicional» es un supuesto sobre ese punto exacto.
3. **La relación temporal**: la asignación del tratamiento es anterior a la medición del resultado, y las covariables declaradas pre-tratamiento lo son de verdad (Capítulo 6).
4. **La regla de elegibilidad**: cualquier corte administrativo que excluyó filas antes de la tabla — geografía, fecha, umbrales — y que determina la población del estimando.

La ficha es un requisito previo a toda estimación, y es parte del registro: cuando cambia el origen — otra regla de elegibilidad, otro canal de asignación —, el estimador vigente pierde su procedencia válida aunque su código no haya cambiado, y el registro debe reflejarlo. La extensión observacional vs experimental del escenario `real/jobs` es el ejemplo corriente: la

misma pregunta corre sobre una submuestra aleatorizada y sobre una observacional, y la ficha de procedencia es exactamente lo que las distingue (este contraste fundó la literatura moderna de evaluación de programas, de (LaLonde 1986) y (Dehejia y Wahba 1999), que el Cap. 2 recorre como traza).

4.4 El contrato C-09: registro de versiones y linaje

Contrato C-09 · Registro de versiones y linaje

Operaciones	*registrar*: crea una versión inmutable con manifiesto, a partir del artefacto validado (estimación + ficha), citando el hash del conjunto de datos; `verificar_integridad`: recomputa los hashes de los archivos de una versión y los compara con el manifiesto; *promover*: apunta un alias (típicamente `champion`) a una versión; *resolver*: lee a qué versión apunta el alias.
Entrada	El artefacto estimado y su ficha validada, el hash SHA-256 del conjunto de datos, y la decisión de puerta que acompaña (Parte IV; aquí queda el registro). El hash del código, idealmente el commit del repositorio limpio.
Salida	Una carpeta de versión `vNNNN` con `estimacion.json`, `tarjeta.json` y `manifest.json`; el alias `champion` apuntando a la versión promovida. Leer alias devuelve la versión íntegra.
Invariantes	(1) Las versiones no se editan: registrar crea, nunca sobrescribe; un cambio produce otra versión. (2) El manifiesto de cada versión contiene los hashes de sus archivos y el dataset con el que estimó. (3) La integridad recomputa y compara; no confía en nada más. (4) El alias se reescribe de forma atómica (archivo temporal y renombrado, nunca edición in-place), de modo que ningún lector ve un apuntador a mitad de cambio. (5) Una versión con puerta `falla` nunca se promueve; una con `alerta` solo si alguien la acepta explícitamente. (6) Todo registro cita además del hash del dato, la semilla y la versión de código con la que se estimó.
Modos de fallo	Alguien edita un archivo ya registrado (la integridad lo detecta en el siguiente chequeo). La tabla de datos es sustituida sin registro (el hash dejó de coincidir). Una promoción que escribe el alias in-place y un lector concurrente recoge un apuntador roto. Registrar sin la decisión de puerta que acompaña. Versiones duplicadas por ruido de formateo externo al contenido.
Prueba de aceptación	(a) Se registra `v0001` y la integridad pasa. (b) Se muta una copia de cualquier archivo de la versión y la verificación falla,

nombrando el archivo. (c) Champion se promueve y resuelve en la versión promovida; el proceso nunca entrega un apuntador parcial. (d) Una versión con puerta `falla` no promociona por más que se invoque. (e) Dos registros en secuencia sobre contenido igual dejan dos versiones históricas, no una sobreescrita.

Caso canónico · El registro y una mutación silenciosa

Con los archivos reales del escenario `real/jobs`, el script del repositorio registra una versión y luego intenta estropearla. El manifiesto de `v0001` queda así — cifras recalculadas, no citadas de memoria:

Archivo	sha256
`observed_train.csv`	`cd5a772ead64f416…`
`observed_validation.csv`	`cee91dab6987ee3c…`
`observed_test.csv`	`2093dbef51d20b43…`
`splits.json`	`101274c23c9c9747…`

El hash del entrenamiento coincide con el declarativo del productor en `splits.json` — la mesa de la verdad entre dos computaciones independientes. Después se inyecta una mutación: dos filas espurias al final del entrenamiento. El hash pasa a `1de3fd53c1baf11b…` y la puerta de integridad del C-09 bloquea la promoción: **los datos mutados no son la versión registrada**. El alias `champion` apunta a `v0001` tras una escritura atómica: archivo temporal y renombrado, nunca edición in-place.

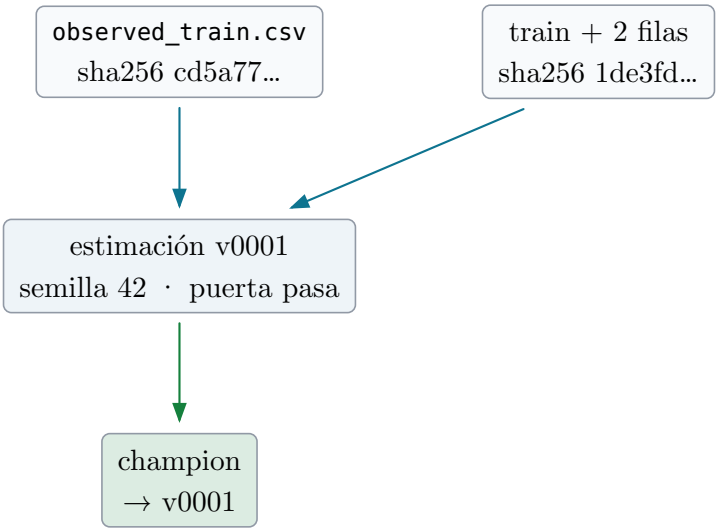


Figura 4.1: El linaje mínimo auditable: cada versión de dataset con su hash alimenta la estimación que la consumió, y el alias queda apuntando a la versión promovida. El mutante de la derecha no llega al alias: la puerta de integridad lo detiene en el registro.

El registro cita también la semilla y el código. La semilla porque la diferencia de medias es una función del azar si el estimador lo tiene; el código como el commit del árbol limpio — si hay cambios sin confirmar, el registro no cita ningún hash de código y la reproducibilidad queda deuda (Figura 4.1): la implementación de referencia decide justamente esto con una función de «commit limpio» que solo informa el hash de HEAD si el árbol está limpio.

Repositorio de referencia del libro

Registro local de referencia (versiones inmutables, manifest con hashes, alias atómico `champion`, sin MLflow): `04_produccion_causal/src/causalops/registro_local.py`. Caso reproducible: `Causal_AI/scripts/cap04_linaje.py` (con los hashes reales del escenario). La promoción, la puerta y la ficha son contratos de la Parte IV: aquí queda que el registro los consume, no los decide.

4.5 Ejercicios y notas bibliográficas

4.5.1 Ejercicios conceptuales

1. Un equipo renombra `train_v3.csv` a `train.csv` «porque downstream espera ese nombre». ¿Qué rompe, si produce una nueva versión de la tabla y por qué el hash del registro no es el mismo que el esperado por el promotor de champion? *Pista:* el alias responde a una versión íntegra; el nombre del archivo no es su identidad.
2. Explique por qué el hash del entrenamiento de `real/jobs`, recalculado por dos procesos independientes, tiene que coincidir para que el canal productor-consumidor funcione. ¿Qué falla si los dos procesos recalculan sobre archivos «iguales» pero con distinto orden de filas? *Pista:* el orden de filas es contenido: el hash los distingue; la equivalencia semántica queda con el productor, que ordena de forma determinista.
3. La mutación del caso canónico agrega dos filas al final. Proponga una mutación más sutil que pase el hash del entrenador pero rompa la estimación de igual manera: ¿quién la detecta — el C-09, el C-01, o nadie? *Pista:* si se muta solo `splits.json` dejando la tabla intacta, el linaje deja de ser el del registro.
4. La ficha de procedencia dice `mecanismo: observacional administrativo`. El equipo quiere una frase que suene mejor y escribe `cuasi-experimental`. ¿Qué cambia, si todo lo demás queda igual? *Pista:* la frase es una afirmación sobre la información del asignador; si miente, la puerta no la detecta y el estimando cambia de supuesto. De ahí que la procedencia sea una declaración auditada, no una etiqueta.
5. ¿Es correcto versionar las particiones (Capítulo 5) dentro del mismo registro C-09, o el benchmark bien pudo haberlo dejado adyacente en `splits.json`? Defienda los dos diseños con las exigencias del contrato. *Pista:* el hilo es único e inmutable también si la tabla y su partición viajan juntas; la separación en dos archivos agrega una arista de linaje que hay que auditar igual.

4.5.2 Notas bibliográficas

La inmutabilidad como fundamento de pipelines confiables es de (Kleppmann 2017) (logs y store append-only) y (Reis y Housley 2022) (ciclo de vida del data lake). Ledger de contenido direccionable por hash: la tradición de los content-addressable storage, aquí aplicada al dataset como artefacto auditable. La contraposición experimental/observacional del escenario de escuela LaLonde es de (LaLonde 1986) y (Dehejia y Wahba 1999), inmersa en este libro como traza permanente (Capítulo 2). La lectura de los datos de entrenamiento como diseño en vez de insumo es de (Huyen 2022), Cap. 4. El argumento de que procedencia y promoción son una misma pieza de linaje es la contribución propia del capítulo: los manifiestos del benchmark lo instancian, y `registro_local.py` los implementa.

Qué debe ser verdad en los datos

Que todo número publicado del sistema responda a una versión con hash del dataset, una semilla y un commit limpio, con el alias `champion` como único punto de promoción — reescrito de forma atómica, sin edición in-place. Que la ficha de procedencia del tratamiento nombre el mecanismo de asignación, la información del asignador, la relación temporal y la regla de elegibilidad, y que una mutación del archivo quede detectada por la puerta de integridad, como en el caso canónico. Lo que no se prueba aquí: que la versión promovida siga siendo válida en sus supuestos — eso es trabajo de los contratos de la Parte IV; el registro solo garantiza que nadie intercambió el sujeto del enunciado.

PARTE I

El pipeline que preserva la identificación

Esta parte recorre el pipeline pieza por pieza: la partición de los datos y la pared que separa lo observado de lo reservado para evaluación, y la validez temporal de las variables —qué puede entrar a una matriz de covariables y qué no, decidido por marcas de tiempo y roles, no por poder predictivo. Cada pieza se presenta como un contrato con su prueba de aceptación, y cada afirmación numérica de los casos canónicos se regenera con un script con semilla del repositorio de referencia.

CAPÍTULO 5

Particiones y la pared observado/evaluación

Cuadro sinóptico del capítulo

Problema:	Evaluar es comparar con algo que el pipeline no debe tocar: la prueba es un recurso finito que deja de mentir o empieza a mentir en la primera mirada
Decisión:	La partición es una estructura de datos versionada, sembrada y con lado de la pared asignado a cada fila
Pieza contratada:	El contrato C-10 (partición versionada): semilla, método, disjuntividad y cobertura total
La pared:	Lo observado decide; lo reservado — prueba y verdad — mide una vez
Caso canónico:	Reproducibilidad por semilla (misma huella) y fuga por selección sobre la prueba (R^2 inflado 7 veces, script con semilla)

5.1 El problema: evaluar es comparar contra lo intacto

Todo número sobre el desempeño de un sistema presupone una comparación: el sistema contra lo que habría pasado sin él, el modelo nuevo contra el vigente, el estimador contra su variante barata. Para que esa comparación signifique algo, el juez — el conjunto de evaluación — no puede haber participado en las decisiones que se juzgan: ni en la selección de hiperparámetros, ni en la elección del estimador, ni en la decisión de promover. La exigencia parece obvia y se viola de forma rutinaria, porque las miradas no cuestan nada y la prueba es tentadoramente accesible: se echa un vistazo «solo para comprobar», se ajusta un umbral, se vuelve a mirar, y en la tercera iteración el número que la prueba reporta ya no mide al sistema: mide cuánto se le acopla el equipo a esa muestra concreta.

En ciencia de datos predictiva esto es folklore — el conjunto de prueba se usa una vez, al final, y antes de eso se divide entrenamiento y validación para decidir. Este libro lo trata como

un contrato por dos motivos. Primero, porque en un sistema causal la violación es más cara: la comparación de la (1.1) solo vale si el lado de la comparación no fue seleccionado por quien mide. Segundo, porque el sistema sigue operando tras la primera evaluación, y el conjunto que hoy es «prueba» será mañana materia de un modelo nuevo: el recurso finito no se repone, y hay que gobernarlo (Capítulo 2, las prácticas del bucle). La partición, entonces, no es un paso de muestreo al inicio del proyecto: es una **estructura de datos con dueño, semilla y versión**.

5.2 La partición como estructura de datos

Las tres piezas de una partición honesta son el procedimiento, la semilla y la asignación de lados de la pared:

1. **Los bloques y su rol**: `entrenamiento` para ajustar, `validación` para decidir (hiperparámetros, umbral de promoción, modelo candidato), `prueba` para medir una vez. Los roles deben declararse; el error clásico es usar la validación «una vez más» para lo que solo la prueba debía tolerar.
2. **El método**: aleatorio simple, o estratificado por la variable que no puede quedar desbalanceada — aquí, por tratamiento, de modo que cada bloque preserve la fracción $P(D = 1)$. El método es una decisión del productor: con tratamientos raros o resultados raros, la aleatorización simple crea bloques sin casos y la partición se vuelve inútil para el estimando.
3. **La semilla**: lo que hace al sorteo reproducible. La semilla es parte del contrato, no una configuración descartable: sin ella, ya nadie sabe si la partición de ayer y la de hoy son la misma, y la evaluación consecutiva deja de ser continua.

El benchmark del repositorio de referencia lo lleva literalmente escrito: en `splits.json` del escenario `real/jobs` el productor declara `split_seed: 20240701`, el método `stratified_by_treatment_60_20_20`, los bloques con tamaños 1 033 / 344 / 345, un `file_sha256` y un `row_id_sha256` por bloque, y un bloque `validation` que certifica `covers_all_rows: true` y `no_overlap: true` (Capítulo 4). La partición viaja así como una versión más del dataset: el consumidor recalcula y verifica, y las dos condiciones — todo fila cubierta exactamente una vez, ninguna fila en dos bloques — son lo primero que el contrato de más abajo exige comprobar.

5.3 El contrato C-10: partición versionada

Contrato C-10 · Partición versionada

Operaciones	*dividir*: reparte una tabla en bloques con proporciones y método declarados; *describir*: emite los metadatos de la partición — semilla, método, estratificación, tamaños, hashes por bloque; *verificar*: comprueba disjuntividad y cobertura exacta contra la

	tabla; *etiquetar*: asigna un lado de la pared a cada bloque — qué puede decidir y qué solo mide.
Entrada	La tabla (con su hash, contrato C-09), la semilla, las proporciones por bloque y el método — en particular con qué variables estratificar.
Salida	Los índices de cada bloque y el documento de metadatos versionado junto al dataset. La verificación pasa o falla nombrando la fila en conflicto.
Invariantes	(1) Dada la tabla y la semilla, la partición es determinista: dos invocaciones producen bloques idénticos. (2) Los bloques son disjuntos y su unión es la tabla completa: toda fila cae exactamente en un bloque. (3) La partición viaja con el hash de la tabla: sobre otra tabla, es otra partición. (4) El lado derecho de la pared no entra en ningún ajuste ni decisión; la medición de la prueba se hace una vez. (5) Cambiar la semilla produce otra partición, registrada como cambio de versión y nunca como «la misma». (6) La estratificación se declara y no se elige mirando el resultado.
Modos de fallo	Reusar la semilla ante cada dato nuevo «porque así salía la partición buena»; probar semillas hasta obtener una partición favorable, que es mirar la prueba con más pasos. Un `join` defectuoso que solape una fila en dos bloques sin ninguna advertencia. Separar por tiempo y luego barajar internamente, rompiendo la temporalidad (Capítulo 6). Validación usada tres veces hasta quedar, de facto, entrenamiento.
Prueba de aceptación	(a) La misma semilla dos veces produce la misma huella de bloques; otra semilla, otra huella (caso canónico A). (b) `verificar` confirma disjuntividad y cobertura sobre la tabla real (caso canónico A, y el `validation` de `splits.json`). (c) Con otra tabla, la misma semilla produce particiones distintas y el documento lo refleja. (d) La selección sobre la prueba queda medida: el ejercicio del mejor-de-100 produce el orden de inflación del caso canónico B.

5.4 La pared: lo observado y lo reservado

La partición reparte las filas; la **pared** decide qué lado de la casa toca el pipeline cuando está decidiendo. Del lado izquierdo queda lo observado — entrenamiento, validación, todo lo que puede inspeccionar, graficar y usar para elegir modelo y umbral. Del lado derecho queda lo reservado: la prueba de uso único, y la verdad del DGP cuando la hay — y_0, y_1, τ —, existente solo para evaluación final.

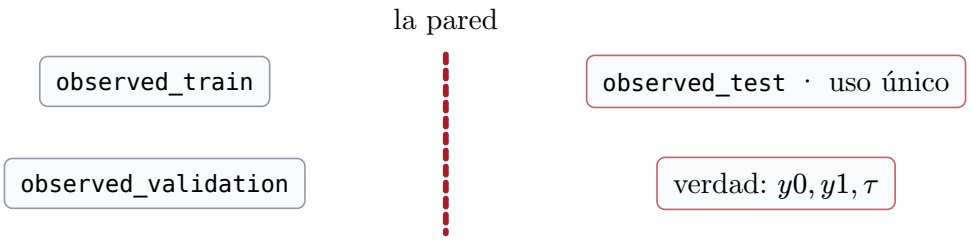


Figura 5.1: La pared del gobierno del dato: el lado izquierdo lo tocan todas las decisiones del pipeline; al lado derecho solo entra la evaluación, una vez. Las columnas de verdad (y_0, y_1, τ) existen en el DGP para medir el prototipo: entrar al entrenamiento, al servicio o a cualquier decisión intermedia es la fuga que el contrato existe para impedir.

La pared no es paranoia: es la estructura sin la cual un sistema «aprende» de su propia evaluación y empieza a reportar un mundo que no existe. En el benchmark la aplica el productor — la verdad vive en archivos `truth_*` que ningún flujo público consume, y el repositorio lo hace cumplir también con chequeos sobre el árbol de trabajo — y en producción tiene dos análogos directos, ya mencionados con el bucle de la figura 2.1: el **holdout permanente** (el grupo que el modelo no gobierna y que permite seguir midiendo el efecto mañana) y el **piso de exploración** (la fracción aleatoria que mantiene soporte común bajo política propia). Las dos son paredes para el dinero del sistema en operación, no solo para el prototipo.

5.5 Caso canónico A: la partición es una función de la semilla

Caso canónico · Reproducir la partición

Con la semilla 20240701 del benchmark y las proporciones 60/20/20, el script del repositorio divide las 1 722 filas y calcula la huella SHA-256 de los conjuntos de índices. La misma semilla, invocada dos veces, produce la misma huella: `cc477d0ca7defb98…` las dos veces. Con la semilla 20240702, la huella pasa a `7d626f58161184f9…`. Los bloques resultan 1 033 / 344 / 345 — los mismos tamaños que declara `splits.json` — sin solape alguno entre entrenamiento y prueba.

La lectura operativa: la afirmación «la evaluación de la semana pasada» solo tiene sentido porque la partición es una función determinista de la semilla y la tabla. Cambiar cualquiera de las dos — una extracción nueva de la tabla, una semilla distinta — y la comparación entre semanas ya no es sobre el mismo juez.

5.6 Caso canónico B: elegir mirando la prueba

Caso canónico · El mejor de 100 modelos de ruido

Tómenese 100 modelos triviales, cada uno con una sola covariable generada como ruido puro estándar, sin relación ninguna con el resultado — no hay señal que explicar en

ninguno. Ahora el equipo puede ser inocente o deshonesto con la prueba. La ruta inocente: reporta el R^2 medio de los 100 sobre la prueba: **0,0011**. La ruta deshonesta: elige el mejor de los 100 mirando esa misma prueba y lo presenta como «el modelo»: su R^2 sobre la prueba es **0,0109**, casi siete veces. La prueba honesta de ese «mejor» modelo — columnas nuevas de la misma familia de ruido, sin selección alguna — vuelve a dar **0,0016**.

Las tres cifras (semilla 20240701, script incluido) son la varianza del azar, nada más; el punto no es cuánto infla en este caso, sino que la inflación no lanza ningún error: la prueba sigue devolviendo números con total normalidad, con su gráfico y su tabla, exactamente igual que antes de ser usada tres veces. La única línea real de defensa es el invariante (4) del C-10: lo reservado mide una vez, y su documento de metadatos registra cuándo y por qué se midió. Cualquier evaluación posterior usa otra partición — o renuncia al calificativo de evaluación.

Repositorio de referencia del libro

Caso completo: `Causal_AI/scripts/cap05_splits.py` (semilla 20240701). Las particiones públicas del benchmark, con hash por bloque: `data_benchmark_causal/causal_benchmark_synthetic/datasets/real/jobs/splits.json` (y sus homólogos `ihdp`, `ihdp_medium`, `latent_confounder`, `10_vars_10_edges`). El registro de versiones que aloja la partición como versión del dataset: `04_produccion_causal/src/causalops/registro_local.py`.

5.7 Ejercicios y notas bibliográficas

5.7.1 Ejercicios conceptuales

1. Un equipo ajusta el umbral de promoción con la validación y, «por completar», vuelve a mirar la prueba para confirmar que aguanta. ¿En qué invariante del C-10 quedó roto el proceso, aunque nadie cambió ningún archivo? *Pista:* (4) y (5): la mirada ya es decisión; si la prueba ajusta el umbral, la partición deja de ser la registrada.
2. Explique por qué estratificar por tratamiento (en vez de aleatorizar de forma simple) es aquí una decisión de productor y no una mejora de desempeño. ¿En qué escenario del benchmark sería obligatoria y por qué? *Pista:* piense en `latent_confounder` o cuando $P(D = 1)$ caiga cerca de 0, o de 1: un bloque sin tratados no identifica nada.
3. Proponga la pared de un sistema en producción mes a mes: ¿dónde va el holdout permanente, dónde el piso de exploración y qué rol cumple el registro de asignación en cada uno? *Pista:* repase la sección del bucle del Cap. 2 una partición de uso único no responde a un sistema que sigue operando.
4. El equipo decide separar por fecha (todo lo previo a junio entrena, lo posterior evalúa) y luego barajar cada bloque «para evitar sesgos». ¿Qué ocurre con la validez temporal

de features y con la interpretación del estimando tras el barajado? *Pista:* la partición temporal no es una partición aleatoria; la versión del Capítulo 6 la obliga, y el barajado convierte una pregunta sobre el futuro en una sobre el pasado.

5. En el caso canónico B, alguien señala que «100 modelos es mucho; con 3 modelos de verdad no pasaría». ¿Tiene razón, en qué medida? Calcule la inflación esperada del mejor-de-3 bajo ruido puro y discuta por qué la regla del C-10 sigue sin admitir la mirada. *Pista:* el punto no es el tamaño de la tasa sino que el número deja de medir generalización desde la primera selección.

5.7.2 Notas bibliográficas

La higiene de entrenamiento, validación y prueba, y la lectura del muestreo como diseño, son de (Huyen 2022) (Cap. 4) y del esquema de pruebas de producción de (Breck et al. 2017) — este capítulo los eleva a contrato versionado. La fuga por mirar la evaluación es la «plumbing» deuda de datos de (Sculley et al. 2015) y el caso operativo que motiva la regla de uso único. El bucle sistema-mundo que obliga al holdout permanente y al piso de exploración se formaliza como predicción performativa en (Perdomo et al. 2020), y su lectura para sistemas que deciden está en el Cap. 2. El tratamiento de la partición como artefacto versionado con hash es la contribución propia del capítulo, en la línea de (Kleppmann 2017) y (Reis y Housley 2022) de tratar artefactos de datos como código, junto al contrato C-09.

Qué debe ser verdad en los datos

Que la partición exista como estructura declarada — semilla, método, bloques, lado de la pared — versionada con el dataset, determinista dada la semilla, disjunta y completa, y que nada del lado reservado entre a ajustar, decidir o «confirmar por las dudas». Que las columnas de verdad (y_0, y_1, τ) existan solo para evaluación, y que en producción el análogo sea el holdout permanente con su registro de asignación. Se comprueba corriendo la doble prueba del caso canónico: la partición se reproduce por huella (A), y la inflación por selección queda medida y fuera de cualquier decisión (B). Que estas garantías hacen de la comparación una medición honesta es verdad; que las hagan **causal** no: la causalidad se la da el estimando declarado (Cap. 1) y los supuestos que el C-01 registra.

CAPÍTULO 6

6 Validez temporal y control de fuga

Cuadro sinóptico del capítulo

Problema:	Una *feature* que mejora el ajuste puede destruir el efecto: a más R^2, peor estimación, si entra por la ventana del tiempo
Decisión:	Auditar cada candidata contra su momento de medida y su rol causal antes de incluirla, no solo contra su poder predictivo
Pieza contratada:	El contrato C-03 (auditoría de *features*): dos especificaciones, X_{base} y X_{ampliada}, y la regla conservadora
Caso canónico:	Efecto verdadero 1,00: de 1,55 sin controles al 0,00 con todas las variables, con R^2 subiendo de 0,69 a 0,99 (script con semilla)
Conexión:	Cierra el límite honesto del C-01 (Capítulo 3): valida lo que el esquema solo declaraba

6.1 El problema: la columna que predice bien

La ingeniería de *features* estándar evalúa una variable candidata con un solo criterio: ¿mejora el desempeño predictivo fuera de muestra? Con ese criterio, toda variable disponible al entrenar y correlacionada con Y es bienvenida — y es exactamente por ahí por donde entra la fuga. En un problema causal el rol de la variable no se deduce de cuánto predice sino de **cuándo se midió y qué es en el grafo**: una feature puede reducir el error y a la vez ser un mediador, un collider o un registro tardío del resultado, y cada una de esas entradas vacía o sesga el coeficiente del tratamiento sin reducir una sola métrica de ajuste. El Cap. 3 lo dejó escrito como límite: el esquema C-01 verifica nombres, no causalidad; la verificación empieza aquí, y empieza por el tiempo.

En un sistema que opera, el momento importa dos veces. Al entrenar, una variable medida después del tratamiento filtra información del resultado y destruye la identificación. Al servir, además, el momento impone otra frontera, puramente operativa: la preparación de variables solo dispone de lo que es sabido en el instante de decidir, y una *feature* que se computa con

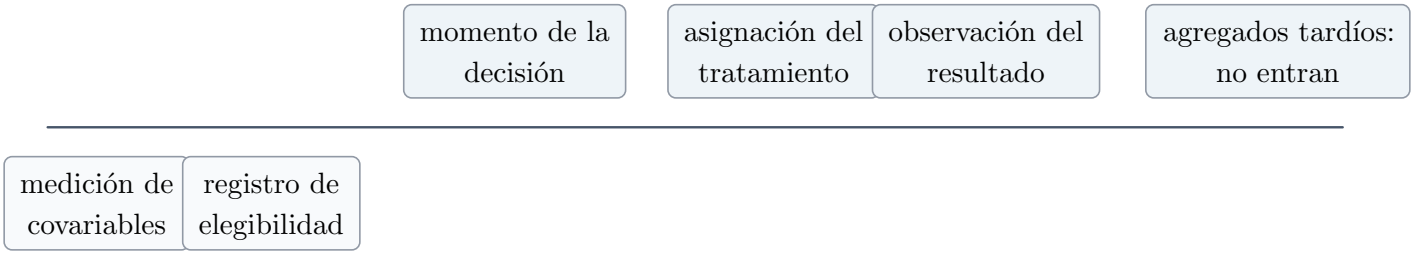


Figura 6.1: El eje del tiempo y la validez de las variables. Solo lo verificable como anterior al momento de la decisión puede entrar en una pregunta de producción; y solo lo anterior al tratamiento puede entrar como control en las dos especificaciones (X_{base} y X_{ampliada}). Los agregados tardíos quedan afuera por las dos puertas.

datos posteriores simplemente **no existe** en producción (o existe con otro valor): es el clásico *training-serving skew*, y su versión causal es peor, porque la estimación retrospectiva que la motivó era causalmente inválida antes de parti la cola. Las dos fronteras se describen con una sola idea que el capítulo opera: la **validez temporal**.

6.2 Qué significa «antes»: validez temporal y el as-of

La validez temporal exige que toda *feature* del pipeline responda con precisión dos preguntas:

1. **¿Cuándo se midió?** Contra un timestamp **de medición**, no de carga. La marca «fecha de ingreso a la base» no es la marca «fecha en que el evento ocurrió», y confundirlas es de las fugas más comunes de data warehouse real.
2. **¿Cuándo se decidió?** La ventana lícita de lo observable se cierra en el instante de la decisión que el sistema apoya — para el servicio, el momento de la consulta; para el entrenamiento retrospectivo, una fecha de decisión declarada por fila, no una fecha global conveniente.

El mecanismo que lo hace cumplir en pipelines, en la literatura de *feature stores*, es el **as-of join** (o *point-in-time join*): cada fila de entrenamiento se cruza con el historial de la *feature* cortando estricto en su fecha de decisión, nunca en «hoy». El *as-of join* no es una optimización: es la única forma en que una tabla retrospectiva imita el conocimiento real que había en producción cuando habría tocado decidir. Su invariante es el que este capítulo eleva a la tabla de decisión de abajo: **lo que no se puede verificar como anterior al momento declarado no entra**.

Momento de medida	Rol declarado	X_{base}	X_{ampliada}
Antes del tratamiento	Confusor	Sí	Sí
Antes del tratamiento	Desconocido	No	Sí
Antes del tratamiento	Mediador, collider o post--tratamiento	No	No
Después del tratamiento, en alguna unidad	Cualquiera	No	No
Sin marca de tiempo, o con faltantes en ella	Cualquiera	No	No

La última fila es la regla conservadora que la implementación hace cumplir: lo no verificable queda fuera. La tabla vale para el **efecto total**: si el estimando cambiara a un efecto directo o mediado, el mediador entraría por diseño y sería otra tabla de decisión —otro estimando, como en la Sección 1.2—. La novedad frente a una ingeniería predictiva es la exigencia de **dos especificaciones**, que la sección siguiente justifica.

6.3 El contrato C-03: auditoría de features

El analista rara vez sabe con certeza el rol de todo candidato; sabe el momento, a veces, y el grafo, con distinto grado de confianza. Si la regla fuera «solo lo declarado confusor», una confusión quedada fuera por cautela se cobra en sesgo; si fuera «todo lo verificable», un mal control entra y se cobra lo mismo. Los dos errores son asimétricos **y ningún criterio automático resuelve esa duda**. La respuesta del contrato es no resolverla: reportarla —con dos especificaciones siempre calculadas y, si difieren de forma material, registradas ambas:

Contrato C-03 · Auditoría de *features*

Operaciones	*auditar*: recibe las candidatas con su rol declarado y su marca de tiempo, junto con la marca del tratamiento, y devuelve una fila por candidata con su decisión de inclusión en cada especificación; *extraer*: devuelve la lista de covariables de una especificación, que pasa al campo de covariables del esquema (C-01) y de la ficha del modelo.
Entrada	Las candidatas con su rol (confusor, mediador, collider, post--tratamiento o desconocido) y su marca de tiempo por unidad, y la marca del tratamiento por unidad. El estimando declarado es el efecto total; para otro estimando cambia la tabla de decisión.
Salida	Una fila por candidata con su rol, la fracción de unidades en que se midió después del tratamiento, si su marca es verificable y las decisiones en X_{base} y X_{ampliada}. Una advertencia por cada candidata con riesgo temporal o de rol ambiguo.

Invariantes	(1) Ninguna candidata se omite en silencio: toda la que aparezca en los roles o en las marcas aparece en la salida, aunque sea para marcarse como no verificable. (2) Lo no verificable como anterior al tratamiento, por falta de marca o por faltantes en ella, no entra en ninguna especificación. (3) Una candidata medida después del tratamiento en alguna unidad no entra en ninguna. (4) Mediador, collider y post-tratamiento no entran en ninguna. (5) X_{base} contiene solo confusores y está incluida en X_{ampliada}, que añade las de rol desconocido. (6) Una candidata sin rol declarado es de rol desconocido. (7) Con los mismos datos y roles, la salida es idéntica.
Modos de fallo	Un rol mal declarado: la auditoría confía en lo declarado, y un collider anotado como confusor entra en las dos. Una marca de tiempo que registra cuándo se cargó el dato y no cuándo se midió: la condición temporal queda cumplida en el papel. Un rol desconocido resuelto por conveniencia. Una sola especificación reportada cuando las dos difieren. Una candidata derivada cuya fórmula usa, aun vía columna intermedia, el resultado medido después.
Prueba de aceptación	(a) Un caso de borde por cada fila de la tabla, más la candidata que solo aparece en los roles y la que tiene faltantes en su marca: cada una cae en su casilla. (b) En el caso canónico de abajo, la auditoría deja $X_{\text{base}} = \setminus \{C \setminus\}$ y $X_{\text{ampliada}} = \setminus \{C, U \setminus\}$, y B, M, K y P quedan fuera de ambas. (c) Las seis estimaciones de la tabla se reproducen, y la brecha entre las dos especificaciones supera con holgura la desviación entre repeticiones.

6.4 Caso canónico: cuatro tipos de feature y una verdad conocida

Caso canónico · El peor modelo es el mejor predictor

Seis variables con verdad conocida, semilla 13: C y U son confusores medidos antes del tratamiento; B es un collider medido antes; M es un mediador, y K y P una medición tardía — respectivamente del sistema y del resultado. El efecto total de D sobre Y es exactamente 1. Con 5 000 unidades y 200 repeticiones, ajustando regresión lineal con cada especificación — y midiendo el R^2 fuera de muestra en la mitad no usada para ajustar:

Controles	Efecto estimado	R^2 fuera de muestra
Ninguno	1,55	0,69
X_{base} (C)	1,35	0,73
X_{ampliada} (C, U)	1,00	0,80
con B (collider previo)	0,88	0,82
con M (mediador)	0,70	0,84
Todas (con K y P)	0,00	0,99

La trayectoria es la moraleja del capítulo en dos columnas: pasar de X_{ampliada} a la selección por ajuste eleva el R^2 de 0,80 a 0,99 y empeora el efecto de 1,00 a 0,00 — porque P es el resultado leído tarde y K medio-sueño en su dirección. En este mundo U es un confusor y la especificación correcta es X_{ampliada}; si fuera un collider, como B, la correcta sería X_{base} (que aquí da 1,35). La brecha entre las dos especificaciones es de 0,35 en un mundo y de 0,09 en el otro, y ambos valores superan la desviación entre repeticiones. Los datos no dicen en cuál estamos: eso lo dice el grafo. Lo que los datos sí dicen, sin ambigüedad, es que ninguna de las dos columnas de la tabla es negociable con el poder predictivo.

Repositorio de referencia del libro

Regeneración de la tabla: `Causal_AI/scripts/cap06_fuga.py` (semilla 13). La función auditora de referencia — recorre candidatas de roles y marcas sin omitir ninguna, trata lo falto de marca como no verificable: reproduce la tabla de decisión con `#code[auditar_features]` y `#code[columnas_X]`. El mismo patrón vive, con datos reales, en los escenarios del benchmark, cuyas covariables declaradas pre-tratamiento se exigen en `identification.json`.

6.5 La regla conservadora y la comparación de especificaciones

Tres consecuencias prácticas cierran el contrato. **Primera: lo no verificable queda fuera.** Entre incluir una variable cuyo momento se ignora y excluirla, el contrato elige excluirla y reportarla: el riesgo de la exclusión (una confusión omitida) se ve en la comparación de las dos especificaciones; el de la inclusión (una fuga) no se ve en ningún lado. **Segunda: la discrepancia es información, no un problema a esconder.** Si X_{base} y X_{ampliada} coinciden materialmente, la ambigüedad de rol no era material y el efecto se reporta sobre la ampliada; si difieren, la identificación tiene que revisarse antes de registrar — la acción *revisar identificación* del vocabulario del Cap. 2 — y las dos estimaciones se registran ambas, porque una ficha que oculta la duda es una ficha falsa. **Tercera: la auditoría confía en el rol declarado, y sabe que lo hace.** Ningún filtro de marcas salva un collider anotado como confusor; por eso el contrato exige que la decisión sobre roles dudosos quede registrada con nombre y razón,

y por eso el *as-of join* no sustituye al grafo: una cosa son las fechas y las formas, y otra la causalidad misma.

6.6 Ejercicios y notas bibliográficas

6.6.1 Ejercicios conceptuales

1. Para el estudio de retornos a la educación, clasifique como candidatas: habilidad cognitiva medida antes de ingresar; puntaje del examen de egreso; cercanía a una universidad; ingreso de los padres. ¿Cuál pasa a X_{base}, cuál a X_{ampliada} y cuál queda fuera? *Pista:* el puntaje de egreso se mide después de gran parte de la escolaridad.
2. Un promedio móvil de 4 semanas del resultado puede ser un control legítimo o una fuga. ¿De qué depende, qué rol le asignaría en cada caso y qué marca de tiempo tiene que tener para pasar la auditoría? *Pista:* si la ventana de la semana t incluye el resultado de semanas en las que el tratamiento ya había cambiado, filtra efecto; la marca debe ser del final de la ventana.
3. En el caso canónico, la última fila tiene $R^2 = 0,99$ y efecto 0,00. Explique, con el grafo, por qué K y P vacían el coeficiente del tratamiento aunque el modelo prediga casi perfectamente. *Pista:* $P \leftarrow Y$ y $D \rightarrow K \leftarrow Y$ abren caminos no causales hasta D: condicionarlos es comparar personas iguales en el resultado, no iguales antes del tratamiento.
4. Un pipeline lambda computa sus agregados en un batch nocturno con corte «hasta ayer a medianoche». Escriba, con las preguntas de la Sección 6.2, el invariante que el *as-of join* debe cumplir para que un evento de las 23:55 entre y uno de las 00:05 no. ¿Qué zona horaria y qué forma de timestamp de medición hace falta registrar para no mentir? *Pista:* la pregunta correcta es «¿cuándo era visible en producción?» — y la medianoche del batch no es la medianoche del evento.
5. La comparación de especificaciones da $\hat{\theta}_{\text{base}} = 0,9$ y $\hat{\theta}_{\text{ampliada}} = 1,4$. Un colega propone «reportar el intermedio 1,15 para no discutir». Interprételo con las tres consecuencias de la Sección 6.5 y con la acción que corresponde del vocabulario de las cinco. *Pista:* 1,15 no es el estimando de ningún plano: la duda es de grafo, no de punto estimado; la acción es revisar identificación y registrar ambas.

6.6.2 Notas bibliográficas

La fuente de la discusión predictiva de la fuga y del *point-in-time correctness* en *feature stores* es (Huyen 2022) (Cap. 5); este capítulo especializa esa higiene al caso causal, donde añade además la distinción de roles y las dos especificaciones. La clasificación de controles legítimos y malos — confusores, mediadores, colliders — se desarrolla con precisión en la edición de método de este libro ((Pearl 2009), (Bareinboim 2025)). El sesgo de post-tratamiento en estudios observacionales, que la tabla de decisión evita mecánicamente, está en (Hernán y Robins 2020). El argumento del contrato de que el rol ambiguo se reporta comparando especificaciones, no

resolviéndolo a la callada, es la regla documentada del capítulo: los números del caso canónico son sus pruebas, reproducibles en el repositorio.

Qué debe ser verdad en los datos

Que toda covariable que entre al estimando tenga marca de tiempo de medición, verificable como anterior al tratamiento y al momento de la decisión; el *as-of join* cortando estricto en la fecha declarada por fila; dos especificaciones siempre calculadas y registradas si difieren; y ninguna candidata omitida en silencio — las que no pasen quedan reportadas. Se comprueba reproduciendo la tabla del caso canónico y, en producción, auditando las advertencias de C-03 contra los timestamps del origen. Lo que esta validez no comprueba: que los roles declarados sean los verdaderos — el auditado temporal y el auditado causal son, deliberadamente, dos cosas distintas, y la segunda sigue siendo trabajo del grafo.

CAPÍTULO 7

Calidad y observabilidad de datos para decisiones

Cuadro sinóptico del capítulo

Problema:	«Calidad» para decisiones no es limpieza: un dato impoluto puede no sostener ninguna comparación causal
Decisión:	Medir comparabilidad (balance), estabilidad (deriva) y completitud (faltantes) como señales del plano de datos, con umbrales calibrados y escritos antes de mirar
Pieza contratada:	El contrato C-02 (comparación de poblaciones), restituido en el plano de datos
Casos canónicos:	Balance: el t-test miente por azar (2,02 rechazos de 40) y Holm lo calla (0,04); PSI/KS sobre la edad real; faltantes 9,6× que la SMD no ve
Conecta:	Las señales de este capítulo son las que el monitor consume en la Parte IV

7.1 El problema: la calidad que importa

Las métricas de calidad de datos estándar — puntualidad, completitud, consistencia, unicidad — son necesarias y no llegan. Contestan si el dato está bien formado; callan en la sola pregunta que este libro hace: ¿sostiene este dato la comparación que el estimando declara? Una tabla impecable por todos esos criterios puede llegar con un desbalance severo entre tratados y controles, con una deriva que la vuelve otra población respecto de la que ajustó el modelo, o con faltantes que mezclan el mecanismo de selección en cada resultado. Ninguna de las tres fallas marca ninguna alarma de limpieza, porque ninguna es una falla de forma: son fallas de **estructura de decisión**, y solo se ven midiendo comparaciones.

Este capítulo instrumenta tres de esas comparaciones como métricas del plano de datos, arrancadas de la discusión de los datos de entrenamiento como diseño ((Huyen 2022), Cap. 4) y de los criterios clásicos de la evaluación de programas ((Rubin 2001), (Imbens y Rubin 2015)). Cada una responde una pregunta distinta: el **balance** — ¿los dos lados del estimando se parecen en lo medible, o la comparación va a apoyarse en extrapolación?; la **deriva** — ¿la población que hoy llega es la misma que la que ajustó el estimador?; los **faltantes diferenciales** — ¿quién desaparece de los datos, y depende de la decisión? Las tres comparten una regla de lectura que es la primera invariante del contrato: **tamaño de efecto, no *p*-valor** — lo que amenaza la comparación es la magnitud del problema, no si se distingue de cero.

7.2 El chequeo de balance

Para cada covariable j, la **diferencia media estandarizada** es

$$\mathrm{SMD}_j = \left(\overline{X}_{j,1} - \overline{X}_{j,0}\right) / \sqrt{\left(s_{j,1}^2 + s_{j,0}^2\right)/2}, \tag{7.1}$$

que no crece con n y no cambia de unidad si la covariable cambia de escala. Los criterios operativos de (Rubin 2001) son $|\mathrm{SMD}_j| < 0,25$ (más estricto, $< 0,1$) y una razón de varianzas $s_{j,1}^2 / s_{j,0}^2$ dentro de $[0,5, 2]$ — y el segundo se olvida con frecuencia: dos grupos con la misma media y dispersiones muy distintas tienen poco soporte común en las colas.

La alternativa que el equipo hará si nadie lo avisa — un t-test por covariable — responde la pregunta incorrecta y la responde mal dos veces: su p-valor mezcla magnitud con tamaño de muestra, y al pavimentar 40 comparaciones al 5 % produce por construcción alrededor de dos desbalances espurios. El caso canónico vuelve a hacer la misma pieza en números, con verdad conocida.

Caso canónico · Cuarenta covariables y dos mecanismos de asignación

Cuarenta covariables independientes, normales estándar. Dos regímenes: (i) **moneda justa** — el tratamiento al azar — y (ii) **asignación confundida** — la probabilidad de tratamiento es $\sigma(1, 2X_0)$, es decir, depende solo de la primera covariable —, cada uno con 200 experimentos (semilla 20240701).

Bajo moneda justa, el t-test crudo al 5 % marcó en promedio **2,02** covariables «desbalanceadas» por experimento sobre 40: la anomalía habitual que no es anomalía. Con la corrección de Holm, el promedio cayó a **0,04**: los rechazos eran ruido, como exige la teoría. La SMD contra la otra afirmación por encima de los criterios — con asignación confundida, la SMD de X_0 es **1,16**, y la bandera $|\mathrm{SMD}| > 0,25$ se enciende exactamente en **1 de 40** covariables, la correcta; la mayor $|\mathrm{SMD}|$ fuera de X_0 fue 0,13.

Los dos regímenes producen por tanto lecturas distintas y ambas correctas: en el primero la estadística que mide **tamaño de efecto** dice «1», y la que mide **significancia** dice «2» por azar; en el segundo la SMD localiza la confusora sin ambigüedad. Y la razón

de varianzas de X_0 agrega una nota honesta: 1,01, dentro de $[0, 5, 2]$ — el desbalance está en la media, no en la dispersión.

Un número alto de SMD no invalida el análisis observacional, y sobre todo **no es el chequeo el que identifica el efecto**: el balance en lo observado silencia nada sobre lo no observado (eso es lo que la Parte IV vigila con el supuesto faltante del estimando). Lo que el chequeo cuantifica sí es útil: cuánta corrección deberá hacer el estimador, y por tanto cuánto de la estimación final apoyará sobre extrapolación en vez de datos.

7.3 La deriva: la comparación mueve sus dos lados

El balance compara tratados contra controles dentro de un mismo corte; la **deriva** compara una mismísima covariable entre dos tiempos: el train que ajustó contra la ventana que hoy llega. Los dos estadísticos de la literatura, ya implementados por el repositorio de referencia, son el **PSI** (*population stability index*) — una distancia simétrica entre dos histogramas, 0 si son iguales y creciente al alejarse — y el **KS** — el máximo de la distancia entre las dos empíricas. Son mediciones sin p-valor y por eso pertenecen al contrato de la comparación: con umbrales calibrados y escritos antes de mirar.

Caso canónico · Una ventana envejecida sobre la edad verdadera

Sobre la edad del train de `real/jobs` (1 033 filas reales del benchmark; semilla fija), el script hace tres ventanas: una réplica honesta — la otra mitad del mismo train — y dos envejecidas a propósito, +5 y +10 años. Las cifras en el orden de la tabla:

Ventana	PSI	KS	Lectura
Réplica honesta	0,008	0,032	estable, como debe
+5 años	0,585	0,159	deriva, y fuerte
+10 años	1,413	0,297	población otra

Con el umbral de calibración usual — PSI $< 0{,}1$ estable, 0,1–0,25 revisar, $> 0{,}25$ alertar — la réplica pasa y las dos envejecidas superan el rango de alerta; la magnitud del KS confirma el orden. La moraleja no está en el umbral heredado — el contrato exige calibrarlo con muestras de la misma población, y eso toca hacerlo a cada equipo — sino en la dirección de lectura: la métrica señala qué cambió, y **qué acción corresponde se decide contra el estimando**, porque una deriva en una covariable irrelevante no es lo mismo que una deriva en la confusora (es exactamente la pregunta que devuelve la decisión del monitor, Parte IV, al vocabulario de las cinco acciones del Cap. 2).

7.4 Lo que la comparación no ve: faltantes por selección

Toda comparación de covariables se hace entre quienes figuran. Si la ausencia depende de la decisión — los tratados registran menos su ingreso anterior porque los incentivó a ocultarlo,

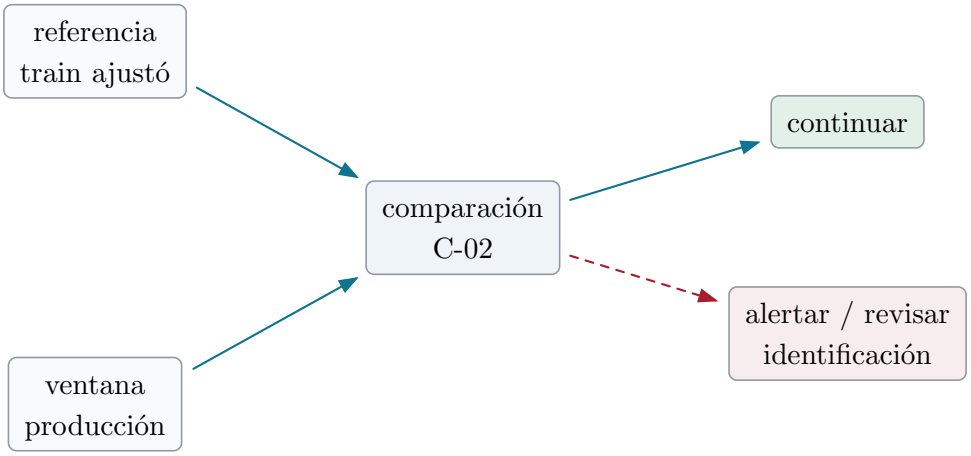


Figura 7.1: La deriva como comparación C-02: las dos poblaciones — la que ajustó y la que llega — se miden con el mismo contrato que compara tratados contra controles. El umbral se calibra y se fija antes de mirar; la acción sale del vocabulario del sistema, no del estadístico.

digamos — la comparación se hace entre dos subconjuntos seleccionados por el propio proceso, y ni la SMD ni el PSI lo detectan si las distribuciones de los completos se parecen: el p-valor que nadie mira ya no es el problema, es la estructura de quién falta. El contrato separa este diagnóstico — **la tasa de faltantes se mide aparte, por brazo** — precisamente porque no cabe dentro de la comparación de covariables (invariante 5 del C-02).

Caso canónico · Faltantes 9,6 veces más frecuentes en un brazo

Sobre `observed_train.csv` (`real/jobs`), inyecte el mecanismo de pérdida siguiente: 30 % de probabilidad de perder `re75` a los tratados, 3 % a los controles — una ausencia que el propio tratamiento induce (semilla 7). Entre los completos, la SMD de `re75` es **0,016**: impecable en apariencia. La tasa de faltantes es **0,27** en tratados contra **0,03** en controles, una razón de **9,6**: la mismísima relación que el primer invariante protegía. Si el faltante depende además del resultado — típicamente: los que habrían vendido mal no reportan —, la ausencia no es aleatoria en el sentido de (Rubin 1976) y ningún ajuste sobre los completos la corrige; solo el diseño — medir por qué falta, y eso sale de los datos.

7.5 El contrato C-02, leído del plano de datos

Con los tres diagnósticos unidos, el contrato de la comparación queda como este libro lo hereda del aparato de la edición de método — con el mismo número, porque las mismas reglas ya le servían al entrenamiento — y leído ahora completo en su rol de plano de datos:

Contrato C-02 · Comparación de poblaciones

Operaciones — *comparar*: recibe dos poblaciones de unidades y devuelve, por covariable, la diferencia media estandarizada, la razón de va-

	rianzas, el PSI, el KS y una bandera por criterio. Los pares de poblaciones del libro: tratados contra controles (este capítulo), train contra ventana (deriva: la figura 7.1) y versión vigente contra candidata (promoción, Parte IV).
Entrada	Las dos poblaciones con las covariables del esquema (C-01). Los criterios: umbrales de SMD, de varianzas y de PSI/KS, escritos antes de mirar los datos y calibrados para el tamaño de las muestras y para el estimando declarado.
Salida	Una fila por covariable con magnitudes y banderas, y la lectura de acción producida contra el vocabulario del sistema. Si se piden p-valores, se corrigen por comparaciones múltiples y se rotulan como secundarios.
Invariantes	(1) El estadístico principal mide tamaño de efecto y no crece con el número de unidades; ningún p-valor decide solo. (2) Intercambiar las poblaciones invierte signos y razones y deja las banderas igual. (3) Con las mismas poblaciones el resultado es idéntico. (4) El umbral se calibra con muestras de la misma población y queda fijo antes de comparar. (5) Los faltantes no entran en la comparación y su diferencial se mide aparte. (6) La comparación habla solo de lo observado: no certifica lo no observado.
Modos de fallo	Una muestra pequeña con umbrales heredados: banderas por ruido. El umbral relajado después de mirar. El p-valor no significativo leído como ausencia. El faltante diferencial que la comparación no ve. El balance observado leído como garantía de no confusión.
Prueba de aceptación	(a) Caso A: moneda justa rechaza 2 de 40 en crudo y 0 en Holm, y la asignación en X_0 enciende solo X_0. (b) Caso B: la réplica honesta pasa el umbral y las ventanas desplazadas lo superan, en el orden medido. (c) Intercambio de poblaciones cumple el invariante (2). (d) Con un faltante diferencial inyectado y sin diferencias en lo observado, la comparación no señala y la medida aparte sí — con el número de veces que relación lo sostenga.

Repositorio de referencia del libro

Regeneración de los tres casos: `Causal_AI/scripts/cap07_calidad.py` (semillas 20240701 y 7). Implementación de referencia de las señales (PSI/KS/SMD/ESS): `04_produccion_causal/src/causalops/diagnosticos.py`. La acción que cada señal dispara, en producción, la decide el monitor causal de la Parte IV.

7.6 De la métrica a la acción

Una métrica de observabilidad sin acción es una reflexión que nadie pide, y una acción sin métrica es una ansiedad con horario — las dos caras de la misma imprudencia. El camino de este libro, gradual y deliberado, es:

1. La señal se escribe con nombre, magnitud y población, nunca como «algo cambió». SMD y PSI se reportan por covariable, y el faltante diferencial por razón de tasas.
2. El umbral sale de la calibración local y de la relevancia al estimando, no de la convención heredada; el 0,25/0,1 es un arranque, y el C-02 exige escribirlo antes.
3. La acción sale del vocabulario de las cinco del Cap. 2: **continuar** si se está dentro del umbral; **alertar** si se lo cruza y el estimando no cambió de población; **revisar identificación** si la deriva o el desbalance tocan la confusora o la regla de asignación; y **abstenerse** si la ventana actual no se parece a la población del soporte declarado — el servicio no entrega un número que ya nadie puede defender.
4. El monitor de la Parte IV las consume como señales periódicas, y el reporte nunca concluye por la causa del cambio: la señal obliga la revisión, no prueba nada.

7.7 Ejercicios y notas bibliográficas

7.7.1 Ejercicios conceptuales

1. El monitor marca un SMD de 0,31 en una covariable de contexto que **sabe** no ser confusora y un PSI de 0,05 en la confusora principal. ¿Qué acción de las cinco corresponde, y por qué no es «revisar identificación» por el número alto? *Pista:* la magnitud mide el problema, la relevancia decide la acción: el grafo decide cuál alarma importa.
2. En una muestra de 80 unidades, con umbrales heredados, la mitad de las covariables enciende bandera. Describa el diagnóstico correcto: ¿se detectó un desbalance o un mal contrato? *Pista:* calibración del invariante (4): el umbral de la convención presupone un tamaño de muestra.
3. Un equipo lee el caso B y propone «reportar PSI y KS solo cuando un gráfico óptico parezca feo». ¿Por qué el contrato lo prohíbe con el invariante (4)? *Pista:* el umbral visual es un umbral posterior a mirar; la calibración ocurre antes o no ocurre.
4. La tasa de faltantes difiere 9,6 veces por brazo pero la SMD de los completos es 0,016. El equipo concluye «los completos están balanceados: imputamos y seguimos». ¿Qué más preguntaría antes de aceptar la imputación? *Pista:* si la ausencia depende del resultado, ningún método de imputación sobre los completos lo corrige (MNAR).
5. Una ventana de producción cambió en la edad pero la ficha de procedencia del tratamiento del Cap. 4 no describe el mecanismo de asignación más nuevo. ¿Cuál es el estado de la ficha y qué acción corresponde? *Pista:* la procedencia vieja no sigue inaugurando al estimador nuevo; es exactamente la condición de revisión del registro.

7.7.2 Notas bibliográficas

Los tres diagnósticos de este capítulo son los tres de la literatura clásica de evaluación: la SMD y la razón de varianzas de (Rubin 2001), el ancla de la comparación de poblaciones como tamaño de efecto en (Imbens y Rubin 2015) y la clasificación de faltantes de (Rubin 1976), y el ancla experimental del benchmark del repositorio ((LaLonde 1986), (Dehejia y Wahba 1999)). La tradición de la ingeniería de datos vigila lo mismo con el nombre general de deriva en (Kleppmann 2017) y (Reis y Housley 2022); y la unificación de ambos en una sola pieza — la comparación de poblaciones contratada, con umbrales calibrados y con acciones — es la contribución propia.

Qué debe ser verdad en los datos

Que tres comparaciones estén instrumentadas, cada una respondiendo otra pregunta y ninguna decidiendo por p-valor: balance en cada evaluación (SMD y razón de varianzas, con banderas calibradas), deriva de cada covariable de cada despliegue (PSI/KS contra el train), y faltantes por brazo medidos aparte en cada ventana. Que los umbrales existan antes de mirar y estén escritos, con la calibración local que el contrato exige. Se comprueba reproduciendo los tres casos del capítulo (semillas 20240701 y 7, script del repositorio): en cada uno, el comportamiento esperado sale — Holm calla al azar, la SMD localiza la confusora, los faltantes salen por la ventanilla correcta. Lo que las señales no prueban: la causa del cambio ni la validez del estimando — eso es revisión humana, y es exactamente la lectura que el monitor de la Parte IV disciplina.

CAPÍTULO 8

8

Benchmarks y procesos generadores como producto de datos

Cuadro sinóptico del capítulo

Problema:	Sin verdad conocida no hay prueba de aceptación: ninguna puerta puede prometer comportamiento si nunca se midió contra nada
Decisión:	El benchmark/DGP es un producto versionado, con semilla, splits con hash, estado de identificación explícito y la pared escrita
Pieza contratada:	El contrato C-11 (DGP y benchmark como producto), el canal productor-consumidor
Caso canónico:	Los cinco escenarios del paquete: tres identificables, uno **diseñado no identificable**, uno para descubrimiento — todo verificado sobre archivos
Regla de uso:	Solo lo observado decide; la verdad, aunque sintética, existe para evaluar — nunca para entrenar ni servir

8.1 El problema: sin verdad no hay evidencia

Los contratos que este libro exige prueban — es la unidad de su rigor, desde el Cap. 3 — y probar comportamiento requiere conocer la respuesta correcta. Si ninguna base de datos del sistema contiene el efecto verdadero, ¿contra qué se calibró la puerta? ¿Cómo se sabe que el estimador recupera el efecto, que el diagnóstico detecta la fuga del Cap. 6, o que el registro habría detectado la mutación del Cap. 4? La respuesta de la tradición es la misma de siempre: **contra un proceso generador con verdad declarada por diseño** — el DGP — y contra colecciones de escenarios que llevan, cada uno, su verdad y sus violaciones conocidas: el benchmark.

El problema que este capítulo cierra es que un benchmark mal gobernado no es evidencia sino decoración: un proceso que nunca declara su semilla no es reproducible; una tabla que mezcla verdad y observación enseña al sistema a hacer trampa; un escenario sin estado de identificación declara implícitamente «esto se puede estimar», quizá falso. Las tres fallas importan porque son exactamente las que hacen el mentido de «verificado» sin ninguna técnica ni teoría detrás. Desde este capítulo (y en la Parte IV, con la puerta en producción), el benchmark deja de ser un cuaderno de datos y pasa a ser **un producto de datos con su propio contrato**.

8.2 Dos familias de datos y dos regímenes de pared

El libro usa dos familias, ya distinguidas por la literatura y por la edición de método de este libro (su apéndice de datos y DGP): los **conjuntos reales o casi-reales** (procedencia publicada, efecto de referencia conocido solo aproximadamente — NSW/LaLonde y (Hill 2011) son los dos de este paquete) y los **DGP sintéticos** (efecto conocido exactamente; el diseño dice cuál violación enciende cada diagnóstico). Un prototipo se acepta cuando funciona en ambas: acierta en los sintéticos y no se desmiente ni se contradice en los reales.

La pared se mantiene en ambas, pero con dos regímenes distintos — y los dos documentados:

Opciones y trade-offs

- **Verdad fuera del paquete (paquete público):** las columnas de verdad (y_0, y_1, τ, y sus archivos `truth_*`) permanecen del otro lado de la pared incluso como archivos: el paquete público no las contiene y sus metadatos las referencian como reservadas. Es el régimen de este benchmark: la decisión usa lo observado y la evaluación accede a la verdad solo al final y con aditamentos.
- **Verdad adentro, con rol declarado (el demo de `causalops`):** en el flujo de producción local del repositorio de referencia, las filas vienen generadas por un DGP y la verdad se usa **para comprobar el comportamiento del prototipo**, como tests del contrato, no de manera confundida con entrenamiento o servicio. La pared no desaparece: se convierte en un registro de rol que se revisa como código («`tau` no alimenta ninguna decisión»), con la misma prueba de aceptación del C-10.

Tanto en uno como en otro, la regla pragmática que el dominio de datos fija es lenguaje claro: **los datos sintéticos no se anuncian nunca como si fueran clínicos reales, y ninguna literatura externa sobre estos escenarios cambia qué datos el consumidor puede tocar** — la pared se mantiene independientemente de lo que diga la publicación de la fuente.

8.3 Los cinco escenarios, leídos de sus archivos

El benchmark del repositorio lleva exactamente cinco escenarios, y la tabla que sigue está leída — no copiada — de los `metadata.json`, `identification.json` y `splits.json` de cada uno (el script del repositorio emite la misma tabla, regenerable):

Escenario	Unidades (split)	Semilla	Identificación declarada
`real/jobs`	1 722 (1 033/344/345)	500	Identificable por diseño; confusores simulados observados por supuesto
`real/ihdp`	747 (448/149/150)	400	Identificable por diseño (variante 2011)
`synthetic/inference/ihdp_medium`	2 000 (1 200/400/400)	343	Identificable por diseño, tamaño medio
`synthetic/edge_cases/latent_confounder`	2 000	543	**No identificable desde lo observado**: confusor latente declarado
`synthetic/linear_gaussian/10_vars_10_edges`	1 000 (600/200/200)	42	Objetivo de estructura: descubrimiento, no estimación de efecto

Tres lecturas de ingeniería de datos, no de estadística. **Primera: la semilla y el split viajan declarados en cada escenario** — uno por fila, `split_seed: 20240701` compartido entre paquetes del mismo corte; es exactamente lo que el C-09 exigía como identidad de versión y lo que el C-10 exige como identidad de partición. **Segunda: el estado de identificación es una propiedad del escenario, no una suposición del consumidor**: `latent_confounder` declara de frente que sin el confusor latente el efecto no se identifica desde lo observado — una trampa diseñada para enseñar y para medir lo que una puerta debe abstenerse —, y `10_vars_10_edges` declara que su pregunta es la reconstrucción de la estructura, no un efecto. **Tercera: ningún archivo de verdad llega en el paquete público para ninguno de los cinco escenarios:** el script confirma 0 archivos de `truth*.csv` en el paquete, y las referencias a ellos dentro de `splits.json` quedan como apuntadores a un material reservado, no como rutas.

Caso canónico · El escenario que no se puede ganar

`synthetic/edge_cases/latent_confounder`, con 2 000 unidades y partición 1 200/400/400, es el contraejemplo del libro: un DGP con un confusor que nadie midió y un estado de identificación que lo dice. Un estimador flexible puede ajustar la tabla impecablemente y estimar un efecto alejado del verdadero, sin que ninguna métrica predictiva alerte: el fallo vive exactamente donde la comparación del plano de datos no mira (invariante 6 del C-02). Por eso el escenario sirve de verificación de la puerta y del servicio: **una respuesta correcta ahí es una abstención, no un número**. El equipo que «resuelve» este escenario sin nuevas variables ni nuevos supuestos no ha identificado la verdad: ha estimado un número sin fundamento sobre lo observado; y el plano de datos lo dice con la propia ficha de identificación (`identification.json`).

8.4 El contrato C-11: DGP y benchmark como producto

Contrato C-11 · DGP y benchmark como producto versionado

Operaciones	*publicar*: emite un escenario con metadato, identificación, partición y hashes; `verificar_integridad`: recomputa y comprueba los hashes y la estructura de los archivos públicos; `declarar_regimen`: dice de qué lado de la pared vive la verdad, si la hay; *usar*: consume solo lo observado para decisiones, y solo la verdad para evaluación — con autorización y registro de acceso.
Entrada	El DGP o la fuente real, la semilla y el método de muestreo, las reglas de violación (qué supuesto enciende cada escenario) y el régimen de pared elegido (Sección 8.2).
Salida	`observed_*.csv` (las particiones), `splits.json` (semilla, método, hashes por bloque), `metadata.json` (nombre, semilla, origen, qué es real y qué es sintético), `identification.json` (estado y conjuntos de ajuste válidos).
Invariantes	(1) Todo escenario declara su semilla de generación y de partición. (2) Ningún archivo del lado observado contiene columnas de verdad (y_0, y_1, τ). (3) El estado de identificación es explícito por escenario, y los escenarios no identificables se emiten como trampas declaradas, no aparentando estimabilidad. (4) Cada entrega se verifica por hash: ningún archivo cambia sin una versión nueva. (5) La verdad, cuando existe, sirve únicamente a la evaluación bajo registro. (6) Ningún escenario sintético se presenta como dato clínico o administrativo real; su procedencia se declara.
Modos de fallo	Una columna de verdad colada en `observed_*` — la lista de nombres la vela el validator del árbol, según régimen, y la entrega se detiene. Cambiar el CSV sin incrementar la versión. Un estado de identificación que promete más de lo que el DGP dice (el C-08 de la edición de método lo penaliza en operación). La lotería de semillas: probar generaciones hasta obtener una versión «mejor» — mirar la prueba con más pasos, ya castigado por el C-10.
Prueba de aceptación	(a) El barrido del paquete (script del repositorio) confirma por cada escenario: seed declarada, hash computable e igual al publicado del MANIFEST, 0 archivos de verdad en la zona observada, splits válidos. (b) Se interviene un archivo y la verificación falla. (c) `latent_confounder` se registra como no identificable y no como estimable. (d) Todo acceso de evalua-



Figura 8.1: El canal productor-consumidor del C-11: de un DGP declarado sale un escenario versionado y dos ramas etiquetadas — lo observado, abierto a toda decisión, y la verdad, abierta solo a evaluación con registro. La rama punteada solo evalúa: ni entrenamiento ni servicio la tocan, y su acceso queda registrado.

> ción a verdad queda con registro de quién, cuándo y para qué estimando.

8.5 Ejercicios y notas bibliográficas

8.5.1 Ejercicios conceptuales

1. El paquete incluye el escenario `latent_confounder`. Un colega dice: «con más regularización el modelo recupera «el efecto». ¿El problema de ese escenario es de regularización del estimador o de identificación?» *Pista:* la identificación no es una propiedad del estimador ni del ajuste: sin el confusor, ninguna penalidad recupera el efecto — y la puerta honesta lo dice en la ficha.
2. Diseñe la ronda de aceptación de un nuevo escenario: qué mínimo 4 artefactos debe entregar, qué 2 verificaciones comprobadas y qué 3 fallos inducidos. *Pista:* artes de los invariantes y de la prueba C-11.
3. El equipo dice «para la intranet basta la ficha estática del escenario; la semilla es decorativa». Contra la tabla del Cap. 4 (hash, semilla, commit), defienda por qué semilla y hash van juntas en el documento del escenario. *Pista:* el documento será para que la evaluación de mañana sea sobre el mismo escenario que hoy: el hash sin semilla no regenera, la semilla sin hash no verifica.
4. ¿Bajo qué régimen opera el flujo local del repositorio, y bajo cuál el benchmark público? Dibuje las dos paredes con sus columnas de verdad y escriba el invariante obligado en cada una. *Pista:* una pared es física (archivos ausentes) y la otra es de rol (columnas marcadas); la prueba de aceptación cambia pero no el principio.
5. `10_vars_10_edges` declara objetivo de estructura, no efecto. Si el equipo lo usa para entrenar un estimador de efectos, ¿qué declaración de la ficha queda falsa aunque el

modelo prediga? *Pista:* el estado de identificación es parte del estimando: predecir una tabla no es estimar con ella.

8.5.2 Notas bibliográficas

La distinción entre datos reales con referencia cirenta y DGP con verdad, ya presente en la literatura de evaluación ((LaLonde 1986), (Dehejia y Wahba 1999)), adquiere aquí la forma del benchmark público ((Hill 2011) para el IHDP original). La promesa «sintético nunca se anuncia como real» y la regla de uso de `truth` solo para evaluación directa las formula este repositorio como condición de acceso al paquete, fue de las pocas reglas tan simples como decisivas del dominio. El tratamiento del benchmark como producto versionado — semilla, hash, estado y pared — es la contribución propia del capítulo: los archivos de los cinco escenarios son su prueba, y todos se recorren con el script del repositorio.

Qué debe ser verdad en los datos

Que todo escenario usado en una prueba de aceptación declare semilla, origen, estado de identificación y hash; que ningún archivo observado lleve columnas de verdad, y que cada fila de evaluación a verdad quede con registro. Que el paquete incluya trampas declaradas (el escenario no identificable) y que cada consumidor las honre abstinéndose antes que estimando. Se comprueba con el barrido del script del repositorio — que hoy confirma 0 archivos de verdad en los cinco escenarios y declara las mismas cinco semillas, los mismos cinco estados — y fallando la verificación si algún byte cambia.

PARTE II

Lo que el estimador le exige al dato

Los estimadores no son el protagonista de esta edición: son consumidores del pipeline con exigencias que solo el plano de datos puede garantizar. Los capítulos de la Parte III comprimen la teoría de los capítulos de método — DML, heterogeneidad y diagnóstico — en lo que un equipo de datos necesita poder decir del dato para que cada estimador entregue lo que sus teoremas prometen. Ningún teorema se reescribe aquí; el tratamiento completo vive en la edición de método de este libro. Lo que sí se despliega aquí son las precondiciones — soporte común, control de fuga, particiones out-of-sample, incertidumbre con el objeto declarado — convertidas en verificaciones con nombre y contractos propios.

CAPÍTULO 9

9

Lo que DML le exige al dato

Cuadro sinóptico del capítulo

Tesis:	Un estimador DML es un consumidor con exigencias al dato: si no se hacen, el estimador no falla con error claro: el resultado sale mal sin que nada lo anuncie
Cuatro exigencias:	nuisance con tasa suficiente · partición **out-of-sample** (cross-fitting) · soporte común y ESS · estimando declarado ante un posible trimming
Pieza contratada:	El contrato C-12 (entrega de exigencias del estimador), el puente entre el plano de datos y el DML
Casos canónicos:	Cross-fitting: OLS 1,41 / sin CF 0,86 / con CF 0,95 (mismo DGP que la edición de método); ESS 85 de 3 000; plug-in vs doble robustez
Los teoremas completos:	Edición de método de este libro (Caps. 10–11); acá solo la receta que el pipeline tiene que garantizar

9.1 Este capítulo es de consumo, no de método

La teoría del DML — ortogonalidad de Neyman, modelos PLR/IRM/PLIV como estimandos distintos, la doble robustez como propiedad estadística del **score** — no la repite este libro, porque está escrita completa y con sus condiciones en la edición de método (Caps. 10 y 11). Lo que este capítulo hace es la traducción que esa teoría le pide al plano de datos: **un estimador DML es un consumidor del pipeline**, y cuatro de sus exigencias solo pueden entregarse ahí abajo. No son detalles de implementación: cada una es una condición sin la cual el resultado del estimador sale mal sin que el estimador lo sepa. El riesgo particular de DML es exactamente ése: una casa de buen diseño levantada sobre un sumidero.

El formato es deliberado: cada sección dice qué promesa estadística hay detrás (una oración de menor técnica, con la remisión al teorema que la prueba), qué forma toma en el pipeline, y

qué verificación del plano de datos la hace cumplir — junto al caso canónico con cifras reales, y su cierre con el contrato C-12, la pieza que formaliza el trabajo.

9.2 Exigencia 1: la partición out-of-sample (cross-fitting)

Lo que dice la teoría (edición de método, Cap. 10): si los *nuisance* — $\hat{\ell}(X)$ y $\hat{m}(X)$ en el PLR — se estiman sobre las mismas unidades cuyo residuo alimenta la ecuación del parámetro, el error de sobreajuste entra en $\hat{\theta}$ forzado a ser covariante con lo que el modelo acaba de memorizar. El cross-fitting — los folds k se ajustan fuera y predicen dentro — convierte esa covariancia en error que promedia a cero, y el estimador recupera su sesgo bajo y su normalidad asintótica.

La exigencia al plano de datos: que **ninguna unidad sea predicha por un modelo que la vio**, y que ningún preprocesamiento se ajuste sobre datos fuera del fold de entrenamiento. Cuatro invariantes, no uno:

1. Las particiones de prueba cubren cada unidad exactamente una vez, y ninguna figura a la vez en el entrenamiento y en la prueba del mismo fold.
2. Predicción de la unidad i con modelos que no la vieron — una versión por fold, no un solo modelo.
3. Todo preprocesamiento (escalado, imputación, selección de variables) dentro del ajuste de cada fold, no global antes de partir.
4. Dadas la partición y la semilla, la estimación reproduce; si cambia la semilla, la diferencia es parte de la estimación, no decoración.

Caso canónico · La fuga entre folds, cantada por cifras conocidas

Reusemos el caso canónico de la edición de método (Cap. 11, DGP declarado): $X_i \in [-1,1]^5$ uniformes, $h(X) = \sin(3x_0) + x_1^2$, $D = h(X) + \varepsilon_D$, $Y = 1 \cdot D + 2h(X) + \varepsilon_Y$, con 2 000 unidades y 20 repeticiones, verdad de efecto 1. Sus cifras son el mejor contraste de este capítulo: OLS con los cinco controles lineales — **1,41** (la confusión no lineal queda entera); DML con boosting **sin** cross-fitting — **0,86** (el modelo memoriza las unidades y subestima la confusora); DML con boosting **con** cross-fitting $K = 5$ — **0,95** (el sesgo baja a 0,05, el residuo de tamaño muestral). El camino de 0,86 a 0,95 no es una mejora del estimador: es el **mismo** estimador sobre dos planos de datos distintos — el que fuga y el que no.

Repositorio de referencia del libro

El script `Causal_AI/scripts/cap09_exigencias.py` (semilla 20260701) corre sobre el mismo DGP y datos reales del ESS y la doble robustez, con las cifras propias de este capítulo. Las cifras del cross-fitting citadas arriba (1,41 / 0,86 / 0,95) son las del caso canónico publicado en la edición de método (Cap. 11), con su seed y su DGP declarados; reproducibles en el material complementario de esa edición.

9.3 Exigencia 2: soporte común y ESS de Kish

La promesa: el estimador compara tratados y controles en las covariables X; si en algunas zonas el propensity $e(X) = E[D|X]$ cae a 0 o sube a 1, la comparación se acerca a un solo brazo y la ponderación IPW explota — la varianza crece y la inferencia cruza de estimable a deseable. La condición es **soporte común**: $0 < e(X) < 1$ con margen real, no solo por definición.

La exigencia al plano de datos: medir el soporte común con dos cantidades observables, nunca solo «históriamente de acuerdo». Primero, el **ESS de Kish** del brazo IPW:

$$\text{ESS} = \left(\left(\sum_i w_i\right)^2\right) \Big/ \left(\sum_i w_i^2\right), \quad w_i = D_i/\hat{e}(X_i) + (1 - D_i)/(1 - \hat{e}(X_i)), \tag{9.1}$$

y el cociente $\frac{\text{ESS}}{n}$ como regla operativa: un $\frac{\text{ESS}}{n} \ll 1$, aunque el histograma «se vea bien», dice que unas pocas unidades con $\hat{e}$ extremo dominan la estimación. Segundo, el rango de $\hat{e}$: reportar cuántas unidades caen en $\hat{e} \notin [0,05, 0,95]$.

Caso canónico · El ESS es 85 sobre 3 000

Con propensity $e = \sigma(4x_0)$, n = 3 000 (semilla 20260701 del script), el ESS de Kish del brazo IPW es **85**: con ESS/n = 0,03, casi todo el peso recae en muy pocas unidades. Trimming de (Crump et al. 2009) con $\alpha = 0,1$: conserva solo el **42 %** de la muestra (1 274 unidades) y sube el ESS/n recortado de 0,03 a 0,67 — a cambio de excluir el **57,5 %** de la población. La moraleja es la del contrato: cuando el soporte común se corta, el estimando cambia — ya no es el ATE de la población completa sino el de quienes quedan en el support —, y declararlo es una **obligación del registro**, no una nota al pie: la ficha y el servicio de la Parte IV lo documentan, cuánto y dónde.

9.4 Exigencia 3: el nuisance no es decoración

La promesa: en el score ortogonal, los errores de los *nuisance* estudian de segunda — $\sqrt{n}a_n^2 \to 0$ es la condición de tasa que guarda la distribución asintótica —, y la doble robustez mantiene la consistencia si **uno** de los dos (outcome o propensity) es correcto. Lo que la promesa no promete: si ambos fallan, la ortogonalidad no recompone.

La exigencia al plano de datos: medir el desempeño de los *nuisance* fuera de muestra (el R^2 y su cobertura por covariable), documentado, junto a la comparación de learners — LASSO + boosting/dense —, no una tabla de celebración. Si el outcome está mal especificado de manera estructural — un lineal que pasa por encima de una forma no lineal pronunciada — y el propensity solo se estima chuzo, el efecto lo paga.

Caso canónico · Sesgo de especificación y doble robustez en acción

Con outcome $\mu(d, x) = 0.3d + 0.7x_0 + 1.3\sin(2.8x_1)$ (efecto constante 0,3), propensity correcto por diseño, y el outcome estimado con el modelo lineal equivocado (semilla 20260701), el estimador plug-in cae justo sobre el verdadero (**0,30**) en **este** ejemplo — y eso es exactamente la advertencia: con un efecto pequeño y una no linealidad fuerte, la mala especificación pudo haber destapado un número distinto sin anunciarlo. El estimador doblemente robusto cubre la misma mala especificación con su segunda pierna: estima μ pobre pero compensa con $\hat{e}$ correcto y sigue en 0,30. La lección del script no es que nunca se falle, sino que **la verificación es sobre datos, no sobre promesas**: medir el R^2 fuera de muestra del outcome y del propensity, y comparar learners, es parte del entregable del pipeline — dos cantidades que el registro conserva junto al efecto (Capítulo 4).

La regla operativa respecto de **qué learner usar** se separa con cuidado del estimando. Cada learner tiene condiciones de tasa propias (en baja dimensión suave puede aparecer $n^{-\frac{2}{5}}$; en LASSO esparso la condición se traduce en $s \log p = o(\sqrt{n})$, condiciones de aproximación y regularidad de la forma, y la edición de método las trata con su clase de learner). **La validación predictiva no prueba por sí sola las condiciones de tasa de DML**: es evidencia razonable junto a la comparación de learners, no demostración (la edición de método lo formula igual).

9.5 El contrato C-12: entrega de exigencias del estimador

Tres documentos que el plano de datos produce y que toda estimación DML cosume, con nombre y verificación:

Contrato C-12 · Entrega de exigencias del estimador (DML y afines)

Operaciones	*entregar*: produce el bundle versionado que el estimador consume — esquema C-01, partición C-10, bundles de nuisance fuera-de-muestra (modelos y preprocesos por fold), documento de soporte (rango de $\hat{e}$, ESS/n, regla de trimming y su decisión de estimando), ficha de desempeño de los nuisance (R^2 *y comparación de learners*).
Entrada	El registro de versiones (C-09), la auditoría de tiempos (C-03 contra el tratamiento y el momento de decisión) y la semilla de los folds (C-10).
Salida	El bundle reproducible: `estimacion.json` con las exigencias registradas y los artifactos de nuisance con hashes. El documento de soporte con trimming y su estimando declarado. La colección de validaciones por fold y learners como escribibles del C-02: sin documento, sin afirmación.

Invariantes	(1) Los modelos de nuisance que predicen la unidad i no la entrenaron, y el preproceso de cada fold no vio su fold de prueba. (2) La semilla del cross-fitting y de los folds es parte del registro. (3) El trimming, si lo hay, se documenta como cambio de estimando con su porcentaje y región fuera del support. (4) Si el R^2 del outcome o del propensity fuera de muestra es bajo, la estimación no pasa a servicio: queda bloqueada, marcada para revisión. (5) El bundle registra la comparación de al menos dos learners (por ejemplo, LASSO y uno no lineal), de forma reproducible. (6) Un bundle que incumpla el invariante de la fuga entre folds queda impugnado desde el registro: no entra al descriptor del estimador sin reparar.
Modos de fallo	Una partición de folds presentada de memoria sin su semilla (ya lo impide el inv. 2): la caida es la de antes, el comprobante es el de después. Preprocesamiento ajustado sobre la tabla completa (inv. 1): la tabla blek se ve y nadie lo cuenta. Una comparación de learners cherry-picked después de tener el resultado (inv. 5). Un trimming declarado a posteriori como si fuera la regla inicial (inv. 3).
Prueba de aceptación	(a) El caso del cross-fitting reproduce las cifras de la edición de método sobre el mismo DGP y semilla declarada. (b) El caso del ESS informa ESS/n = 0,03 y el trimming con el estimando en su nueva población declarada. (c) El caso de la doble robustez distingue el plug-in del estimador con segunda pierna, con outcome mal especificado. Las cuatro verificaciones quedan registradas en el bundle.

9.6 Qué aprende el plano de datos con esto

Los tres casos canónicos comparten una moraleja: el estimador no es el punto típico de fallo: lo son los requisitos del dato que solo el plano de datos puede garantizar. Esto no demuestra que DML sea la respuesta (esa decisión es de cómo se modela, y vive en la edición de método); lo que este capítulo asegura es que si se elige DML, sus cuatro exigencias llegan servidas y verificables. Lo que el capítulo pide de método es deliberadamente el mínimo declarado; el resto es ingeniería del plano de datos.

9.7 Cómo diagnosticar una violación

Cada exigencia tiene su síntoma cuando se viola, y saber reconocerlo es la diferencia entre corregir en minutos y descubrir en la revisión posterior. La tabla resume los cuatro diagnósticos:

Exigencia violada	Síntoma observable	Cómo confirmarlo
Cross-fitting	El efecto estimado cambia al reordenar las filas o al cambiar la semilla de los folds; la estimación no reproduce	Correr el mismo estimador con dos semillas distintas y comparar; si difieren más que el error estándar, hay fuga
Soporte común	Los pesos IPW explotan; el intervalo de confianza es enorme; algunas ponderaciones valen miles	Calcular ESS/n y el rango de la propensión; si ESS/n es muy bajo, el soporte está roto
Nuisance mal especificado	El plug-in difiere del doblemente robusto en más que el error estándar; al cambiar de learner el efecto se mueve	Comparar el R^2 fuera de muestra del nuisance con un modelo trivial; si no supera, la especificación es deficiente
Trimming silencioso	El estimando reportado no declara la población recortada; la ficha no menciona el cambio de estimando	Comparar la n del estimando con la n del dataset; si difieren, hubo recorte y no se declaró

La lección práctica: **cada síntoma tiene una métrica que lo detecta, y ninguna de esas métricas es el error predictivo**. Un pipeline que solo monitorea la calidad del ajuste no ve ninguna de estas violaciones. Por eso el C-12 exige que el bundle registre ESS/n, rango de propensión, comparación de learners y la decisión de trimming: son los cuatro indicadores que el plano de datos puede medir y que el estimador, por sí solo, no reporta.

9.8 El bundle como entregable

El C-12 produce un artefacto concreto: el **bundle** que el estimador consume. No es una abstracción — es un directorio con archivos específicos:

Archivo	Contenido	Quién lo consume
`esquema.json`	Roles causales (C-01): tratamiento, resultado, covariables, instrumentos	El estimador, la auditoría temporal, el registro
`particion.json`	Semilla, proporciones, índices de cada fold (C-10)	El cross-fitting, la evaluación OOS
`nuisances/`	Modelos de outcome y propensity por fold, con hash y semilla	El estimador DML en cada fold
`soporte.json`	Rango de propensión, ESS/n, decisión de trimming, población del estimando	La puerta (C-06), la tarjeta (C-05), el servicio (C-04)
`desempeno.json`	R^2 fuera de muestra por nuisance, comparación de learners	La puerta, la revisión de identificación

El bundle es lo que hace audible la frase «el plano de datos entrega las exigencias del estimador». No es un documento que alguien lee: es un directorio que el estimador carga, que la puerta evalúa y que el registro conserva. Un bundle que falta alguno de esos archivos no es un bundle: es una tabla suelta que viola el invariante del Capítulo 2.

9.9 El trimming como cambio de estimando

El trimming de (Crump et al. 2009) merece su propia discusión porque es la decisión más fácil de tomar mal. La regla — recortar las unidades con propensión fuera de $[\alpha, 1-\alpha]$ — es estadísticamente correcta para reducir la varianza, pero tiene una consecuencia que el pipeline no puede ignorar: **el estimando cambia**. Ya no es el ATE de la población completa sino el ATE de quienes quedan dentro del soporte.

Esa distinción no es un matiz técnico: cambia quién queda incluido en la afirmación y quién no. Si el 57,5 % de la población queda fuera (como en el caso canónico), la afirmación publicada cubre al 42,5 % restante, y reportarla como «ATE de la campaña completa» es simplemente falso. La ficha del modelo (Capítulo 4) y el servicio (Capítulo 13) documentan la población del estimando exactamente por eso: para que la abstención del Capítulo 13 sepa dónde el efecto no aplica.

La decisión correcta no es «trimming sí o no» sino: **¿qué estimando sirve a la decisión que el sistema apoya?** Si la decisión necesita el efecto sobre toda la población, el trimming rompe la pregunta. Si la decisión necesita el efecto sobre quienes tienen soporte, el trimming es la respuesta correcta y el estimando cambia. Lo que no es aceptable es recortar y no declararlo.

9.10 El fallo que cada exigencia previene y el que no

Cada exigencia del C-12 tiene un fallo que previene y uno que no. Distinguirlos es la diferencia entre usar el contrato y usarlo como decoración:

Exigencia	Fallo que previene	Fallo que NO previene
Cross-fitting	El estimador memoriza las unidades y subestima la confusora: 0,86 vs 0,95	La confusión no observada que ninguna partición corrige
Soporte común	La ponderación IPW explota porque pocas unidades dominan: ESS 85 de 3 000	Que la población recortada no sea la que la decisión necesita
Nuisance con tasa	La mala especificación estructural del outcome: plug-in vs DR en 0,30	Que un learner gane por sobreajuste a la validación
Estimando ante trimming	Reportar el ATE recortado como si fuera el completo: 42 % vs 100 %	Que el trimming sea la decisión correcta para la pregunta

La lectura: cada exigencia cierra una puerta específica. El cross-fitting cierra la puerta de la fuga; el soporte común cierra la puerta de la ponderación explosiva; el nuisance cierra la puerta de la mala especificación; el estimando declarado cierra la puerta del recorte silencioso. Pero ninguna cierra la puerta de la confusión no observada: esa la cierra la teoría causal, no la ingeniería de datos.

9.11 Cuándo sabes que tu DML está mintiendo

Hay cinco señales que, juntas, dicen que el resultado de un DML no es confiable aunque el número parezca razonable:

1. **El efecto cambia al cambiar la semilla.** Si reordenar las filas o cambiar la semilla de los folds mueve el efecto más que el error estándar, hay fuga. El cross-fitting es la corrección; sin él, la estimación no reproduce.
2. **Los pesos IPW valen miles.** Si algunas ponderaciones $1/\hat{e}$ son órdenes de magnitud mayores que otras, unas pocas unidades dominan la estimación y el intervalo es engañosamente estrecho.
3. **El plug-in y el doblemente robusto difieren.** Si los dos estimadores — uno con nuisance lineal y otro con no lineal — dan efectos distintos fuera del error estándar, al menos uno está mal especificado. La dirección de la diferencia dice cuál.
4. **Cambiar el learner cambia el efecto.** Si pasar de LASSO a boosting mueve el efecto más que el error estándar, la especificación del nuisance no es robusta y el resultado es un artefacto del learner elegido.
5. **El estimando no declara su población.** Si la ficha no dice sobre quién vale el efecto, nadie puede saber si el recorte de soporte cambió la población de la afirmación.

Ninguna de estas señales es el error predictivo del nuisance. Un pipeline que solo monitorea R^2 no ve ninguna de las cinco. Por eso el C-12 exige métricas específicas: ESS/n, r2 OOS, comparación de learners, y la decisión de trimming con su población declarada.

9.12 Ejercicios y notas bibliográficas

9.12.1 Ejercicios conceptuales

1. Calcule la SMD y las desviaciones del caso del cross-fitting: OLS tiene sesgo $+0{,}41$ y sin CF $-0{,}14$; ¿qué distingue la naturaleza del problema en cada uno, y cuál se arregla con mejor especificación y cuál solo con mejor pipeline? *Pista:* uno es problema de especificación del modelo (es problema del método), el otro de fuga de dados (naturaleza pipeline).
2. Con la población de la Ex. 2 (ESS = 85 sobre 3 000), describa cuánto de la muestra tiene $\hat{e} < 0,05$ y diga cuál es el estimando post-trimming. ¿Es correcto reportar el mismo estimando como «ATE de la campaña completa»? *Pista:* ni objetivo declarado sobre

poblacion cambió: 42 % conservado implica que el 58 % quedó fuera por la regla de Crump, y la ficha lo documenta.

3. La comparación de learners de este capítulo añade LASSO y un no lineal. Dé las dos hipótesis sobre la esparsidad y suavidad y cómo se relacionan con la condición $\sqrt{n}a_n^2 \to 0$ de la edición de método. *Pista:* con s variables verdaderas y p totales (LASSO): $s \log p = o(\sqrt{n})$; la forma funcional no lineal exige condiciones de aproximación y regularidad propias.
4. ¿Por qué en el caso de outcome mal especificado la doble robustez no sube a exactamente 0,347 o baja a 0,28, cuando las cifras muestran que sigue en 0,30? Explique el papel del propensity «exacto» por diseño del DGP. *Pista:* si el propensity fuera además mala especificado, ninguna pierna de DR compensa; el control pretó la falla exclusiva al outcome.
5. Liste tres documentos que el registro espera anexos al estimador, y etiquete cada uno: ficha del estimador (Cap. 4), documento de soporte (C-12) o artefacto servible (Parte IV). *Pista:* ESS/n y el rango de $\hat{e}$ comparten dos destinos; la regla de trimming va a la ficha; la partición del cross-fitting queda en el documento.
6. Su equipo corre DML y el efecto cambia de 1 500 a 1 800 al pasar la semilla de 42 a 43. Diagnostique la causa y proponga la corrección. *Pista:* revisar la tabla de diagnósticos de la sección de este capítulo; la no-reproducibilidad ante cambio de semilla es síntoma de fuga en cross-fitting.
7. Su bundle registra ESS/n = 0,03 pero no incluye la decisión de trimming. ¿Qué invariante del C-12 se viola y qué debería pasar con la estimación? *Pista:* (3): el trimming se documenta como cambio de estimando; sin documentación, la estimación queda impugnada (6).
8. El plug-in da 0,25 y el doblemente robusto da 0,40 con el mismo dataset. El SE es 0,05. ¿Qué le dice la diferencia, y qué verificación adicional haría antes de confiar en uno u otro? *Pista:* la diferencia de 0,15 es tres SE: al menos un nuisance está mal especificado; comparar learners antes de decidir.

9.12.2 Notas bibliográficas

La teoría que este capítulo comprime vive entera en la edición de método: la ortogonalidad de Neyman y las condiciones de tasa con (Chernozhukov et al. 2018) (Caps. 10 y 11 de esa edición). La regla de trimming con varianza asintótica es derivada por (Crump et al. 2009) y el ESS ponderador es el de (Kish 1965). La contribución del capítulo es la lectura de consumo: los teoremas no se reescriben — su promesa se convierte en verificaciones con nombre propio del plano de datos (C-12), con sus cifras canónicas citadas y sus casos ejecutables en el repositorio.

Qué debe ser verdad en los datos

Que ningún modelo de nuisance prediga una unidad de su propio entrenamiento, y ningún preproceso vea su fold de prueba. Que la semilla del cross-fitting quede registrada. Que el soporte común se reporte con el rango de $\hat{e}$ y el ESS/n, y que un trimming, si lo hay, cambie el estimando y lo anuncie como tal. Que los nuisances entreguen su R^2 fuera de muestra y la comparación de al menos dos learners. Se comprueba reproduciendo los casos del capítulo — las cifras del cross-fitting citadas de la edición de método, el ESS = 85 de 3 000, la doble robustez con outcome mal especificado — con los scripts del repositorio. Lo que este capítulo certifica no es el efecto en cada dataset: es que el plano de datos entrega las cuatro condiciones para que el estimador cumpla su promesa.

CAPÍTULO 10

10 Lo que la heterogeneidad y la política le exigen al dato

Cuadro sinóptico del capítulo

Problema:	Del ATE a quién tratar: el pipeline que entrega un CATE sin validar entrega un ranking sin garantía
Tesis:	Heterogeneidad es una afirmación predictiva sobre tau(x): se valida fuera de muestra como cualquier predictor — con PEHE, calibración y Qini
Trade-off:	Ganancia de política vs costo de tratar: el umbral se elige sobre valor estimado OOS, no sobre ajuste
Pieza contratada:	Extensión C-12 (política): bundle + Qini + valor IPW OOS; criterio Q-04
Casos canónicos:	tau(x)=0,3+1,2 · x0: PEHE S 0,035 / T 0,048; política $\tau > 0$ valor 0,56 vs 0,07 aleatorio (semilla 20260710)

10.1 Este capítulo también es de consumo

Los Capítulos 12, 14, 21 y 22 de la edición de método desarrollan T-learner, S-learner, X--learner y policy learning con sus teoremas. Aquí no se eligen learners: se traduce qué le piden al dato. Un CATE no es un parámetro causal nuevo que la ingeniería deba «entender»: es un predictor de $\tau(x)$ entrenado con pseudo-outcomes o con dos regresiones, y el pipeline le exige lo mismo que a cualquier predictor — entrenamiento separado de validación, métrica OOS y declaración de cobertura. La política es un corte $\pi(x) = 1[\hat{\tau}(x) > c]$; su valor no se infiere: se estima OOS con ponderación por el inverso de la propensión.

10.2 Exigencia 1: CATE como predictor con verdad en el DGP

Lo que dice la teoría (edición de método, Cap. 12): bajo no confusión y soporte, $\tau(x) = E[Y(1) - Y(0)|X = x]$ se identifica y puede aprenderse con meta-learners. Su riesgo no es identificable sin verdad — por eso en datos reales nunca se conoce el PEHE. En un DGP sí.

La exigencia al dato: el bundle C-12 ya trae partición y propensity OOS. Para CATE se añade: pseudoresultados o dos modelos por brazo entrenados solo en train, y evaluación en test contra τ verdadero del DGP (PEHE, correlación, calibración por quintil). Sin partición OOS, el PEHE es optimista y el ranking es tautológico.

Caso canónico · S-learner vs T-learner sobre heterogeneidad en x0

DGP con $\tau(x) = 0,3 + 1,2x_0$, $n_{\text{tr}} = 1500$, $n_{\text{te}} = 1000$, semilla 20260710 del script. Ambos learners son lineales y el DGP es lineal en x_0: el problema es deliberadamente fácil para aislar la exigencia de validación.

Learner	PEHE OOS	corr($\hat{\tau}, \tau$)
S-learner	0,035	1,00
T-learner	0,048	1,00

τ medio 0,33 y desv. 1,18: la heterogeneidad es real (correlación x_0–τ de 1,00 por construcción). Los dos learners aciertan porque la forma es lineal; el punto no es quién gana por 0,013 sino que **sin test OOS con verdad no habría forma de saberlo**: en datos reales ese ranking se decide con Qini, no con PEHE (sección siguiente).

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap10_heterogeneidad.py` (semilla 20260710) reproduce la tabla. Mismos datos alimentan la curva Qini de la sección siguiente.

10.3 Exigencia 2: el valor de una política se estima, no se asume

Lo que dice la teoría (Cap. 21): el valor de una política π es $V(\pi) = E[Y(\pi(X))]$ y se estima por IPW OOS: promedio $\hat{V}$ sobre $1[D = \pi(X)]Y/e_{\pi(X)}$. Su Qini es el área entre la curva de ganancia ordenada por $\hat{\tau}$ y la aleatoria.

La exigencia al dato: toda política declara umbral c, proporción tratada y valor IPW OOS con IC; y la comparación es contra aleatorio y contra tratar-a-todos, no contra el mejor in-sample. La propensión $\hat{e}$ usada es la del bundle C-12 OOS, no reestimada para la política.

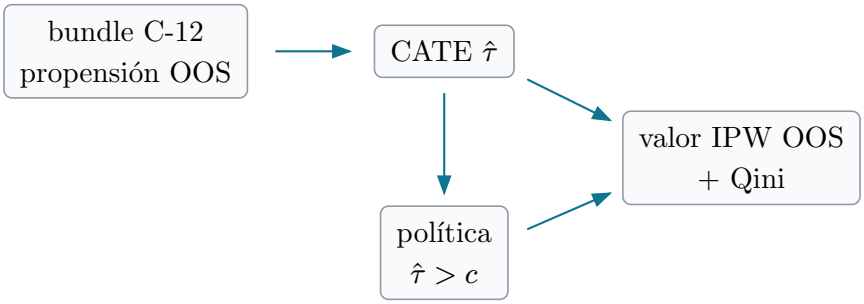


Figura 10.1: Heterogeneidad y política como extensión C-12: del bundle con propensión OOS al CATE y su valor IPW OOS. Sin OOS el ranking es tautológico.

Caso canónico · La política $tau>0$ contra el azar

Sobre el mismo test OOS ($n = 1000$), valor IPW (mayor es mejor):

Politica	Trata	Valor IPW
Tratar a todos	100 %	0,257
Aleatorio 50 %	50 %	0,068
S-learner $\hat{\tau} > 0$	60 %	0,548
T-learner $\hat{\tau} > 0$	60 %	0,562

La política $\hat{\tau} > 0$ supera a tratar-a-todos en 0,29 y al azar en 0,48 con el mismo presupuesto de 60 %. La curva Qini correspondiente (orden por $\hat{\tau}$) domina a la diagonal aleatoria en todo el rango — pero esa dominancia se declara solo OOS: in-sample toda política con umbral afinado parece ganadora.

10.4 Contrato C-12 extensión: heterogeneidad y valor de política

Contrato C-12 · Exigencias del estimador — heterogeneidad y política

Operaciones	*evaluar_cate*: entrena S/T-learner solo en train y reporta PEHE/corr OOS contra tau del DGP; *valorar_politica*: estima V(pi) IPW OOS y curva Qini/Uplift OOS; *declarar_umbral*: fija c y proporción tratada antes de mirar valor.
Entrada	Bundle C-12 (partición, propensity OOS, semilla) + learners declarados + umbral candidato.
Salida	Tabla PEHE/corr OOS por learner + valor IPW OOS por política con IC + Qini OOS + declaración de proporción tratada.
Invariantes	(1) Todo CATE se evalúa OOS contra verdad del DGP o, sin verdad, con Qini IPW OOS — nunca in-sample. (2) Valor

	IPW usa propensity del bundle, no reestimado. (3) Umbral y % tratado se declaran antes de mirar valor.
Modos de fallo	Reportar PEHE train como generalización. Elegir c maximizando valor in-sample. Usar propensity reestimado para inflar V(pi).
Prueba de aceptación	(a) DGP tau de x0 (semilla 20260710) PEHE menor a 0.05 y ranking correcto. (b) Politica con umbral 0 supera aleatorio OOS.

10.5 Ejercicios y notas bibliográficas

10.5.1 Ejercicios conceptuales

1. Con el DGP del caso canónico, un learner con PEHE 0,035 parece excelente. ¿Por qué en datos reales ese número no existe y qué lo sustituye? *Pista:* Qini IPW OOS.
2. Si eliges c maximizando valor in-sample y luego reportas ese valor, ¿qué invariante violas y cuál es el sesgo? *Pista:* C-12 ext (3).
3. Dos políticas tratan 60 % pero una ordena mejor por $\hat{\tau}$. ¿Cómo lo ves en la Qini sin mirar PEHE? *Pista:* área entre curva y diagonal.

10.5.2 Notas bibliográficas

Meta-learners y Qini: (Künzel et al. 2019) y (Athey y Imbens 2016) (Causal Forest). Valor IPW de política: (Athey y Wager 2021). La lectura como extensión C-12 es propia: sin propensity OOS no hay valor honesto.

Qué debe ser verdad en los datos

Que todo CATE traiga su evaluación OOS — PEHE contra verdad sintética o Qini IPW OOS — y toda política su valor IPW OOS con % tratado declarado antes. Se comprueba con el script semilla 20260710: sin OOS el ranking colapsa; con OOS la política $\tau > 0$ domina al azar en 0,48. Lo que no certifica: que el orden sea causalmente correcto en datos reales — eso exige el diagnóstico del capítulo siguiente.

CAPÍTULO 11

11 Diagnóstico sin verdad

Cuadro sinóptico del capítulo

Problema:	Sin verdad no hay métrica: ¿cómo saber si el efecto estimado es señal o artefacto?
Respuesta:	Tres pruebas que no necesitan verdad: placebo por permutación, control negativo y sensibilidad a confusor oculto
Criterio:	Falsación OOS: si el placebo da señal o el control negativo se mueve, la identificación falla
Pieza contratada:	Contrato C-08 (sensibilidad) + Q-02 (supuestos clasificados); umbral Gamma-star
Casos canónicos:	Placebo 500 perms: nula [−0,078,0,071] vs ATE 0,77; control neg. p=0,64; Gamma 0,8 cruza, 1,2 invierte (semilla 20260711)

11.1 Sin verdad, con falsación

Los Capítulos 9–10 validan contra verdad sintética OOS. En producción no hay τ: el dato no trae su etiqueta causal. La salida no es renunciar a validar sino cambiar de métrica: falsar. Tres ideas con contrato:

- **Placebo por permutación:** rompe D–Y permutando D; bajo la nula de no efecto la distribución nula es el placebo y el ATE observado debe ser atípico.
- **Control negativo:** un outcome Y_{nc} que comparte confusores con Y pero no debe ser afectado por D; si D «afecta» a Y_{nc}, la confusión es observable.
- **Sensibilidad:** cuánto confusor oculto U haría cruzar el IC de «ATE» el 0 (Gamma--star paramétrico del DGP demo).

Las tres son OOS por construcción y no necesitan verdad: necesitan un DGP compañero donde la violación es inyectable.

11.2 Placebo y control negativo

Lo que dice la teoría (edición de método, Cap. 14): placebo y control negativo son tests de falsación, no de confirmación: un p grande no prueba identificación, solo no la refuta.

La exigencia al dato: toda estimación entrega junto a su IC un p-placebo (posición del ATE en la nula permutada, 500 perms) y un ATE de control negativo con p; ambos OOS y con semilla.

Caso canónico · El placebo que no da señal y el control que no se mueve

DGP base ATE=0,8, $n = 2000$, semilla 20260711. ATE observado 0,767 IC [0,696,0,838] $p < 10^{-9}$.

Prueba	ATE	IC placebo / p
Placebo 1 perm	−0,009	nula sd 0,037
Placebo 500 perms	−0,002	[−0,078, 0,071]
Control negativo	−0,017	p=0,64

El placebo vive centrado en 0 con sd 0,037; el control negativo no se mueve ($p = 0,64$). El ATE real (0,77) es atípico en esa nula ($p \approx 0$). Falsación pasada no certifica causalidad — pero su fallo (placebo con señal) habría abortado el deploy.

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap11_diagnostico.py` (semilla 20260711) reproduce la tabla; la nula de 500 perms es la del contrato C-08.

11.3 Sensibilidad: cuánto confusor haría cruzar el cero

La promesa: Rosenbaum Gamma y el paramétrico γ_u dicen cuánto U no observado haría que el IC incluya el 0. No es p-valor: es umbral de robustez declarado.

La exigencia al dato: reporte Gamma-star — menor γ tal que IC toca 0 — con DGP inyectado $Y = 0,5X_0 + 0,8D + \gamma_u U + \varepsilon$.

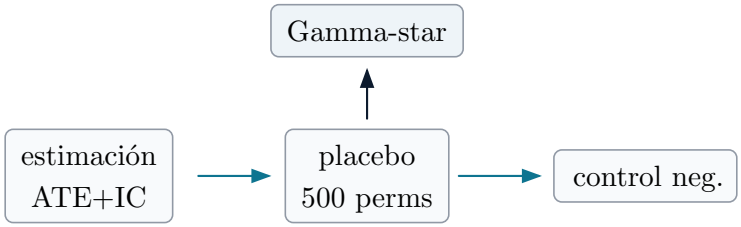


Figura 11.1: Diagnóstico sin verdad: placebo, control negativo y sensibilidad convergen en Gamma-star. Sin verdad igual hay falsación OOS.

Caso canónico · Gamma 0,8 roza, 1,2 invierte

Mismo DGP, γ_u creciente:

γ_u	ATE	IC 95 %	Lectura
0,0	0,767	[0,696,0,838]	robusto
0,5	0,586	[0,502,0,670]	robusto
0,8	0,115	[0,017,0,213]	roza 0
1,2	−0,214	[−0,335,-0,092]	invierte

Gamma-star≈0,8 cruza, 1,2 invierte signo. El contrato exige declarar Gamma-star en la tarjeta (C-05) como «qué tamaño de confusor oculto haría fracasar la conclusión».

11.4 Contrato C-08: sensibilidad y falsación

Contrato C-08 · Sensibilidad y falsación sin verdad

Operaciones	*placebo*: permuta D y reporta nula + p-placebo OOS; *control_negativo*: estima ATE sobre Y_nc que no debe moverse; *sensibilidad*: inyecta U con gamma creciente y reporta Gamma--star.
Entrada	Bundle C-12 + Y_nc declarado + grid de gamma_u para DGP de sensibilidad.
Salida	p-placebo + nula 500 perms + ATE_nc con p + curva ATE(gamma) y Gamma-star declarado en tarjeta.
Invariantes	(1) Placebo y control negativo son OOS con semilla. (2) Fallo de cualquiera aborta deploy (Q-02). (3) Gamma-star se declara en C-05, no se oculta.
Modos de fallo	p-placebo con 20 perms y declarar p. Elegir Y_nc post-hoc que «pasa». Reportar Gamma-star sin IC.
Prueba de aceptación	(a) Placebo nula [−0,078,0,071] contiene 0 y excluye ATE 0,77. (b) Control negativo p=0,64. (c) Gamma-star 0,8 toca, 1,2 invierte.

11.5 Ejercicios y notas bibliográficas

11.5.1 Ejercicios conceptuales

1. Tu placebo da p=0,04. ¿Pasó o falló? ¿Qué acción del sistema corresponde? *Pista:* Q-02, revisar identificación.
2. Eliges Y_nc después de ver que pasa. ¿Qué invariante violas? *Pista:* Y_nc se declara con el estimando.
3. Gamma-star =0,3 en tu tarjeta. ¿Qué le dices al decisor? *Pista:* poco robusto: confusor pequeño cruza.

11.5.2 Notas bibliográficas

Placebo por permutación: (Imbens y Rubin 2015); control negativo: (Lipsitch et al. 2010); sensibilidad Rosenbaum Gamma: (Rosenbaum y Rubin 1983). Lectura pipeline: todo falsación es OOS aquí.

Qué debe ser verdad en los datos

Que toda estimación llegue con p-placebo 500 perms, control negativo y Gamma-star declarado. Se comprueba con semilla 20260711: nula correcta, control inmóvil, Gamma--star 0,8/1,2. Sin eso no hay deploy — aunque el ATE sea «significativo».

PARTE III

Operación

Esta parte lleva el plano de datos a producción: la puerta que decide qué versión llega al alias champion, el servicio que responde con efecto e incertidumbre dentro del soporte y se abstiene fuera, el monitor que vigila las señales en ventanas de producción, y la tarjeta que viaja con el artefacto documentando estimando, supuestos y límites. Los cuatro capítulos comparten una regla: ninguna pieza promete más de lo que puede verificar, y toda decisión queda registrada.

CAPÍTULO 12

12

La puerta causal en el plano de datos

Cuadro sinóptico del capítulo

Problema:	Una versión nueva de datos o features puede romper la identificación aunque el estimador no cambie
Decisión:	La puerta evalúa señales OOS contra umbrales declarados y produce pasa, alerta o falla antes de promover
Pieza contratada:	Contrato C-06 (compuerta de registro): solo la versión en pasa llega al alias champion
Casos canónicos:	v0001 pasa, v0002 alerta por deriva, v0003 falla por solapamiento roto (semilla 20260712)
Conecta:	El registro C-09 guarda las versiones; el monitor C-07 produce las señales en producción

12.1 Promover datos, no solo modelos

En un sistema predictivo, la decisión de promover es comparación de métricas: si la versión nueva gana en validación, reemplaza a la vieja. En un sistema causal esa comparación es insuficiente, porque el objeto que se promueve no es solo un modelo: es un **bundle** de datos, features y estimador cuya combinación sostiene una afirmación causal. Una versión nueva del dataset — una reextracción, un cambio en el origen, un filtro de negocio — puede romper la identificación sin que el error predictivo se mueva. La puerta causal existe para interceptar exactamente eso: es la pieza que decide, con señales medibles, si una versión puede llegar a producción.

La puerta no sustituye el juicio: lo disciplina. Las señales orientan una revisión; no prueban la causa del cambio. Lo que la puerta garantiza es más modesto y más útil: que ninguna versión con identificación rota llega al alias `champion` sin que alguien lo vea y lo acepte explícitamente.

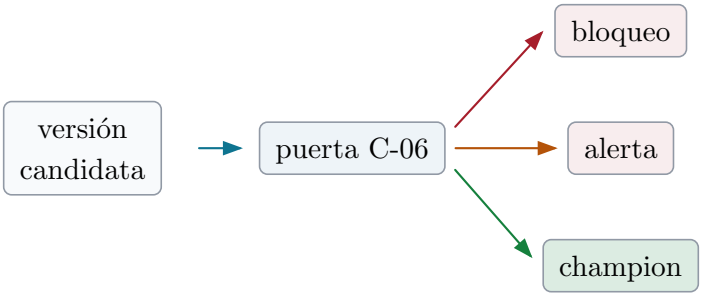


Figura 12.1: La puerta causal evalúa tres señales contra umbrales declarados y produce tres estados. Solo la versión en pasa llega al alias champion; alerta y falla exigen decisión humana explícita.

12.2 Las tres señales de la puerta

La puerta consume las mismas señales del Capítulo 7 (C-02) y las traduce en estados de promoción. Tres señales, tres preguntas distintas:

1. **Solapamiento.** ¿Hay con quién comparar en la nueva versión? Se mide con la fracción de unidades con propensión estimada en zona de comparación. Si cae bajo el mínimo, la puerta falla: el estimando deja de ser computable.
2. **Deriva.** ¿La población de la versión nueva se parece a la que ajustó? Se mide con el PSI por covariable. Si cruza el umbral, la puerta alerta: la población cambió y hay que revisar la identificación antes de reentrenar.
3. **Balance.** ¿La asignación del tratamiento cambió de forma? Se mide con el SMD máximo entre tratados y controles. Si cruza el umbral, la puerta alerta: el mecanismo de asignación no es el declarado.

12.3 El contrato C-06: compuerta de registro

Contrato C-06 · Compuerta de registro causal

Operaciones	*evaluar*: recibe una versión candidata y produce estado pasa, alerta o falla con motivo nombrado; *promover*: apunta el alias champion a la versión solo si el estado es pasa; *bloquear*: rechaza la promoción de una versión en falla y notifica; *aceptar_alerta*: permite promover una versión en alerta solo con aceptación explícita registrada.
Entrada	La versión candidata (bundle C-12), las señales del C-02 (solapamiento, PSI, SMD) y los umbrales declarados en la tarjeta.
Salida	El estado de la puerta con motivo, la decisión de promoción registrada y el alias champion actualizado o bloqueado.
Invariantes	(1) Ninguna versión en estado falla llega al alias champion. (2) Una versión en alerta solo se promueve con aceptación explícita de una persona, registrada. (3) El motivo de cada estado nombra

	la señal y su valor contra el umbral. (4) Los umbrales vienen de la tarjeta (C-05) y no se ajustan después de mirar la señal. (5) Toda decisión de promoción queda registrada con hash de la versión y estado de la puerta.
Modos de fallo	Promover una versión en falla. Ajustar el umbral después de ver la señal para que pase. Registrar la promoción sin el motivo de la puerta. Reentrenar en lugar de revisar la identificación cuando la señal es deriva.
Prueba de aceptación	(a) La versión sana pasa y llega a champion. (b) La versión con deriva alerta y no se promueve sin aceptación. (c) La versión con solapamiento roto falla y queda bloqueada. Semilla 20260712 reproduce los tres estados.

12.4 Caso canónico: tres versiones, tres estados

Caso canónico · Tres versiones del mismo dataset, tres decisiones de puerta

Sobre `observed_train.csv` de `real/jobs` (semilla 20260712 del script), tres versiones candidatas evaluadas contra los umbrales declarados `smd_max` 0,25, `psi_alerta` 0,25, `solape_min` 0,50:

Versión	n	PSI	SMD	Solape	Estado
v0001 sana	700	0,011	0,22	100 %	**pasa**
v0002 deriva	1 033	1,347	0,19	100 %	**alerta**
v0003 positividad	270	3,237	5,24	0 %	**falla**

v0001 es una submuestra sana de la misma población: todas las señales dentro de umbral, la puerta produce `pasa` y el alias champion la acepta. **v0002** envejece la población diez años: el PSI de la edad dispara la alerta — la identificación puede seguir siendo válida, pero alguien tiene que revisarla antes de reentrenar. **v0003** hace la asignación casi determinista por ingreso previo: el solapamiento cae a 0 %, la puerta produce `falla` y la promoción queda bloqueada. Solo v0001 llega a champion; v0002 y v0003 quedan en el registro con su estado y su motivo, esperando revisión humana.

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap12_puerta.py` (semilla 20260712) reproduce la tabla de los tres estados. La implementación de referencia de la puerta con estados y alias atómico: `04_produccion_causal/src/causalops/puerta.py` y `registro_local.py`.

12.5 Ejercicios y notas bibliográficas

12.5.1 Ejercicios conceptuales

1. La versión v0002 (deriva) produce alerta, no falla. ¿Por qué la puerta no la bloquea automáticamente, y qué decisión humana exige? *Pista:* la deriva cambia la población pero no necesariamente rompe la identificación; alguien tiene que revisar el grafo antes de decidir.
2. Con el umbral de solapamiento en 0,50, la v0003 falla. Un equipo propone bajarlo a 0,10 para que pase. ¿Qué invariante del C-06 viola esa propuesta? *Pista:* los umbrales vienen de la tarjeta y no se ajustan después de mirar la señal.
3. ¿Qué diferencia hay entre reentrenar y revisar identificación como respuesta a una alerta de deriva? *Pista:* reentrenar ajusta el estimador a la nueva población; revisar identificación pregunta si la nueva población sigue siendo la del estimando declarado.

12.5.2 Notas bibliográficas

La compuerta de registro es la pieza del Capítulo 19 de la edición de método (model card y registro) traducida al plano de datos. Las señales que consume son las del C-02 (Capítulo 7 de este libro): balance, deriva y solapamiento con umbrales calibrados. La decisión de promover con estados y alias atómico es la que implementa `causalops.puerta` con `registro_local` en el repositorio de referencia.

Qué debe ser verdad en los datos

Que ninguna versión con identificación rota llegue a producción sin revisión humana. Se comprueba con la semilla 20260712: la versión sana pasa, la versión con deriva alerta y exige aceptación explícita, la versión con solapamiento roto falla y queda bloqueada. Lo que la puerta no garantiza: que la versión en pasa sea causalmente válida — solo que sus señales observables están dentro de umbral. La identificación sigue siendo una declaración, no una medición.

CAPÍTULO 13

13

Servicio y abstención

Cuadro sinóptico del capítulo

Problema:	El servicio que siempre responde sirve números que nadie puede defender fuera del soporte
Decisión:	El servicio responde con efecto, intervalo y versión dentro del soporte; fuera, se abstiene sin dar número
Pieza contratada:	Contrato C-04 (servicio de estimación): el bundle viaja, el dato no
Casos canónicos:	Dentro del soporte responde 1596.0 IC [1004.8, 2187.3]; fuera (age 99) se abstiene (semilla 20260713)
Conecta:	El bundle viene del registro C-09; el soporte de la tarjeta C-05; la puerta C-06 decide qué versión se sirve

13.1 El bundle viaja, el dato no

Un servicio causal en producción no lee la tabla de entrenamiento: lee el **bundle** — la versión registrada del registro, con su estimación, su tarjeta y su soporte declarado. Esa separación es la diferencia entre un notebook que corre y un sistema que opera. El dato de entrenamiento puede pesar gigabytes y contener columnas que el servicio no debe ver (la pared del Capítulo 5); el bundle pesa kilobytes y contiene exactamente lo que el servicio necesita para responder: efecto, intervalo, versión, y el rango del soporte.

La implicación operativa es directa: el despliegue — Docker, Kubernetes, una función serverless — empaqueta el bundle, no los datos. La imagen es pequeña, reproducible y auditable: cualquier persona puede verificar que el número servido es el registrado.

13.2 La abstención es una respuesta de primera clase

Un servicio predictivo devuelve siempre un número: la predicción es su trabajo. Un servicio causal tiene una opción más que es más honesta que cualquier número: la abstención. Si la

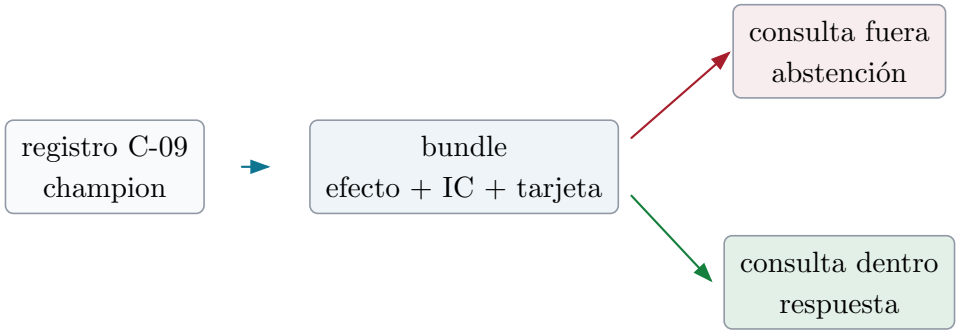


Figura 13.1: El servicio sirve el bundle del registro, no los datos de entrenamiento. Dentro del soporte responde con efecto e intervalo; fuera, se abstiene sin número.

consulta cae fuera del soporte — la población donde el efecto fue estimado —, el número que se daría sería una extrapolación sin respaldo, y servirlo en silencio es exactamente el fallo que este libro existe para evitar.

La abstención no es un error técnico: es una respuesta operativa con estado, motivo y versión. Devuelve HTTP 200, no 500; el motivo nombra la covariable y el rango del soporte; y la versión dice a qué registro pertenece la respuesta. Un cliente puede distinguir entre «no puedo responder» y «fallé al responder», y esa distinción es la que permite operar.

El soporte se declara en la tarjeta (C-05): el rango de cada covariable en el conjunto de entrenamiento, o una regla más refinada si el sistema la tiene. El servicio lo verifica en cada consulta: si cualquier covariable del perfil cae fuera, la respuesta es abstención sin ningún número.

13.3 El contrato C-04: servicio de estimación

Contrato C-04 · Servicio de estimación causal

Operaciones	*consultar*: responde a un perfil con efecto, intervalo, versión y nota de cobertura, o con abstención y motivo; *salud*: reporta la versión servida y su estado; *resolver_version*: lee el alias champion del registro y carga el bundle correspondiente.
Entrada	El perfil de consulta (valores de las covariables del esquema) y el bundle del registro (C-09) con su soporte declarado en la tarjeta (C-05).
Salida	Dentro del soporte: estado respuesta, efecto, IC 95, unidad, qué cubre el IC, versión, nota. Fuera: estado abstención, motivo nombrando la covariable y el rango, versión. Ningún número en la abstención.
Invariantes	(1) El servicio no lee datos de entrenamiento: solo el bundle del registro. (2) La respuesta incluye siempre efecto, intervalo, qué cubre el intervalo y versión. (3) Fuera del soporte declarado, la

	respuesta es abstención sin ningún número. (4) Dos consultas idénticas devuelven respuestas idénticas (determinismo). (5) El motivo de la abstención nombra la covariable y el rango del soporte. (6) La versión servida es la que el alias champion resuelve, no una hardcodeada.
Modos de fallo	Servir un número fuera del soporte. Leer la tabla de entrenamiento en lugar del bundle. Responder sin versión o sin qué cubre el intervalo. Una abstención que devuelve el efecto en otro campo. Dos consultas idénticas con respuestas distintas.
Prueba de aceptación	(a) Una consulta dentro del soporte responde con efecto 1596.0 e IC [1004.8, 2187.3] sobre real/jobs (semilla 20260713). (b) Una consulta con age 99 se abstiene con motivo que nombra age y el rango [18, 54], sin número. (c) Dos consultas idénticas devuelven la misma respuesta.

13.4 Caso canónico: respuesta y abstención

Caso canónico · El servicio que responde y el que se abstiene

Sobre `observed_train.csv` de `real/jobs` (semilla 20260713 del script), el servicio sirve el bundle v0001 del registro. Dos consultas lo ejercitan:

Consulta	Estado	Respuesta
Perfil dentro	respuesta	efecto 1596.0, IC [1004.8, 2187.3], versión v0001, nota de cobertura poblacional
age 99 (fuera)	abstención	motivo: age 99 fuera de [18.0, 54.0], versión v0001, sin número

El perfil dentro del soporte (age 30, educ 10, black 1, hispan 0, married 0, nodegree 1, re74 2000, re75 1500) recibe el efecto con su intervalo y su versión. El perfil con age 99 — muy por encima del rango de entrenamiento — recibe abstención con el motivo nombrando la covariable y el rango. Determinismo: dos consultas idénticas devuelven exactamente la misma respuesta. El humo del servicio (tres verificaciones del contrato) pasa.

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap13_servicio.py` (semilla 20260713) reproduce la tabla de respuesta y abstención. La implementación de referencia del servicio con FastAPI y abstención: `07_docencia/semana_4/solucion/servicio.py` y `servicio_core.py` el Dockerfile correspondiente empaqueta el bundle, no los datos.

13.5 Despliegue: local y nube

El mismo contrato corre en local y en nube, porque el bundle es portable. En local, `uvicorn servicio:app --port 8000` sirve la API en el puerto de la máquina. En nube, el mismo servicio se empaqueta en una imagen Docker que copia el bundle (no los datos) y se despliega en cualquier plataforma que ejecute contenedores: Kubernetes, Cloud Run, Lambda con adaptadores. La prueba de humo es idéntica en los dos casos: dentro del soporte responde, fuera se abstiene, determinismo.

Lo que cambia entre local y nube no es el contrato: es la operación alrededor. En nube se añaden secretos, escalado, logging y monitoreo de plataforma; el contrato C-04 sigue siendo el mismo y la prueba de aceptación sigue siendo las tres verificaciones del caso canónico. Un despliegue que pasa el humo en local y falla en nube no tiene un problema de contrato: tiene un problema de infraestructura, y se diagnostica como tal.

13.6 Ejercicios y notas bibliográficas

13.6.1 Ejercicios conceptuales

1. El servicio responde HTTP 200 tanto para la respuesta como para la abstención. ¿Por qué la abstención no es un error 4xx o 5xx, y qué gana el cliente con esa distinción? *Pista:* un error 500 significa «fallé»; una abstención significa «no puedo sostener ese número aquí».
2. Una consulta llega con age 55, apenas por encima del rango [18, 54]. El equipo propone «responder igual, con una nota». ¿Qué invariante del C-04 rompe esa propuesta? *Pista:* (3): fuera del soporte la respuesta es abstención sin ningún número; no hay «número con nota».
3. ¿Por qué el servicio lee el bundle del registro y no la tabla de entrenamiento? Dé dos razones: una de operación (tamaño, pared) y una de auditoría (reconstrucción). *Pista:* la pared del Capítulo 5 prohíbe que la verdad llegue al servicio; el hash del bundle permite reconstruir qué versión se sirvió.

13.6.2 Notas bibliográficas

El servicio de estimación es el Capítulo 15 de la edición de método (despliegue) traducido al plano de datos: el bundle viaja, el dato no. La abstención como respuesta de primera clase es la decisión del Capítulo 2 de este libro (las cinco acciones del sistema). La implementación de referencia con FastAPI, humo y Docker está en el repositorio de la semana 4 de docencia: `servicio.py`, `servicio_core.py`, `humo.py` y `Dockerfile`.

Qué debe ser verdad en los datos

Que el servicio responda siempre con efecto, intervalo, qué cubre el intervalo y versión dentro del soporte, y que fuera del soporte la respuesta sea abstención sin ningún número,

con motivo que nombra la covariable y el rango. Que el servicio lea el bundle del registro y no la tabla de entrenamiento. Que dos consultas idénticas devuelvan la misma respuesta. Se comprueba con la semilla 20260713: dentro responde 1596.0 [1004.8, 2187.3], age 99 se abstiene con motivo, determinismo. Lo que el servicio no garantiza: que el número dentro del soporte sea causalmente correcto — eso lo decidieron la identificación, la estimación y la puerta antes de llegar aquí.

CAPÍTULO 14

Observabilidad de supuestos en producción

Cuadro sinóptico del capítulo

Problema:	El servicio puede seguir respondiendo con normalidad mientras los supuestos que sostienen el efecto dejan de valer
Decisión:	Cuatro señales causal-aware, no solo drift de inputs: marginal, mecanismo de asignación, instrumento y estabilidad del efecto
Pieza contratada:	Contrato C-07 (monitoreo causal): ventanas, señales, umbrales calibrados y una sola acción
Casos canónicos:	Cada cambio inyectado enciende su señal y casi ninguna otra: shift 100 %/(i), regla 100 %/(ii), instrumento 100 %/(iii), efecto 95 %/(iv) (semilla 20260714)
Conecta:	Las señales son las del C-02; la acción es una de las cinco del Capítulo 2; la puerta C-06 decide la promoción

14.1 El monitoreo genérico no basta

El monitoreo estándar de ML vigila si la distribución de las entradas o de las predicciones se aleja de la de entrenamiento. Ese monitoreo es necesario y no suficiente para un sistema causal: el servicio del Capítulo 13 puede seguir respondiendo con total normalidad mientras aparecen señales que obligan a revisar el efecto servido. Ninguna señal aislada identifica automáticamente su causa; lo que hacen es indicar que la evidencia usada para describir el sistema ya no representa la ventana reciente.

La diferencia con el monitoreo genérico es de objeto: no se vigila solo la distribución de X y de $\hat{Y}$, sino la **relación** entre covariables y tratamiento, la fuerza del instrumento cuando el diseño lo usa, y la estabilidad del efecto reestimado. Eso exige mantener disponible el pipeline de estimación completo, no solo el modelo servido.

14.2 Las cuatro señales

Definición 14.1 ***(Las cuatro señales del monitoreo causal)***

(i) *Drift marginal* de covariables: PSI y ASMD por variable, con la referencia fija. (ii) *Drift del mecanismo de asignación*: el propensity aprendido en la referencia se puntúa sobre la ventana y su AUC se compara con el de validación. (iii) *Fuerza del instrumento*: el F de la primera etapa para el instrumento excluido, no el F global del modelo. (iv) *Estabilidad del efecto*: el efecto se reestima en la ventana y se contrasta formalmente con el de referencia, propagando ambos errores estándar.

Cada señal responde una pregunta distinta, y su diseño evita una trampa específica:

1. **La (i) no usa p-valores.** Con ventanas grandes el KS rechaza ante diferencias irrelevantes; con pequeñas, no detecta las materiales. Se reporta tamaño de efecto (PSI, ASMD) con umbrales calibrados.
2. **La (ii) no reajusta el propensity en la ventana.** Si la regla pasa a depender de otra covariable con fuerza similar, el propensity reajustado es tan predecible como el viejo y el AUC no se mueve. Puntuar el modelo de referencia sí lo detecta.
3. **La (iii) no usa el F global.** El F del modelo mide el ajuste conjunto de todos los regresores; el del instrumento excluido mide su aporte específico.
4. **La (iv) no compara un punto contra un intervalo.** El contraste correcto es entre dos estimaciones, cada una con su error estándar: el contraste z divide la diferencia de efectos por la raíz de la suma de las dos varianzas.

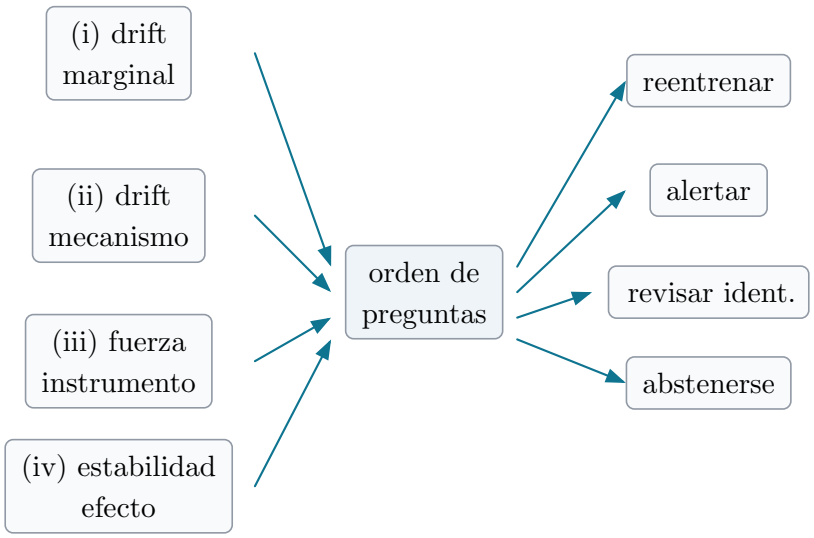


Figura 14.1: Las cuatro señales convergen en un orden de preguntas cuya primera falla decide la acción. Abstenerse va primero: sin soporte no hay estimador que juzgar. Revisar identificación va antes que alertar: un supuesto roto no se arregla con un aviso.

14.3 El contrato C-07: monitoreo causal

Contrato C-07 · Monitoreo causal

Operaciones	*evaluar*: recibe una ventana reciente y devuelve el reporte de las cuatro señales con su valor, umbral y estado, y una sola acción. *calibrar*: toma ventanas del periodo de referencia y fija los umbrales para una tasa de falsas alarmas declarada.
Entrada	La referencia fija (datos de entrenamiento con su huella). Los roles causales y el efecto con su error estándar de la tarjeta (C-05). Una función que reestima el efecto con el pipeline completo. Los umbrales vigentes con su tasa de falsas alarmas y fecha de calibración.
Salida	Para cada señal: valor, umbral y estado — nunca un semáforo sin número. Una sola acción, la de la primera pregunta que falla según el orden de la Figura 14.1.
Invariantes	(1) La referencia es fija: toda comparación usa la misma. (2) Todo umbral lleva su tasa de falsas alarmas y su fecha de calibración. (3) El propensity se puntúa con el modelo de la referencia, no se reajusta. (4) El efecto se reestima, no se predice con el modelo antiguo, y el contraste propaga los dos errores estándar. (5) Con la misma ventana y los mismos umbrales, el reporte es idéntico. (6) Una ventana demasiado pequeña o con faltantes se reporta como sin evidencia, nunca como todo en orden.
Modos de fallo	Umbral heredado de otro sistema: alarmas constantes o silencio. Reajustar el propensity en cada ventana: el cambio de regla pasa inadvertido. Comparar el efecto reciente contra el intervalo antiguo en vez de contrastar dos estimaciones. Vigilar solo el efecto y reaccionar tarde, o solo las marginales y alertar sin consecuencia.
Prueba de aceptación	Sobre un generador con verdad conocida: (a) sin cambio, la tasa de ventanas con señal no supera la declarada; (b) un desplazamiento marginal enciende la (i); (c) un cambio de regla enciende la (ii) y no la (i); (d) un instrumento sin fuerza enciende la (iii); (e) un cambio de efecto enciende la (iv). Semilla 20260714.

14.4 Caso canónico: cuatro cambios, cuatro señales

Caso canónico · Cada cambio enciende su señal y casi ninguna otra

DGP con verdad: $X_1, X_2, X_3 \sim N(0,1)$, $Z \sim N(0,1)$ instrumento, $D \sim \text{Bernoulli}(\sigma(X_1 + 0,8Z))$, $Y = 0,5D + X_1 + 0,5X_3 + \varepsilon$. Referencia de 3 000 unidades, ventanas de 400, 100 réplicas por escenario (semilla 20260714 del script). La referencia estima ATE 0,486 (SE 0,040; verdad 0,5).

Cambio inyectado	(i)	(ii)	(iii)	(iv)
Ninguno	0 %	0 %	0 %	3 %
X_3 + 1 DE	100 %	0 %	0 %	1 %
Regla X_1 pasa a X_2	0 %	100 %	0 %	3 %
Instrumento sin fuerza	0 %	0 %	100 %	2 %
Efecto 0,5 a 0,1	0 %	0 %	0 %	95 %

Cada cambio contemplado enciende la señal diseñada para verlo y casi ninguna otra. La tercera fila es la clave del capítulo: la señal (ii) detecta una regla de asignación distinta **aunque ninguna marginal se mueva** — exactamente lo que el monitoreo genérico no ve. La última fila muestra que solo la (iv) detecta el cambio de efecto, en el 95 % de las ventanas. Sin cambio, la única señal que se enciende es la (iv) en el 3 %, por debajo del 5 % nominal.

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap14_monitor.py` (semilla 20260714) reproduce la tabla completa. Las cuatro señales del repositorio de referencia: `04_produccion_causal/src/causalops/monitor.py` y `diagnosticos.py`.

14.5 De la señal a la acción

Un reporte con cuatro semáforos deja sin responder lo importante: qué hacer. El orden de preguntas de la Figura 14.1 se defiende por lo que cada fallo deja sin sentido. Sin soporte, las demás señales no significan nada: **abstenerse**. Un instrumento debilitado invalida la inferencia que depende de él; un cambio de asignación abre una revisión sin anticipar su veredicto: **revisar identificación**. Una población distinta puede dar un efecto válido para otro objetivo: **alertar**. Un cambio de efecto que llega con todo lo anterior en orden sí justifica actualizar: **reentrenar**, con la puerta del Capítulo 12 decidiendo la promoción.

Las cuatro señales vigilan supuestos observables. El que más pesa — que no haya confusión que las covariables medidas no recojan — no tiene señal: el sistema puede estar quieto en las cuatro y sesgado por una causa que nadie midió. Eso no se arregla con un quinto semáforo; se declara en la tarjeta (C-05) con su margen frente a la decisión y su fecha de revisión.

14.6 Ejercicios y notas bibliográficas

14.6.1 Ejercicios conceptuales

1. La señal (iv) con $\alpha = 0,05$ tiene una tasa nominal de falsas alarmas del 5 % por ventana. Si el sistema evalúa una ventana cada día, ¿cada cuántos días esperaría una falsa alarma? Proponga una regla para reducirla. *Pista:* una cada veinte días; exigir dos ventanas consecutivas baja la tasa y sube la demora.
2. Explique por qué reajustar el propensity en cada ventana falla justo en el caso que importa. *Pista:* si la regla pasa a depender de otra covariable con fuerza similar, el modelo reajustado es tan predecible como el viejo.
3. Un equipo monitorea solo la distribución de $\hat{Y}$. Construya un escenario donde esa distribución permanezca estable y la identificación esté rota. *Pista:* cambio de regla de asignación sin mover marginales (fila 3 del caso canónico).

14.6.2 Notas bibliográficas

El monitoreo genérico de drift es de (Huyen 2022) (Capítulo 8) y (Kästner et al. 2024). El umbral $F > 10$ para instrumentos débiles proviene de (Stock y Yogo 2005). Los criterios de balance de la señal (i) son los de (Rubin 2001) ya usados en el Capítulo 7. La lectura como contrato C-07 con orden de preguntas y cinco acciones es la contribución de este libro.

Qué debe ser verdad en los datos

Que las cuatro señales se evalúen sobre ventanas con la misma referencia fija, con umbrales calibrados y su fecha, y que el reporte termine en una sola acción — nunca cuatro semáforos sin consecuencia. Que el efecto se reestime, no se prediga. Se comprueba con la semilla 20260714: cada cambio inyectado enciende su señal y casi ninguna otra; sin cambio, la tasa de falsas alarmas no supera la declarada. Lo que el monitoreo no garantiza: que la ausencia de señales implique ausencia de confusión no medida — eso se declara en la tarjeta, no se vigila aquí.

CAPÍTULO 15

Tarjetas, registro y auditoría end-to-end

Cuadro sinóptico del capítulo

Problema:	Nadie puede reconstruir qué se estimó, sobre qué datos, con qué supuestos y bajo qué umbrales, sin leer el código
Decisión:	La tarjeta viaja con el artefacto y es la única fuente de verdad sobre el estimando, los supuestos y los umbrales
Pieza contratada:	Contrato C-05 (tarjeta causal): estimando, supuestos clasificados, umbrales, limitaciones y uso previsto
Casos canónicos:	El flujo completo sobre real/jobs: pregunta → esquema → splits → ATE 1596.0 → tarjeta → v0001 → puerta pasa → champion → servicio (semilla 20260715)
Cierra:	El libro entero en una corrida: cada contrato de los capítulos 3–14 deja su artefacto en la traza

15.1 El cierre del libro en una corrida

Este capítulo no introduce una pieza nueva del pipeline: introduce la pieza que las une. La tarjeta causal es el documento que viaja con el artefacto y declara, en un formato que una máquina puede validar, lo que el sistema sostiene y lo que no: el estimando, la población, los supuestos con su clase, los umbrales de la puerta, las limitaciones y el uso previsto. Sin ella, cada pregunta de auditoría — ¿qué se estimó?, ¿sobre qué versión del dato?, ¿con qué supuestos?, ¿bajo qué umbrales se promovió? — se responde leyendo código o preguntando a quien lo escribió, y ambas respuestas caducan.

La corrida end-to-end de este capítulo ejercita los doce contratos del libro en orden, sobre el escenario `real/jobs` del benchmark. Cada paso deja un artefacto verificable; la traza completa cabe en una pantalla y un equipo puede auditarla en minutos.

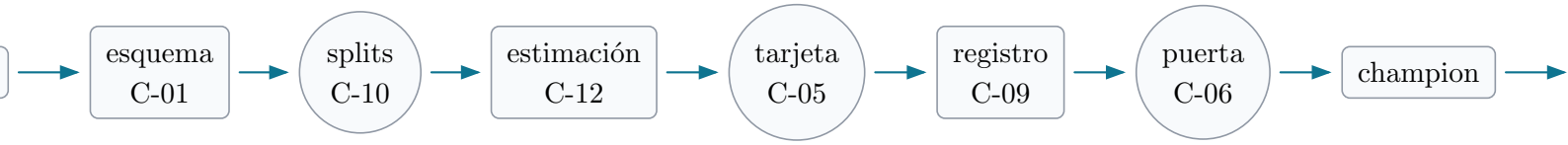


Figura 15.1: La corrida end-to-end del libro: cada contrato deja su artefacto y la traza es auditable de punta a punta. La tarjeta (C-05) es el documento que une todas las piezas.

15.2 El contrato C-05: tarjeta causal

Contrato C-05 · Tarjeta causal del modelo

Operaciones	*construir*: produce la tarjeta desde los resultados de la estimación y la puerta, con los supuestos clasificados; *validar*: comprueba la estructura contra el esquema (campos obligatorios, clases de supuesto, umbrales presentes); *registrar*: la tarjeta viaja con la versión en el registro (C-09) y el servicio la lee para responder.
Entrada	La pregunta declarada (estimando, población, supuestos). Los resultados de la estimación (efecto, SE, IC, semilla). El resultado de la puerta (chequeos con valores y umbrales). La sensibilidad de los supuestos no observables.
Salida	Un documento JSON válido con: estimando, datos (n, hash, fuente), estimación (método, efecto, SE, IC, semilla), supuestos (observables con prueba y umbral; no observables con justificación, sensibilidad y criterio de revisión), umbrales de la puerta, limitaciones y uso previsto.
Invariantes	(1) Todo supuesto no observable lleva su sensibilidad o la declaración explícita de que no hay cuantificación válida, y su criterio de revisión. (2) Todo supuesto observable lleva su prueba, su valor y su umbral. (3) El hash del dataset y la semilla son parte de la tarjeta. (4) La tarjeta valida contra el esquema antes de registrarse: una tarjeta inválida no se promueve. (5) Las limitaciones incluyen el soporte: la tarjeta dice dónde el efecto no aplica. (6) El uso previsto declara qué decisiones soporta y cuáles no.
Modos de fallo	Una tarjeta sin supuestos no observables: parece más limpia y miente. Un supuesto observable sin umbral: la puerta no puede evaluarlo. Una tarjeta que no valida contra el esquema y se registra igual. Un uso previsto que promete más de lo que el estimando cubre.
Prueba de aceptación	(a) La tarjeta del caso canónico valida contra el esquema con estimando, datos, estimación, supuestos clasificados y umbrales.

(b) El registro la guarda con hash y semilla. (c) El servicio la lee para responder dentro del soporte y abstenerse fuera. (d) Una tarjeta sin supuestos no observables o sin umbrales no valida. Semilla 20260715.

15.3 Caso canónico: la corrida completa

Caso canónico · De la pregunta al servicio en nueve pasos auditables

Sobre `observed_train.csv` de `real/jobs` (semilla 20260715 del script), la corrida completa produce esta traza:

Paso	Pieza	Artefacto que deja
1	Pregunta	ATE de treatment sobre re78, población de 1 033 unidades, supuestos declarados
2	Esquema C-01	tratamiento=treatment, resultado=re78, 8 covariables
3	Splits C-10	hash OK, semilla 20240701, la pared vigente
4	Estimación	ATE 1 596,0, SE 301,6, IC [1 004,8 – 2 187,3], ESS/n 0,71
5	Tarjeta C-05	estimando + supuestos clasificados + umbrales, válida contra esquema
6	Registro C-09	v0001 con manifest, hash del dataset y semilla
7	Puerta C-06	sana pasa, deriva alerta, positividad falla
8	Champion	v0001 — solo la ventana sana en pasa se promueve
9	Servicio C-04	dentro: respuesta con efecto e IC; age 99: abstención sin número

La trazabilidad completa: pregunta → esquema → splits (hash) → ATE 1 596,0 → tarjeta → v0001 → puerta pasa → champion → servicio. El hash del dataset (`cd5a772ead64f416…`) permite a cualquier persona verificar que el número servido es el registrado. El humo de la corrida pasa: hash OK, puerta pasa para la ventana sana, servicio responde dentro y se abstiene fuera.

Repositorio de referencia del libro

Script `Causal_AI/scripts/cap15_tarjeta.py` (semilla 20260715) reproduce la traza completa. La implementación de referencia de la tarjeta con validación contra esquema: `04_produccion_causal/src/causalops/tarjeta.py` el flujo end-to-end: `flujo.py` el esquema: `04_produccion_causal/esquemas/causal-model-card.local.schema.json`.

15.4 Qué audita una persona con esta traza

La corrida end-to-end convierte las preguntas de auditoría en verificaciones de artefactos. Esta sección desarrolla cada una, porque la diferencia entre un sistema auditable y uno que solo parece auditable está en los detalles:

1. **¿Qué se estimó?** La tarjeta lo dice: ATE de treatment sobre re78, efecto total, población de entrenamiento. No se necesita leer código ni preguntar a quien lo escribió: el JSON es la fuente de verdad.
2. **¿Sobre qué datos?** El hash del dataset en la tarjeta y el manifest del registro lo verifican: recomputar el SHA-256 del archivo de entrenamiento y comparar con el declarado. Si no coincide, la versión registrada no es la que se sirvió — y eso es un fallo del registro, no del estimador.
3. **¿Con qué supuestos?** La tarjeta los clasifica: observables con prueba y umbral, no observables con justificación y criterio de revisión. Un supuesto observable sin umbral no es un supuesto: es un deseo. Un supuesto no observable sin criterio de revisión es una deuda que nadie programó.
4. **¿Bajo qué umbrales se promovió?** La tarjeta declara los umbrales y la puerta registró su decisión con el motivo. Si alguien cambió un umbral después de ver la señal para que la versión pasara, la fecha del umbral en la tarjeta y la fecha de la promoción en el registro no coinciden — y esa discrepancia es auditable.
5. **¿Sigue siendo válido?** El monitor del Capítulo 14 evalúa las ventanas contra la misma referencia y produce una acción. La referencia es fija: si alguien la cambió sin registrar, las alertas no son comparables en el tiempo.
6. **¿Qué pasa si la consulta cae fuera del soporte?** El servicio se abstiene sin número, y la tarjeta declara el soporte. La abstención no es un error 5xx: es una respuesta de primera clase con estado y motivo.

Ninguna de estas respuestas requiere leer código. Esa es la diferencia entre un prototipo y un sistema auditable, y es el cierre del libro: el plano de datos entrega, para cada afirmación causal que el sistema hace, la evidencia de que la afirmación sigue siendo computable, registrada y vigente.

15.5 El proceso de auditoría paso a paso

Un auditor que llega a esta traza sin contexto previo puede verificarla en cinco pasos, en este orden:

Paso	Qué verificar	Cómo
1	La pared	Intentar cargar `truth_*.csv`; debe fallar con `FileNotFoundError` — ese fallo es el contrato cumplido

Paso	Qué verificar	Cómo
2	El hash del dataset	Recomputar el SHA-256 del CSV de entrenamiento y comparar con el manifest del registro; si no coincide, la versión no es la que se sirvió
3	La puerta	Verificar que ninguna versión en estado `falla` llegó al alias `champion`; si hay promociones en `alerta`, buscar la aceptación explícita registrada
4	La tarjeta	Validar el JSON contra el esquema: estimando, supuestos clasificados, umbrales, limitaciones, uso previsto; una tarjeta sin no observables es inválida
5	El servicio	Enviar una consulta dentro del soporte (debe responder con efecto e IC) y una fuera (debe abstenerse sin número); dos idénticas deben devolver lo mismo

Los cinco pasos se corren en minutos. El tiempo que no se gasta en verificar se gasta en discutir: **¿es el estimando correcto para la decisión que el sistema apoya?** — y esa pregunta no la responde la traza, la responde el dominio.

15.6 Mantenimiento de la tarjeta

La tarjeta no es un documento que se escribe una vez: es un artefacto vivo que viaja con cada versión. Cuando algo cambia — el estimador, el dataset, los umbrales, los supuestos — la tarjeta cambia con él, y el registro conserva la historia:

Qué cambia	Qué pasa con la tarjeta	Qué queda en el registro
El dataset	El hash cambia; la tarjeta se reescribe con el nuevo hash	Una versión nueva con su manifest
El estimador	El método y la estimación cambian; la tarjeta se reescribe	Una versión nueva con su estimación
Los umbrales	La tarjeta se reescribe con los nuevos umbrales y su fecha de calibración	La puerta registra la decisión con los umbrales vigentes
Un supuesto	Si es observable: cambia su valor o su umbral; si es no observable: cambia su sensibilidad o su criterio de revisión	El monitor evalúa con los umbrales de la tarjeta vigente
El soporte	Los rangos cambian; la abstención del servicio se recalibra	El servicio lee la tarjeta de la versión vigente

La regla es simple: **cada versión del sistema tiene su tarjeta, y el alias `champion` apunta a una de ellas**. Si alguien quiere saber qué supuestos sostenían la versión de la semana pasada, lee la tarjeta de esa versión — no la vigente. El registro (Capítulo 4) garantiza que esa historia no se pierde.

15.7 La tarjeta como gobernanza

Hay una lectura del C-05 que va más allá de la documentación: la tarjeta es la pieza que hace **gobernables** los sistemas causales. Sin ella, gobernar un sistema de decisión significa confiar en quien lo construyó; con ella, significa verificar artefactos que cualquiera puede leer.

La distinción tiene consecuencias operativas. Un sistema sin tarjeta puede ser correcto, pero su corrección es invisible: nadie puede distinguir entre «funciona porque los supuestos se sostienen» y «funciona porque nadie ha mirado». Un sistema con tarjeta puede estar mal, pero su error es **auditable**: alguien puede leer los supuestos, ver cuál se rompió y saber qué corregir. La diferencia entre los dos mundos no es la calidad del sistema sino la **velocidad de corrección** cuando algo falla.

La tarjeta también cambia la conversación entre quien construye el sistema y quien lo usa. Sin tarjeta, la conversación es «confía en mí, funciona». Con tarjeta, la conversación es «estos son los supuestos que sostienen el número, aquí están sus pruebas y sus umbrales, y aquí está qué pasaría si se rompen». La segunda conversación es incómoda — obliga a declarar lo que no se sabe — pero es la única que escala más allá del equipo que construyó el sistema.

15.8 Qué queda fuera del alcance

Este capítulo cierra el libro, y lo cierra declarando lo que el sistema **no** hace. La traza end-to-end garantiza que cada afirmación causal del sistema es computable, registrada y vigente. No garantiza que la afirmación sea **correcta** — la corrección depende de que los supuestos declarados sean verdaderos, y eso lo decide el dominio, no la ingeniería.

Tres límites que el libro deja explícitos:

1. **La confusión no observada no tiene señal.** El monitor del Capítulo 14 vigila cuatro señales observables; ninguna detecta una causa no medida que afecte al tratamiento y al resultado. La tarjeta lo declara como supuesto no observable con su sensibilidad, pero la sensibilidad es una cota, no una prueba.
2. **La validez externa no se computa.** La tarjeta declara el soporte de las covariables, pero no puede declarar que la población futura se parecerá a la de entrenamiento. Eso lo vigila el monitor con deriva de covariables, pero la deriva no prueba que la función causal sea la misma.
3. **El estimando correcto es una decisión, no un cálculo.** Todo el aparato de este libro entrega la mejor estimación posible del estimando declarado. Si el estimando declarado no sirve a la decisión que el sistema apoya, ningún rigor estadístico lo arregla.

Estos límites no son defectos del sistema: son las fronteras honestas de lo que la ingeniería de datos puede garantizar. Lo que este libro ha construido es la condición necesaria — el pipeline que preserva la identificación — para que la decisión causal sea defendible. La condición suficiente sigue siendo humana: el dominio que declara los supuestos, la auditoría que los verifica y el criterio que decide cuándo dejar de creer en el número.

15.9 Ejercicios y notas bibliográficas

15.9.1 Ejercicios conceptuales

1. Una tarjeta llega sin supuestos no observables. ¿Por qué el contrato la rechaza, y qué parecería estar prometiendo esa tarjeta? *Pista:* sin la clase no observable declarada, la tarjeta sugiere que todo supuesto es verificable — y eso es falso por construcción.
2. El servicio lee la tarjeta para responder. ¿Qué tres campos de la tarjeta usa en cada consulta, y qué pasaría si faltara cada uno? *Pista:* el efecto y su IC (la respuesta), el soporte (la abstención) y la versión (la trazabilidad).
3. Su equipo reentrena con datos nuevos y la puerta produce alerta. Describa los artefactos que quedan en el registro y qué decisión humana falta. *Pista:* la versión nueva con su manifest y su estado alerta; falta la aceptación explícita registrada antes de promover.

15.9.2 Notas bibliográficas

La tarjeta causal extiende la model card de (Mitchell et al. 2019) con la clasificación de supuestos del Capítulo 2 de este libro y los umbrales de la puerta del Capítulo 12. El registro con hash y alias champion es la pieza del Capítulo 4 (C-09). La corrida end-to-end que cierra el capítulo es la demostración que la guía del proyecto final del curso exige: escenario sano se registra y promueve, positividad bloquea, cambio de identificación alerta.

Qué debe ser verdad en los datos

Que toda afirmación causal del sistema llegue con su tarjeta válida — estimando, supuestos clasificados, umbrales, limitaciones y uso previsto — registrada con hash y semilla, promovida por una puerta que dejó su decisión con motivo, y servida por un servicio que responde dentro del soporte y se abstiene fuera. Se comprueba con la semilla 20260715: la traza completa de nueve pasos sale en verde y cada artefacto es verificable sin leer código. Lo que la traza no certifica: que los supuestos declarados sean verdaderos — eso lo decide el dominio, la auditoría humana y el tiempo. Lo que sí certifica: que nadie puede cambiar el sujeto del enunciado sin que el registro lo detecte.

APÉNDICE A

Índice temático

Los términos se listan por parte, con el capítulo donde se introducen o desarrollan. Un término puede aparecer en varios capítulos: se anota el principal.

Término	Cap.	Término	Cap.
abstención	2, 13	aleatorización	1
alias champion	4, 12	AIPW	9
as-of join	6	ATE / ATT / CATE	1
auditoría end-to-end	15	benchmark como producto	8
bundle	4, 13	calibración de umbrales	7, 14
comparación de poblaciones	7	contraste	1
contrato C-01 (roles)	3	contrato C-02 (poblaciones)	7
contrato C-03 (temporal)	6	contrato C-04 (servicio)	13
contrato C-05 (tarjeta)	15	contrato C-06 (puerta)	12
contrato C-07 (monitor)	14	contrato C-08 (sensibilidad)	11
contrato C-09 (linaje)	4	contrato C-10 (partición)	5
contrato C-11 (benchmark)	8	contrato C-12 (estimador)	9
control negativo	11	covariable post-tratamiento	6
cross-fitting	9	deriva	7, 14
doble robustez	9	DGP	8
estimando declarado	1, 15	ESS de Kish	7, 9
fuga temporal	6	heterogeneidad	10
holdout permanente	2, 5	identificación	1, 12
inmutabilidad	4	instrumento (fuerza)	14
invariantes	3	linaje	4
manifest con hash	4, 15	mecanismo de asignación	2, 14
monitoreo causal	14	no confusión condicional	1, 15
pared observado/evaluación	5, 8	partición sembrada	5
placebo por permutación	11	plano de datos	2

Término	Cap.	Término	Cap.
plano de método	2, 9	política (umbral)	10
prueba de aceptación	3	propensity score	9, 14
puerta causal	12	Qini	10
reversión de Simpson	1	sensibilidad Rosenbaum	11
SHA-256	4	soporte común	9, 13
splits con semilla	5	SMD	7
supuestos no observables	15	tarjeta causal	15
trimming de Crump	9	validez temporal	6
valor de política IPW	10	ventanas de producción	12, 14
verdad reservada	5, 8		

APÉNDICE B

Índice de autores

Autor	Caps.	Contribución citada
Angrist, J.	1	Diseños IV y LATE
Bareinboim, E.	1	IA causal y PCH
Breck, C.	2	ML Test Score
Card, D.	1, 9	Retornos a la educación con IV
Chernozhukov, V.	9	DML y ortogonalidad
Crump, R. K.	9	Trimming y varianza asintótica
Dehejia, R.	1, 7	NSW y controles observacionales
Hernán, M. A.	1	What if (inferencia causal aplicada)
Hill, J.	8	IHDP y BART
Holland, P. W.	1	Problema fundamental
Huyen, C.	2, 7	Designing ML Systems
Imbens, G.	1	Causal Inference for Statistics
Kastner, D.	14	ML in Production
Kish, L.	9	Tamaño efectivo de muestra
Kleppmann, M.	3, 4	Designing Data-Intensive Applications
Künzel, P.	10	Meta-learners y cross-fitting
LaLonde, R.	1, 7	NSW benchmark
Lipsitch, M.	11	Control negativo
Mitchell, M.	15	Model cards
Neyman, J.	1	Resultados potenciales
Paleyes, A.	2	Challenges in ML deployment
Pearl, J.	1	Causalidad y PCH
Perdomo, J.	2	Predicción performativa
Reis, J.	3	Fundamentals of Data Engineering
Rosenbaum, P.	11	Sensibilidad y Gamma
Rubin, D. B.	1, 7	Resultados potenciales, balance

Autor	Caps.	Contribución citada
Sculley, C.	2	Hidden technical debt in ML
Wager, S.	10	Causal forests

APÉNDICE C

Erratas

Esta página se actualiza con cada reimpresión. En la edición 1.0-rc no hay erratas conocidas. Si encuentra una, repórtela al repositorio de referencia del libro con el número de página y el texto exacto.

Pág.	Dice	Debe decir	Motivo

Bibliografía

Athey, Susan, y Guido Imbens. 2016. «Recursive Partitioning for Heterogeneous Causal Effects». *Proceedings of the National Academy of Sciences* 113 (27): 7353-60.

Athey, Susan, y Stefan Wager. 2021. «Policy Learning With Observational Data». *Econometrica* 89 (1): 133-61.

Bareinboim, Elias. 2025. *Causal Artificial Intelligence: A Roadmap for Building Causally Intelligent Systems*. Columbia CausalAI Lab.

Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, y D. Sculley. 2017. «The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction». *IEEE International Conference on Big Data*,.

Card, David. 1995. «Using Geographic Variation in College Proximity to Estimate the Return to Schooling». *Aspects of Labour Economics*, 201-22.

Chernozhukov, Victor, Denis Chetverikov, Mert Demirer, et al. 2018. «Double/Debiased Machine Learning for Treatment and Structural Parameters». *The Econometrics Journal* 21 (1): C1-68.

Crump, Richard K., V. Joseph Hotz, Guido W. Imbens, y Oscar A. Mitnik. 2009. «Dealing with Small Sample Sizes Complicating Causal Effect Estimation». *Biometrika* 96 (2): 257-79.

Dehejia, Rajeev H., y Sadek Wahba. 1999. «Causal Effects in Nonexperimental Studies: Reevaluating the Evaluation of Training Programs». *Journal of the American Statistical Association* 94 : 1053-62.

Hernán, Miguel A., y James M. Robins. 2020. *Causal Inference: What If*. Boca Raton: Chapman & Hall/CRC.

Hill, Jennifer. 2011. «Bayesian Nonparametric Modeling for Causal Inference». *Journal of Computational and Graphical Statistics* 20 (1): 217-40.

Holland, Paul W. 1986. «Statistics and Causal Inference». *Journal of the American Statistical Association* 81 (396): 945-60.

Huyen, Chip. 2022. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O'Reilly Media.

Imbens, Guido W., y Donald B. Rubin. 2015. *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge University Press.

Kästner, Christian, Eunsuk Kang, y Michael Hilton. 2024. «Machine Learning in Production: From Models to Products». https://mlip-cmu.github.io/book/.

Kish, Leslie. 1965. *Survey Sampling*. Wiley.

Kleppmann, Martin. 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Künzel, Sören R., Jasjeet S. Sekhon, Peter J. Bickel, y Bin Yu. 2019. «Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning». *Proceedings of the National Academy of Sciences* 116 (10): 4156-65.

LaLonde, Robert J. 1986. «Evaluating the Econometric Evaluations of Training Programs with Experimental Data». *American Economic Review* 76 : 604-20.

Lipsitch, Marc, Eric Tchetgen Tchetgen, y Ted Cohen. 2010. «Negative Controls: A Tool for Detecting Confounding and Bias in Observational Studies». *Epidemiology* 21 (3): 383-88.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, et al. 2019. «Model Cards for Model Reporting». *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 220-29.

Neyman, Jerzy. 1923. «On the Application of Probability Theory to Agricultural Experiments». *Annals of Agricultural Sciences* 5 : 1-51.

Paleyes, Andrei, Raoul-Gabriel Urma, y Neil D. Lawrence. 2022. «Challenges in Deploying Machine Learning: A Survey of Case Studies». *ACM Computing Surveys* 55 (6): 1-29.

Pearl, Judea. 2009. *Causality: Models, Reasoning and Inference*. 2.ª ed. Cambridge University Press.

Pearl, Judea, y Dana Mackenzie. 2018. *The Book of Why. The New Science of Cause and Effect*. Basic Books.

Perdomo, Juan, Tijana Zrnic, Celestine Mendler-Dünner, y Moritz Hardt. 2020. «Performative Prediction». *Proceedings of the 37th International Conference on Machine Learning (ICML), PMLR 119*, 7599-609.

Reis, Joe, y Matt Housley. 2022. *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O'Reilly Media.

Rosenbaum, Paul R., y Donald B. Rubin. 1983. «The Central Role of the Propensity Score in Observational Studies for Causal Effects». *Biometrika* 70 (1): 41-55.

Rubin, Donald B. 1974. «Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies». *Journal of Educational Psychology* 66 (5): 688-701.

Rubin, Donald B. 1976. «Inference and Missing Data». *Biometrika* 63 (3): 581-92.

Rubin, Donald B. 2001. «Using Propensity Scores to Help Design Observational Studies: Application to the Tobacco Litigation». *Health Services and Outcomes Research Methodology* 2 : 169-88.

Sculley, D., Gary Holt, Daniel Golovin, et al. 2015. «Hidden Technical Debt in Machine Learning Systems». *Advances in Neural Information Processing Systems 28 (NeurIPS)*,.

Simpson, Edward H. 1951. «The Interpretation of Interaction in Contingency Tables». *Journal of the Royal Statistical Society, Series B* 13 (2): 238-41.

Stock, James H., y Motohiro Yogo. 2005. «Testing for Weak Instruments in Linear IV Regression». En *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, editado por Andrews, Donald W. K., Stock, y James H. Cambridge University Press.

Yule, G. Udny. 1903. «Notes on the Theory of Association of Attributes in Statistics». *Biometrika* 2 (2): 121-34.